Demystifying Artificial Intelligence Sorcery

(Part 1: Expert System, Fuzzy Logic & Neural Network)[a]

Abdelbacet Mhamdi
abdelbacet.mhamdi@bizerte.r-iset.tn

Dr.-Ing. in Electrical Engineering
Senior Lecturer at ISET Bizerte

[a]Available @ https://github.com/a-mhamdi/jlai/



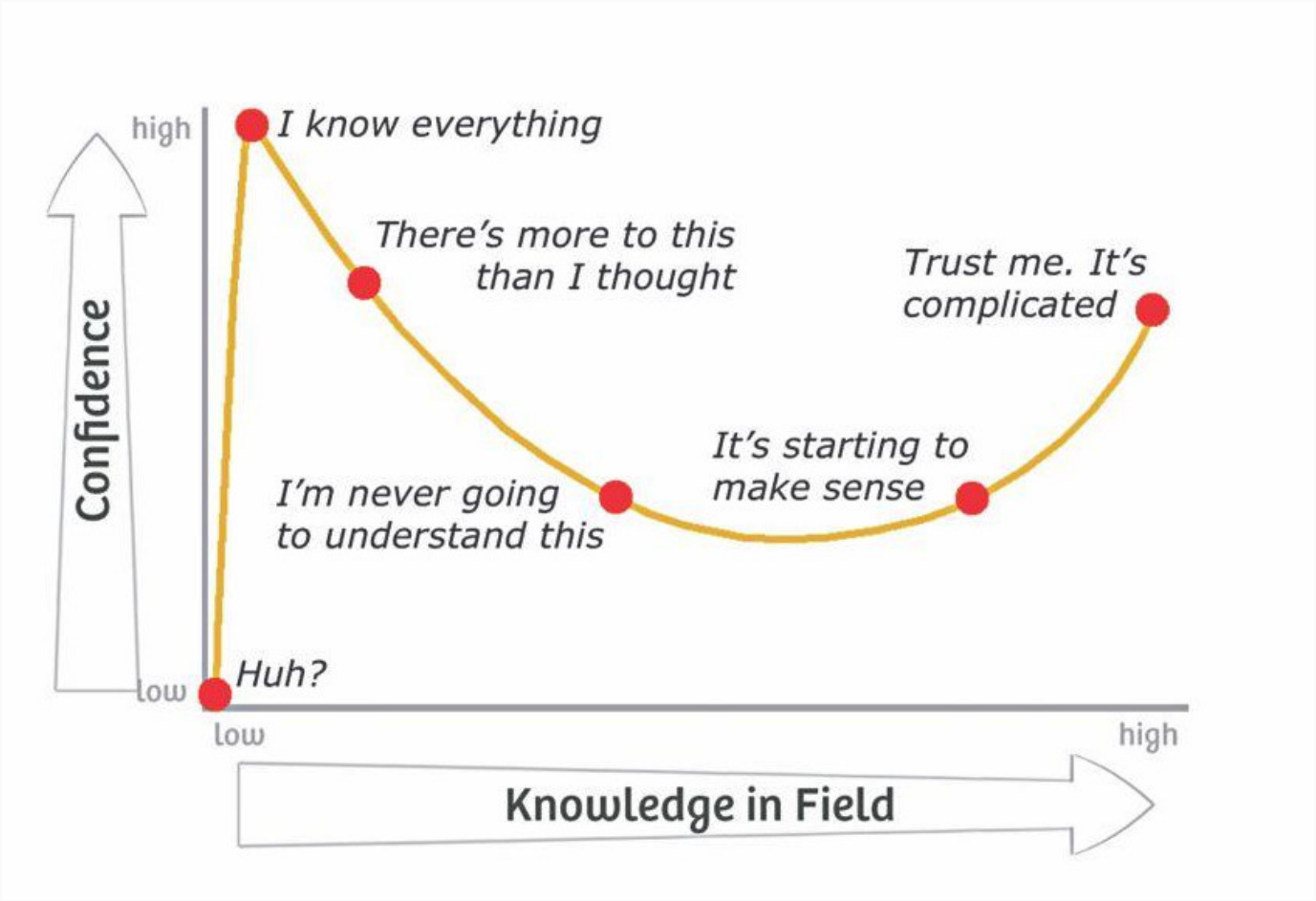
Disclaimer

This document features some materials gathered from multiple online sources.

Please note no copyright infringement is intended, and I do not own nor claim to own any of the original materials. They are used for educational purposes only.

I have included links solely as a convenience to the reader. Some links within these slides may lead to other websites, including those operated and maintained by third parties. The presence of such a link does not imply a responsibility for the linked site or an endorsement of the linked site, its operator, or its contents.

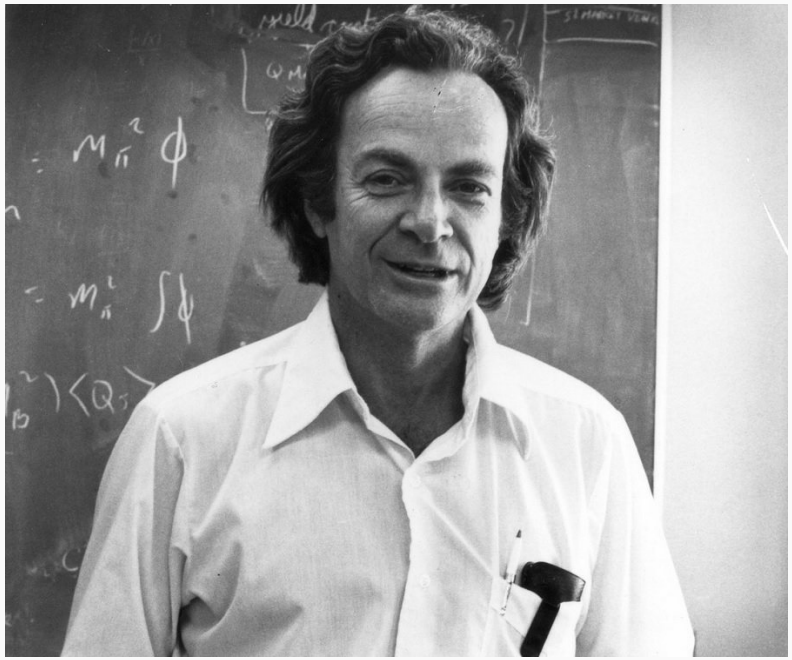
Dunning-Kruger Effect



Kruger, J. and Dunning, D. (1999) *Unskilled and unaware of it: How difficulties in recognizing one's own incompetence lead to inflated self-assessments.* **J Pers Soc Psychol.** 77(6) pp. 1121–1134. doi 10.1037/0022-3514.77.6.1121

"Knowledge isn't free. You have to pay attention."

Richard P. Feynman

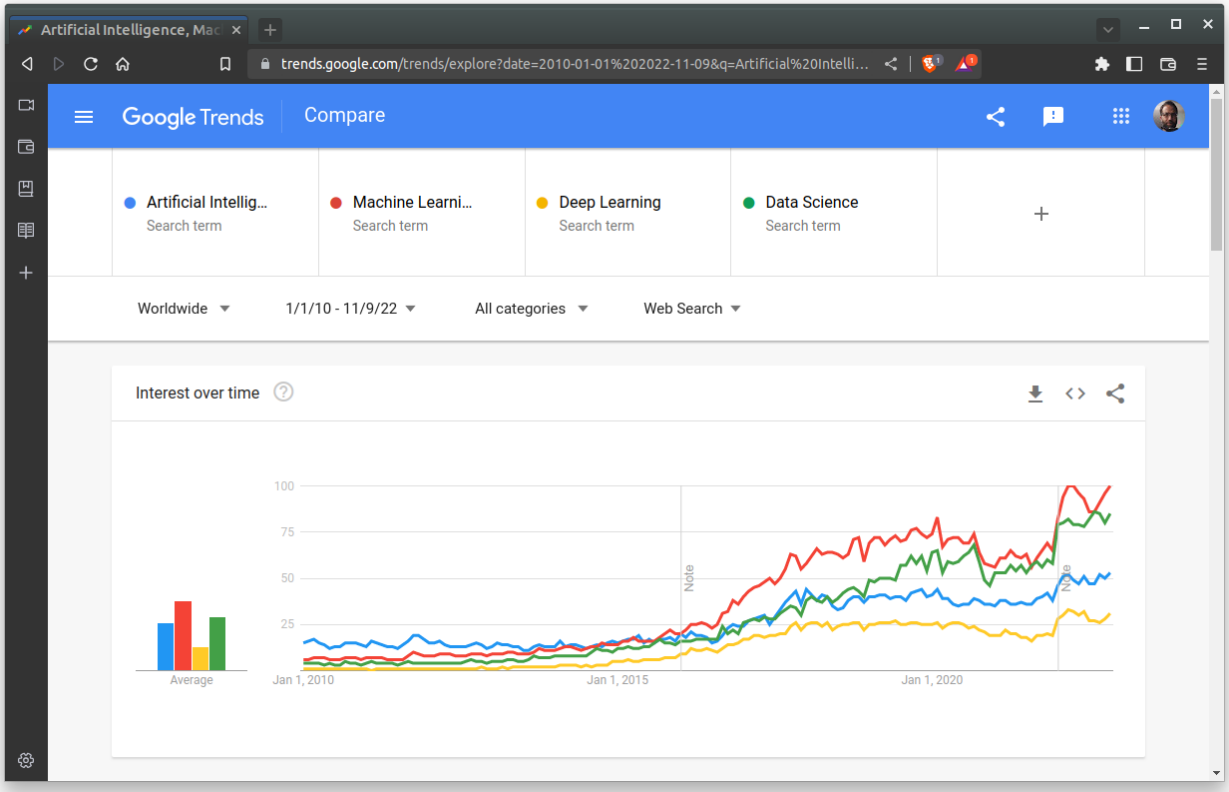


Roadmap

1. An overview
2. Expert System
3. Fuzzy Logic
4. Neural Networks
5. Quizzes

An overview

Trends



"Numbers represent search interest relative to the highest point on the chart for the given region and time.

- A value of 100 is the peak popularity for the term;
- A value of 50 means that the term is half as popular;
- A score of 0 means there was not enough data for this term."

Top Uses


YouTube




Gboard


Google
Translate




amazon








Hi, how can I help?


amazon
alexa


Hi, I'm Cortana.

Artificial intelligence is a branch of computer science which focuses on automation of intelligent behavior.

Some definitions can be categorized into four frames.

Artificial intelligence is a branch of computer science which focuses on automation of intelligent behavior.

Some definitions can be categorized into four frames.

Systems that think like humans

[Bel78]

> "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem-solving, learning..."

Bellman, R. E. *An Introduction to Artificial Intelligence: Can Computers Think?* **Boyd & Fraser Publishing Company.**

[Hau89]

> "The exciting new effort to make computers think[...] *machines with minds*, in the full and literal sense"

Haugeland, J. (1989). *Artificial Intelligence: The Very Idea*. **A Bradford book. MIT Press.**

Systems that think rationally

[CMM85]

> "The study of mental faculties through the use of computational models."

Charniak, E., McDermott, D., and McDermott, D. V. (1985). *Introduction to Artificial Intelligence*. **Addison-Wesley series in computer science and information processing. Addison-Wesley.**

[Win92]

> "The study of the computations that make it possible to perceive, reason, and act."

Winston, P. H. (1992). *Artificial Intelligence*. **A-W Series in Computer Science. Addison-Wesley Publishing Company.**

Systems that act like humans

[Kur92]

"The art of creating machines that perform functions that require intelligence when performed by people."

Kurzweil, R. (1992). *The Age of Intelligent Machines*. **Viking.**

[RK91]

"The study of how to make computers do things at which, at the moment, people are better."

Rich, E. and Knight, K. (1991). *Artificial Intelligence*. **Artificial Intelligence Series. McGraw-Hill.**

Systems that act rationally

[Sch90]

> "A field of study that seeks to explain and emulate intelligent behavior in terms of computational processes."

Schalkoff, R. J. (1990). *Artificial Intelligence: An Engineering Approach*. **McGraw-Hill Computer science series. McGraw-Hill.**

[LS93]

> "The branch of computer science that is concerned with the automation of intelligent behavior."

Luger, G. F. and Stubblefield, W. A. *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*. **Artificial intelligence. Benjamin/Cummings Publishing Company.**

Thought-Provoking Questions



How to achieve intelligence on a computer system

What do we mean by "Intelligence"?

- Single faculty or gathering of abilities
- Learned or existing
- What happens when we learn
- Are creativity and intuition measurable
- Does observable behavior infer to intelligence
- How knowledge is routed in the human brain

Thought-Provoking Questions



How to achieve intelligence on a computer system

What do we mean by "Intelligence"?

- Single faculty or gathering of abilities
- Learned or existing
- What happens when we learn
- Are creativity and intuition measurable
- Does observable behavior infer to intelligence
- How knowledge is routed in the human brain

Turing Test

Alan Turing (1950)

The ability to achieve human level performance in all cognitive tasks, sufficient to fool an interrogator.

- ✓ Natural Language Processing (NLP) *(Communicate in human language)*
- ✓ Knowledge Representation *(Store information)*
- ✓ Automated Reasoning *(Answer questions & draw conclusions)*
- ✓ Machine Learning (ML) *(Adapt to new circumstances, detect & extrapolate patterns)*

Forms of AI

- ☆ Expert Systems *(Based on knowledge or rule settings)*
- ☆ Fuzzy Systems *(Based on fuzzy set theory)*
- ☆ Artificial Neural Networks
- ☆ Genetic Algorithms
- ☆ Belief Networks
- ☆ Hybrid Systems *(Combine two or more approaches)*

Programming Language



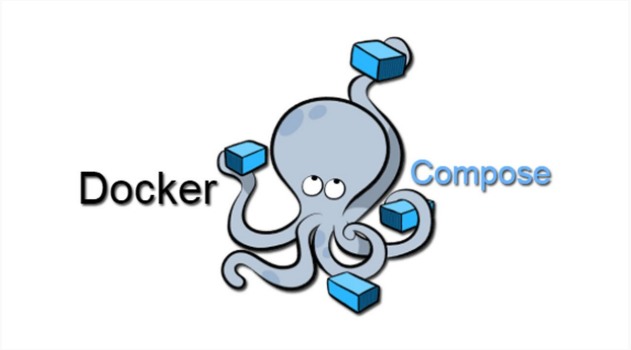
julialang.org/



Development Environments







- ▲ $ docker compose up
- ▼ $ docker compose down





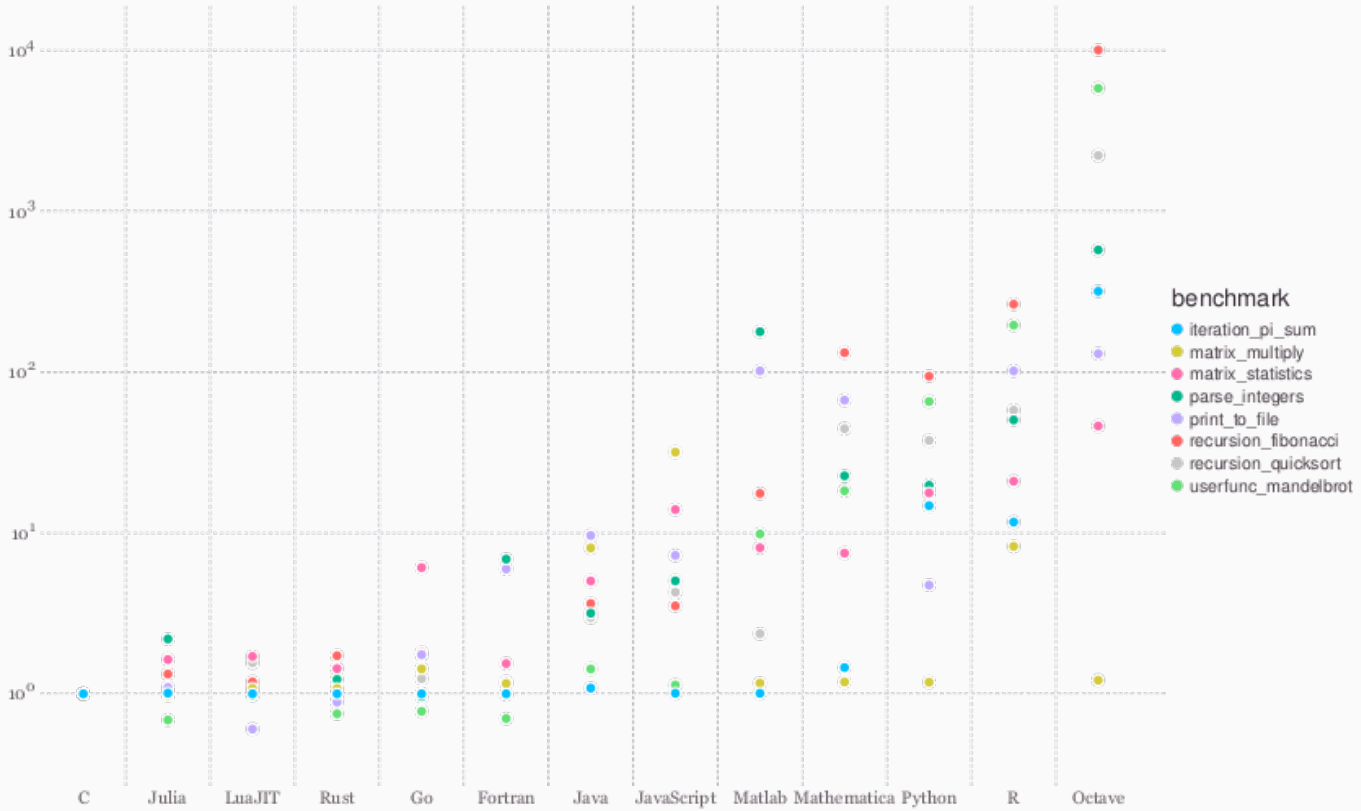


Julia in a Nutshell

- **Fast:** native code for multiple platforms via LLVM;
- **Dynamic:** good support for interactive use *(like a a scripting language)*;
- **Reproducible:** environment recreation across platforms, with pre-built binaries;
- **Composable:** multiple dispatch as a paradigm *(oop & functional programming)*;
- **General:** asynchronous I/O, metaprogramming, debugging, logging; profiling, pkg, ...
- **Open Source:** GitHub repository at https://github.com/JuliaLang/julia.



Julia Micro-Benchmarks (1/2)



https://julialang.org/benchmarks

Julia Micro-Benchmarks (2/2)

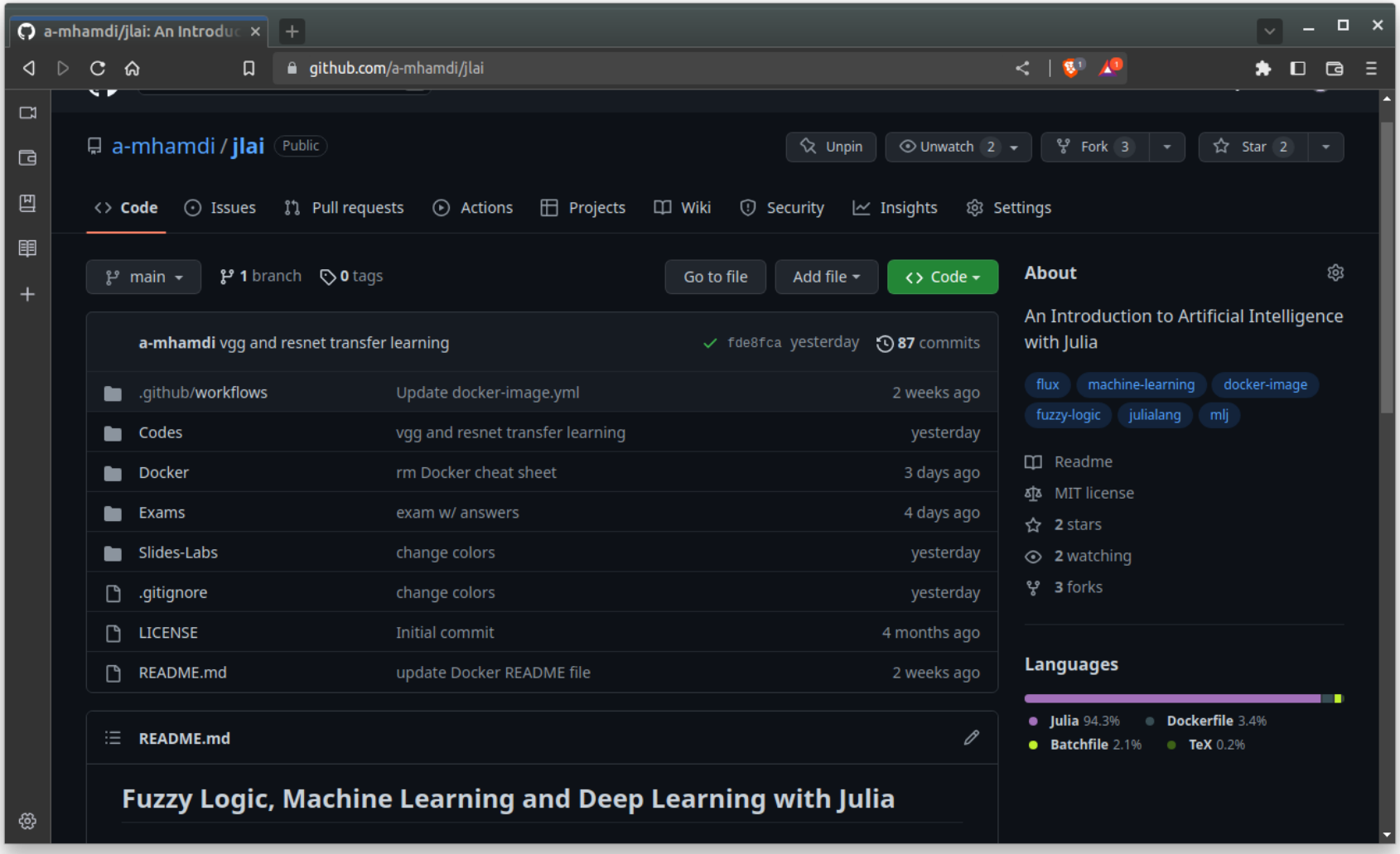
Geometric Means[1] of Micro-Benchmarks by Language

1	C	1.0
2	Julia	1.17006
3	LuaJIT	1.02931
4	Rust	1.0999
5	Go	1.49917
6	Fortran	1.67022
7	Java	3.46773
8	JavaScript	4.79602
9	Matlab	9.57235
10	Mathematica	14.6387
11	Python	16.9262
12	R	48.5796
13	Octave	338.704



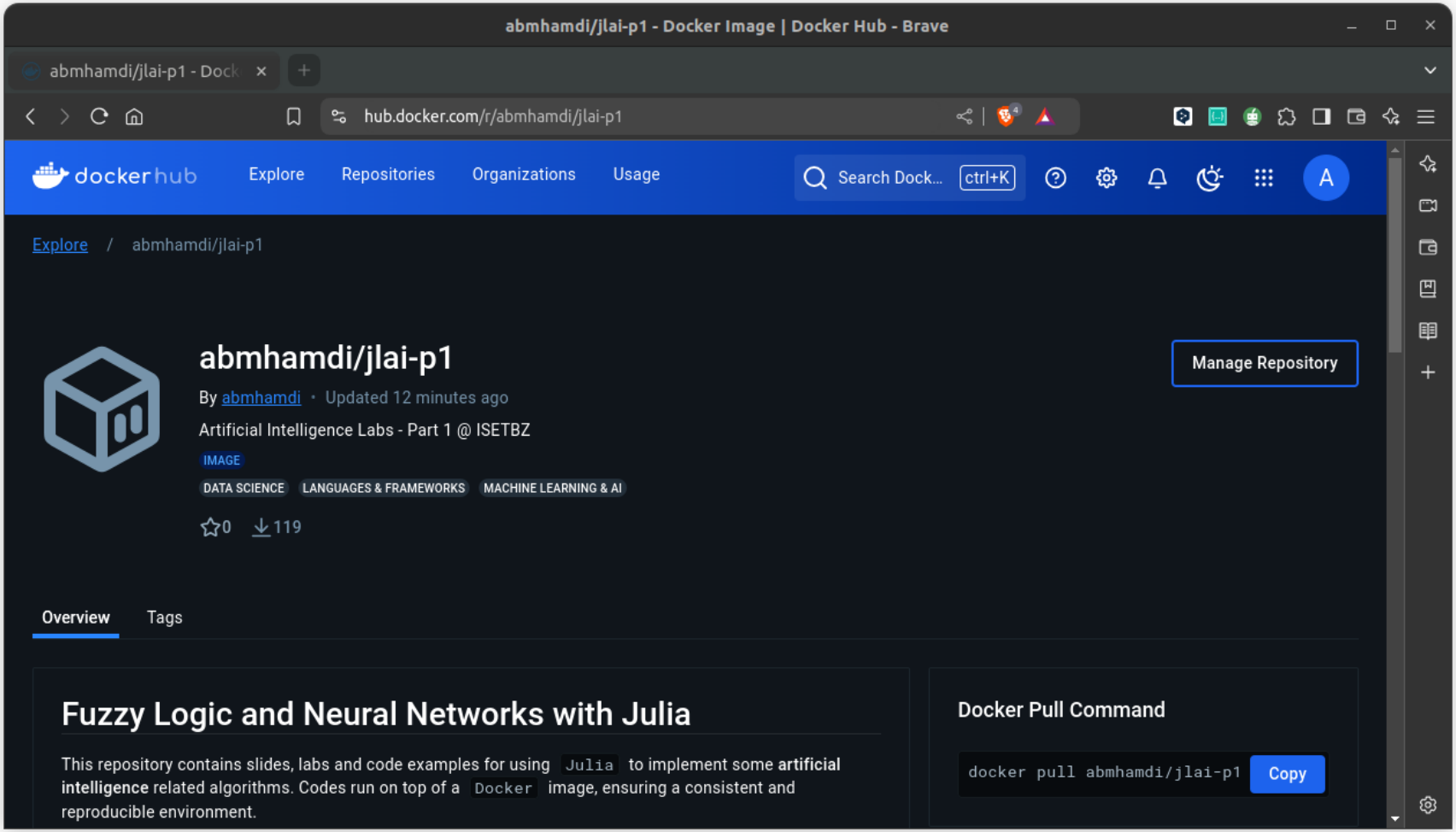
[1] Measure of central tendency expressed as $\left(x_1 \times x_2 \times \cdots \times x_n\right)^{1/n}$

Source Control Management (SCM)



https://github.com/a-mhamdi/jlai

Docker Image



https://hub.docker.com/r/abmhamdi/jlai-p1

Expert System

What is an Expert System?

An **expert system** is a computer program that emulates the decision-making ability of a human expert in a specific domain.

- Part of the broader field of **Artificial Intelligence**
- Encodes specialized knowledge *(rules, facts, heuristics)* rather than general-purpose algorithms
- Designed to **reason** over that knowledge to solve problems, diagnose issues, or give recommendations

Capture human expertise in a form a machine can apply consistently.

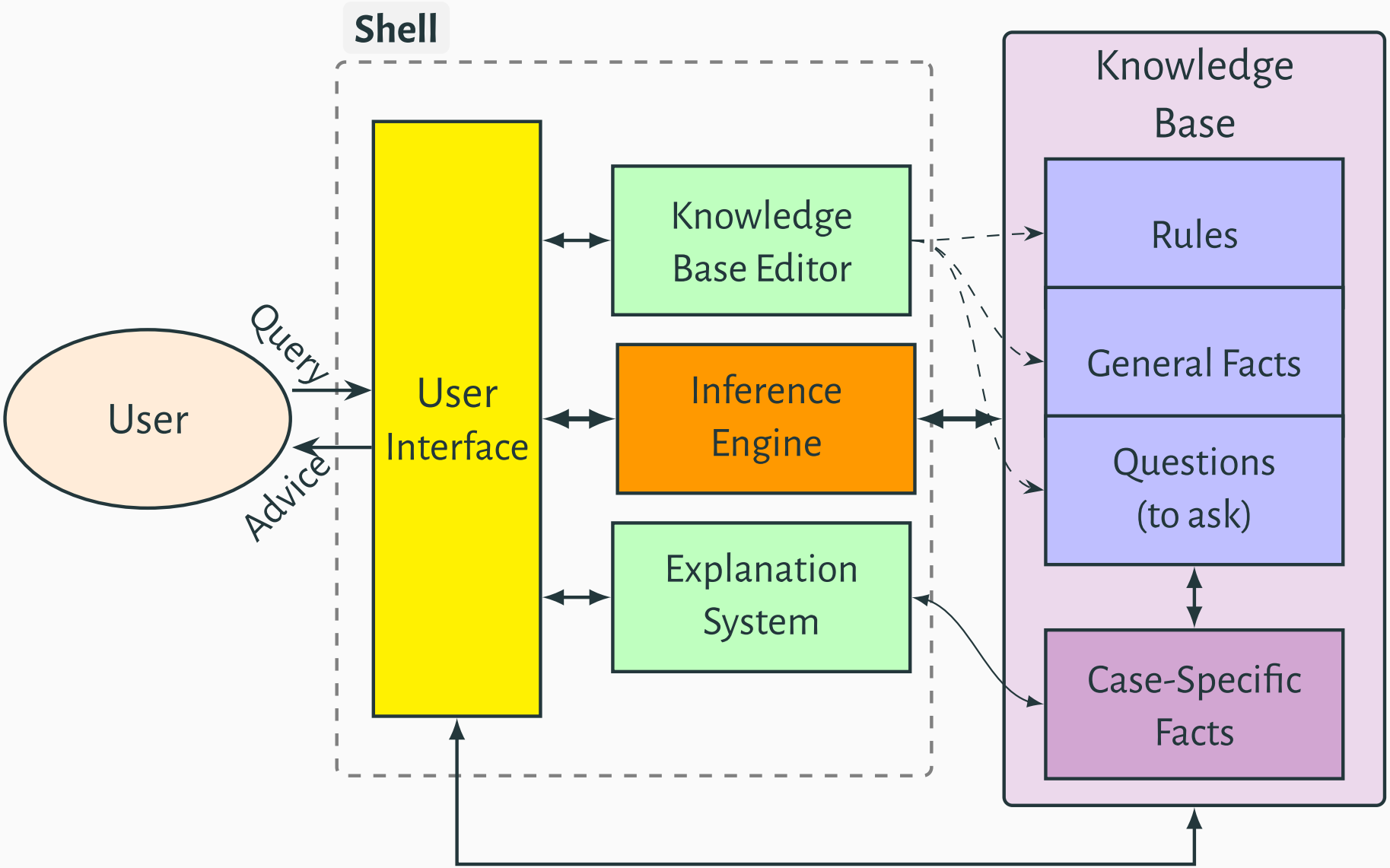
A Brief History

- **1960s–70s**: Early AI research shifts from general problem solvers to domain-specific knowledge
- **DENDRAL** *(Stanford, 1965)*: inferred molecular structure from mass spectrometry data; one of the first true expert systems
- **MYCIN** *(Stanford, 1970s)*: diagnosed bacterial infections and recommended antibiotics; introduced backward chaining and certainty factors
- **1980s**: Commercial boom, expert systems adopted widely in industry *(finance, manufacturing, engineering)*
- **Today**: Concepts persist inside modern rule engines, decision support systems, and hybrid AI *(e.g., neuro-symbolic systems)*

Architecture of an Expert System

Component	Role
Knowledge Base	Domain facts and rules, authored by knowledge engineers with domain experts
Working Memory	Facts known/derived during a specific consultation
Inference Engine	Reasoning mechanism that applies rules to facts *(forward/backward chaining)*
User Interface	Interaction layer, questions to the user, explanations of conclusions
Explanation Facility	Justifies *why* a conclusion was reached *(traces the rule chain)*

Schematic



★ *Adapted from lecture slides by Urbanowicz, URBS Lab*

Key Characteristics

- **Separation of knowledge and reasoning**: knowledge base ≠ inference engine. We can update one without rewriting the other
- **Explainability**: can justify conclusions by tracing the chain of rules applied *(unlike a "black box")*
- **Narrow, deep expertise**: performs well in a specific domain, poorly outside it
- **Handles uncertainty**: via certainty factors, probabilities, or, as we'll see later, fuzzy logic

Full Picture

Knowledge Representation → Reasoning Strategies → Handling Uncertainty

1. What are **rules**, and how do they encode knowledge?
2. **Forward chaining**: reasoning from facts to conclusions
3. **Backward chaining**: reasoning from goals to facts
4. **Conflict resolution**: choosing among competing rules
5. **Fuzzy logic**: reasoning with vagueness and degree

Rules: The Building Blocks of Expert Systems

Production Rule

A rule is a conditional statement of the form:

$$\text{IF } \langle \texttt{condition} \rangle \text{ THEN } \langle \texttt{conclusion / action} \rangle$$

- **Condition (IF-part / antecedent)**: a pattern to be matched against known facts
- **Conclusion (THEN-part / consequent)**: a new fact to assert, or an action to perform
- Rules encode expert knowledge as **if-this-then-that** heuristics

Example

IF `has_feathers(X)` THEN `bird(X)`

Propositional Logic

- Propositional logic reasons about **whole statements** that are either true or false.
- Atoms are indivisible propositions: P, Q, R . . .
- Connectives combine them: $\wedge$ (and), $\vee$ (or), $\neg$ (not), $\longrightarrow$ (implies), $\Leftrightarrow$ (iff).
- **Example:** P = "It rains", Q = "The ground is wet" $P \longrightarrow Q$
- **Limitation:** Propositional logic cannot talk about *objects*, their *properties*, or *relations* between them. Each fact is a separate, unrelated atom. It cannot express "every x that is a bird can fly" in one statement.

Logical Symbols

In formal logic $\neg$ is NOT, $\vee$ is OR and $\wedge$ is AND

Consider the following propositions $\mathcal{E}$ and $\mathcal{S}$

$\mathcal{E}$ "The earth is round"

$\mathcal{S}$ "The sun spins on its axis"

Thus

$\neg\mathcal{E}$ "The earth is not round"

$\neg\mathcal{S}$ "The sun does not spin on its axis"

$\mathcal{E} \vee \mathcal{S}$ "The earth is round **or** the sun spins on its axis"

$\mathcal{E} \wedge \mathcal{S}$ "The earth is round **and** the sun spins on its axis"

$\neg\mathcal{E} \vee \mathcal{S}$ "The earth is not round **or** the sun spins on its axis"

$\mathcal{E} \wedge \neg\mathcal{S}$ "The earth is round **and** the sun does not spin on its axis"

Why First-Order Logic?

- First-order logic (FOL), a.k.a. **predicate logic**, extends propositional logic with:
  - objects (individuals) we can name and quantify over,
  - properties of objects and relations between them,
  - variables and quantifiers to generalize over objects.
- Instead of one flat proposition per fact, we get **one general rule** that applies to many objects.

Example: Propositional logic needs a separate atom for every bird; FOL needs one rule:

$$\forall x \left(\texttt{Bird}(x) \rightarrow \texttt{CanFly}(x)\right)$$

Subjects (Terms)

A **term** refers to an object in the domain of discourse. Three kinds:

- **Constants**: name a specific, fixed object.
  e.g. `maria`, `socrates`, `a`
- **Variables**: placeholders standing for an unspecified object, later bound by a quantifier.
  e.g. x, y, z
- **Functions**: map object(s) to another object (not true/false!).
  e.g. `Mother_of(`x`)`, `Sum(`x`, `y`)`

Terms are the "subjects" logic talks about. They are objects, never statements.

Predicates

A **predicate** expresses a property of, or a relation between, terms. Applying a predicate to terms yields a statement that is true or false.

- Written as $\texttt{Name}(\texttt{term}_1, \ldots, \texttt{term}_n)$.
- The number of terms it takes is its **arity**.
- A predicate is *not* itself true or false until its arguments (terms) are filled in.

Example: $\texttt{Student}(x)$ is a predicate; $\texttt{Student}(\texttt{maria})$ is a statement (true or false).

Predicate Arity: Unary, Binary, n-ary

- **Unary predicate** (1 argument): a property of a single object.
  e.g. `Student(x)`, `is_mammal(x)`, `Red(x)`
- **Binary predicate** (2 arguments): a relation between two objects.
  e.g. `Loves(x,y)`, `Parent(x,y)`, `Teaches(x,y)`
- **Ternary / n-ary predicate** (n arguments): a relation among n objects.
  e.g. `Between(x,y,z)`, `Gives(x,y,z)` (x gives y to z)

Arity is fixed per predicate: `Teaches` always takes exactly 2 arguments in a given knowledge base.

Quantifiers

Quantifiers bind variables and state *how many* objects a formula applies to.

Universal quantifier $\forall x$ – "for all x"

$$\forall x \left(\texttt{Bird}(x) \longrightarrow \texttt{CanFly}(x)\right) \textit{ (every bird can fly)}$$

Existential quantifier $\exists x$ – "there exists an x"

$$\exists x \left(\texttt{Dog}(x) \land \texttt{Brown}(x)\right) \textit{ (some dog is brown)}$$

Key pattern:

- $\forall$ is usually paired with $\longrightarrow$ ("for all x, if ...then ...").
- $\exists$ is usually paired with $\land$ ("there is an x such that ...and ...").

Quantifier Scope and Nesting

Quantifiers can be nested; **order matters**.

- $\forall x \exists y \, \mathtt{Loves}(x, y)$
  everyone loves *someone* (possibly a different person each)
- $\exists y \forall x \, \mathtt{Loves}(x, y)$
  there is *one* person that everyone loves

These two formulas are **not** logically equivalent. Swapping $\forall$ and $\exists$ changes the meaning.

Putting It Together: An Example

English: "There is a student who has taken every course in the department."

Predicates: $\texttt{Student}(x), \texttt{Course}(y), \texttt{Taken}(x, y)$

FOL:

$$\exists x \left(\texttt{Student}(x) \land \forall y \left(\texttt{Course}(y) \rightarrow \texttt{Taken}(x, y) \right) \right)$$

Note the nesting: the $\exists$ is outside, the $\forall$ is inside its scope.

Well-Formed Formulas: Building Blocks

- **Terms**: constants, variables, functions applied to terms.
- **Atomic formula**: a predicate applied to terms, e.g. `Loves(john, maria)`.
- **Complex formulas**: built from atomic formulas using

$$\neg,\ \wedge,\ \vee,\ \rightarrow,\ \Leftrightarrow,\ \forall,\ \exists$$

- **Example:**

$$\forall x \left(\texttt{Teacher}(x) \rightarrow \exists y \left(\texttt{Student}(y) \wedge \texttt{Respects}(x, y) \right) \right)$$

Summary

- Propositional logic: whole statements, no internal structure.
- FOL adds: **terms** (constants, variables, functions) as subjects, and **predicates** as properties/relations over them.
- **Arity** classifies predicates: unary (1 arg), binary (2 args), n-ary.
- **Quantifiers** $\forall$ / $\exists$ generalize formulas over objects; their **order** and **scope** matter.
- Typical patterns: $\forall x\big(P(x) \rightarrow Q(x)\big)$ and $\exists x\big(P(x) \wedge Q(x)\big)$.

Task #1
Give a clear, natural English sentence for each formula.

1. $\forall x \left(\texttt{Man}(x) \longrightarrow \texttt{Mortal}(x)\right)$
2. $\exists x \left(\texttt{Teacher}(x) \land \forall y \left(\texttt{Student}(y) \longrightarrow \texttt{Teaches}(x, y)\right)\right)$
3. $\neg \exists x \left(\texttt{Reptile}(x) \land \texttt{HasFur}(x)\right)$

Translate FOL to English

1. Every man is mortal. *(Equivalently: "All men are mortal.")*
2. There is a teacher who teaches every student.
3. There is no reptile that has fur. *(Equivalently: "No reptiles have fur.")*

Task #2
For each sentence, give the FOL formula.

1. Every cat is an animal.
2. Some numbers are prime.
3. No fish can walk.
4. Only students can enter the library.
   (Hint: "only x can y" means "if y then x", not the other way around.)
5. Every professor has taught at least one course.
6. There is exactly one director of the company.
   (Hint: you need both existence and uniqueness.)
7. Not all birds can fly.
8. Anyone who studies hard will pass.

Translate English to FOL

1. $\forall x \left(\texttt{Cat}(x) \longrightarrow \texttt{Animal}(x)\right)$
2. $\exists x \left(\texttt{Number}(x) \land \texttt{Prime}(x)\right)$
3. $\forall x \left(\texttt{Fish}(x) \longrightarrow \neg\texttt{CanWalk}(x)\right)$ *equivalently:* $\neg\exists x \left(\texttt{Fish}(x) \land \texttt{CanWalk}(x)\right)$
4. $\forall x \left(\texttt{Enters}(x) \longrightarrow \texttt{Student}(x)\right)$
5. $\forall x \left(\texttt{Professor}(x) \longrightarrow \exists y \left(\texttt{Course}(y) \land \texttt{Taught}(x, y)\right)\right)$
6. $\exists x \left(\texttt{Director}(x) \land \forall y \left(\texttt{Director}(y) \longrightarrow y = x\right)\right)$
7. $\neg\forall x \left(\texttt{Bird}(x) \longrightarrow \texttt{CanFly}(x)\right)$ *equivalently:* $\exists x \left(\texttt{Bird}(x) \land \neg\texttt{CanFly}(x)\right)$
8. $\forall x \left(\texttt{StudiesHard}(x) \longrightarrow \texttt{Passes}(x)\right)$

Task #3

Push the negation inward as far as possible *(equivalent formula, no leading ¬ in front of a quantifier)* for each of the following.

1. $\neg \forall x\, P(x)$
2. $\neg \exists x\, P(x)$
3. $\neg \forall x \left(P(x) \longrightarrow Q(x)\right)$
4. $\neg \exists x\, \forall y\, \mathtt{Loves}(x, y)$

Negating Quantified Formulas

Negating a quantifier flips it to the other quantifier and moves the negation onto the inner formula *(FOL analogue of De Morgan's laws)*.

1. $\neg\forall x\, P(x) \equiv \exists x\, \neg P(x)$
2. $\neg\exists x\, P(x) \equiv \forall x\, \neg P(x)$
3. $\neg\forall x\, \big(P(x) \longrightarrow Q(x)\big)$

$$\equiv \exists x\, \neg\big(P(x) \longrightarrow Q(x)\big)$$
$$\equiv \exists x\, \big(P(x) \wedge \neg Q(x)\big) \qquad (\texttt{since } A \longrightarrow B \equiv \neg A \vee B, \text{so } \neg(A \longrightarrow B) \equiv A \wedge \neg B)$$

4. $\neg\exists x\, \forall y\, \texttt{Loves}(x, y)$

$$\equiv \forall x\, \neg\forall y\, \texttt{Loves}(x, y)$$
$$\equiv \forall x\, \exists y\, \neg\texttt{Loves}(x, y)$$

Task #4

Consider the two formulas below, interpreted over the domain of people, with $\mathtt{Loves}(x, y)$ meaning "x Loves y":

(i) $\forall x \exists y\ \mathtt{Loves}(x, y)$ (ii) $\exists y \forall x\ \mathtt{Loves}(x, y)$

1. Give an English reading of each formula.
2. Are (i) and (ii) logically equivalent?

Quantifier Order

D1. English readings.

- (i) $\forall x\, \exists y\, \texttt{Loves}(x, y)$: "Everyone loves someone" *Each person has at least one person they love, but different people may love different people.*
- (ii) $\exists y\, \forall x\, \texttt{Loves}(x, y)$: "There is someone whom everyone loves" *One single person is loved by everybody.*

D2. Are they equivalent?

- No. (ii) $\longrightarrow$ (i) always holds *(if everyone loves that one special person, then in particular everyone loves someone)*, but the converse does not hold in general.
  So (i) is strictly weaker than (ii); they are not logically equivalent.

Task #5

Translate to FOL *(Advanced)*

1. Every student who studies for every exam will pass all of them.
2. There is a barber who shaves everyone who does not shave themselves.
3. For every number, there is a larger number, but no number is larger than itself.
4. Every child has exactly two biological parents.
5. Someone likes everyone who likes them back.
6. No one can fool all of the people all of the time, but everyone can fool some of the people some of the time.
7. A course is a prerequisite for another course only if it must be completed before it.
8. If a student passes every course required for a major, then they graduate with that major, provided no course was skipped.
9. Every even number greater than 2 can be written as the sum of two primes.
10. There is at most one professor who teaches every course in the department.
11. Every non-empty set of natural numbers has a least element.
12. Two people are siblings if and only if they share at least one parent and are not the same person.

1. $\forall x\big(Sx \land \forall y(Ey \rightarrow Dxy) \;\rightarrow\; \forall y(Ey \rightarrow Pxy)\big)$
2. $\exists x\big(Bx \land \forall y(\neg Syy \rightarrow Sxy)\big)$
3. $\forall x\Big(Nx \rightarrow \exists y\big((Ny \land Gyx) \;\land\; \neg Gxx\big)\Big)$
4. $\forall x\Big(Cx \rightarrow \exists y\,\exists z\big(Pyx \land Pzx \land y \neq z \land \forall w(Pwx \rightarrow (w = y \lor w = z))\big)\Big)$
5. $\exists x\Big(Px \land \forall y\big((Py \land Lyx \land x \neq y) \rightarrow Lxy\big)\Big)$
6. $\neg\exists x\Big(Px \land \forall y\,\forall t\big((Py \land Tt) \rightarrow Fxyt\big)\Big) \;\land\; \forall x\,\Big(Px \rightarrow \exists y\,\exists t\big(Py \land Tt \land Fxyt\big)\Big)$
7. $\forall x\,\forall y\big((Cx \land Cy \land x \neq y \land Pxy) \rightarrow Bxy\big)$
8. $\forall x\,\forall m\Big(\big(Sx \land \forall y\big((Cy \land Rym) \rightarrow Pxy\big) \land \neg\exists y(Cy \land Kxy)\big) \rightarrow Gxm\Big)$
9. $\forall x\big((Nx \land Ex \land x > 2) \rightarrow \exists y\,\exists z(Ny \land Py \land Nz \land Pz \land x = y + z)\big)$
10. $\forall x\,\forall z\Big(\big[\big(Px \land \forall y(Cy \rightarrow Txy)\big) \land \big(Pz \land \forall y(Cy \rightarrow Tzy)\big)\big] \rightarrow x = z\Big)$
11. $\forall x\Big((Sx \land Nx) \rightarrow \exists y\big[Mxy \land \forall z\big(Mxz \rightarrow (y = z \lor Lyz)\big)\big]\Big)$
12. $\forall x\,\forall y\Big(Sxy \leftrightarrow \big(\exists z(Pzx \land Pzy) \land x \neq y\big)\Big)$

Why Normal Forms?

- Many reasoning algorithms (resolution, SAT solvers, rule engines) don't work on *arbitrary* formulas. They expect a fixed, uniform **syntactic shape**.
- A **normal form** is such a shape: any formula can be mechanically rewritten into an equivalent formula of that shape.
- **Conjunctive Normal Form (CNF)** is the shape required by:
  - the resolution inference rule,
  - most SAT solvers,
  - and it is the basis for **Horn clauses**, the rule format used by expert systems.

Building Blocks: Literal, Clause

- A **literal** is an atomic proposition or its negation:
  $P, \neg P, Q, \neg Q \ldots$
- A **clause** is a disjunction ($\vee$) of literals:
  $P \vee \neg Q \vee R$
- A formula is in **Conjunctive Normal Form** if it is a conjunction ($\wedge$) of clauses:

$$\underbrace{(P \vee \neg Q)}_{\text{clause}} \wedge \underbrace{(\neg P \vee R \vee S)}_{\text{clause}} \wedge \underbrace{(Q)}_{\text{clause (1 literal)}}$$

- "AND of ORs."

CNF vs. DNF

- **CNF** (Conjunctive Normal Form): AND of ORs
  $(P \vee Q) \wedge (\neg P \vee R)$
- **DNF** (Disjunctive Normal Form): OR of ANDs
  $(P \wedge Q) \vee (\neg P \wedge R)$
- Every propositional formula has an equivalent CNF form and an equivalent DNF form.
- **This course focuses on CNF**, since it is what resolution and Horn-clause rule engines require.

Truth Table ⇒ DNF (Sum of Products)

Rule. Disjunctive Normal Form (DNF), also called **Sum of Products (SOP)**, is built from the rows where the output is 1.

Step 1. Select every row with $F = 1$: rows 1, 2, 4, 7.

Step 2. For each such row, build a **minterm** (a product/AND of literals): a variable appears *as-is* if its value is 1, *negated* if its value is 0.

Step 3. Combine all minterms with OR:

$$F = m_1 \vee m_2 \vee m_4 \vee m_7$$

Canonical DNF / SOP

$$F = (\neg P \wedge \neg Q \wedge R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge \neg Q \wedge \neg R) \vee (P \wedge Q \wedge R)$$

P	Q	R	F	term
0	0	0	0	–
0	0	1	1	m_1
0	1	0	1	m_2
0	1	1	0	–
1	0	0	1	m_4
1	0	1	0	–
1	1	0	0	–
1	1	1	1	m_7

Highlighted rows ($F = 1$) are the ones that contribute a minterm to the SOP expression.

$$m_1 = \neg P \wedge \neg Q \wedge R$$

$$m_2 = \neg P \wedge Q \wedge \neg R$$

$$m_4 = P \wedge \neg Q \wedge \neg R$$

$$m_7 = P \wedge Q \wedge R$$

Truth Table ⇒ CNF (Product of Sums)

Rule. Conjunctive Normal Form (CNF), also called **Product of Sums (POS)**, is built from the rows where the output is 0.

Step 1. Select every row with $F = 0$: rows 0, 3, 5, 6.

Step 2. For each such row, build a **maxterm** (a sum/OR of literals): a variable appears *negated* if its value is 1, *as-is* if its value is 0 (the *opposite* rule from minterms).

Step 3. Combine all maxterms with AND:

$$F = M_0 \wedge M_3 \wedge M_5 \wedge M_6$$

P	Q	R	F	term
0	0	0	0	M_0
0	0	1	1	–
0	1	0	1	–
0	1	1	0	M_3
1	0	0	1	–
1	0	1	0	M_5
1	1	0	0	M_6
1	1	1	1	–

Highlighted rows ($F = 0$) are the ones that contribute a maxterm to the POS expression.

Canonical CNF / POS

$$F = (P \vee Q \vee R) \wedge (P \vee \neg Q \vee \neg R) \wedge (\neg P \vee Q \vee \neg R) \wedge (\neg P \vee \neg Q \vee R)$$

$$M_0 = P \vee Q \vee R$$

$$M_3 = P \vee \neg Q \vee \neg R$$

$$M_5 = \neg P \vee Q \vee \neg R$$

$$M_6 = \neg P \vee \neg Q \vee R$$

DNF and CNF Are the Same Function

Both describe the **same** truth table for $F = P \oplus Q \oplus R$ (three-input parity/XOR):

- **DNF/SOP** reads the 1-rows: OR of AND-terms.
- **CNF/POS** reads the 0-rows: AND of OR-terms.
- Both are logically equivalent, exact representations of F. We pick whichever normal form our algorithm needs (SOP is natural for circuit design; **CNF/POS is what resolution and Horn-clause rule engines require**).

The Conversion Algorithm (Overview)

To convert any formula into CNF, apply these steps in order:

1. **Eliminate** $\Leftrightarrow$: $A \Leftrightarrow B \equiv (A \rightarrow B) \wedge (B \rightarrow A)$
2. **Eliminate** $\rightarrow$: $A \rightarrow B \equiv \neg A \vee B$
3. **Push negations inward** (get Negation Normal Form) using:
   - De Morgan: $\neg(A \wedge B) \equiv \neg A \vee \neg B, \quad \neg(A \vee B) \equiv \neg A \wedge \neg B$
   - Double negation: $\neg\neg A \equiv A$
4. **Distribute** $\vee$ over $\wedge$: $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$

Worked Example – Setup

Convert to CNF:

$$(P \longrightarrow Q) \longrightarrow (R \vee S)$$

Step 1: Eliminate $\Leftrightarrow$.
None present – skip.

Step 2: Eliminate $\longrightarrow$.
Apply $A \longrightarrow B \equiv \neg A \vee B$ to the *outer* implication first, treating $A = (P \longrightarrow Q)$ and $B = (R \vee S)$:

$$\neg(P \longrightarrow Q) \vee (R \vee S)$$

Then eliminate the remaining inner $\longrightarrow$:

$$\neg(\neg P \vee Q) \vee (R \vee S)$$

Pushing Negation Inward

Current formula:

$$\neg(\neg P \vee Q) \vee (R \vee S)$$

Step 3: Push negation inward (De Morgan).

$$\neg(\neg P \vee Q) \equiv \neg\neg P \wedge \neg Q \equiv P \wedge \neg Q$$

So the formula becomes:

$$(P \wedge \neg Q) \vee (R \vee S)$$

No more negations sit in front of a compound formula. This is Negation Normal Form (NNF).

Distributing $\vee$ over $\wedge$

Current formula:

$$(P \wedge \neg Q) \vee (R \vee S)$$

Step 4: Distribute. Using $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$ with $A = (R \vee S)$, $B = P$, $C = \neg Q$:

$$\big((R \vee S) \vee P\big) \wedge \big((R \vee S) \vee \neg Q\big)$$

Final CNF (flattening the nested disjunctions):

$$(P \vee R \vee S) \ \wedge \ (\neg Q \vee R \vee S)$$

Two clauses, as required.

CNF and Horn Clauses

- A **Horn clause** is a CNF clause with **at most one positive literal**.
- Expert-system rules are Horn clauses in disguise:

$$\texttt{is_mammal}(x) \land \texttt{eats_meat}(x) \longrightarrow \texttt{is_carnivore}(x)$$

- Eliminating $\longrightarrow$ and applying De Morgan gives its CNF form:

$$\neg\texttt{is_mammal}(x) \lor \neg\texttt{eats_meat}(x) \lor \texttt{is_carnivore}(x)$$

- Exactly **one** positive literal (`is_carnivore`) and any number of negative ones. This is precisely why rule-based engines can use efficient goal-driven strategies like backward chaining.

A Note on First-Order Formulas

Converting a **first-order** formula to CNF needs two extra steps, inserted before distributing $\vee$ over $\wedge$:

- **Standardize variables apart**: rename bound variables so that no two quantifiers reuse the same variable name.
- **Skolemization**: eliminate $\exists$ by replacing each existentially quantified variable with a fresh function of the universally quantified variables surrounding it (a *Skolem function*); then simply **drop** all remaining $\forall$ quantifiers (they are implicit in CNF).

Example: $\forall x\, \exists y\, \mathtt{Loves}(x, y)$ becomes $\mathtt{Loves}(x, f(x))$, where f is a new Skolem function meaning "the person x loves."

Summary

- CNF = conjunction of clauses = "AND of ORs."
- **Conversion recipe:** eliminate $\Leftrightarrow$, eliminate $\rightarrow$, push $\neg$ inward (De Morgan + double negation), distribute $\vee$ over $\wedge$.
- For FOL: also standardize variables apart and Skolemize before dropping $\forall$.
- CNF clauses with at most one positive literal are **Horn clauses**, the backbone of expert-system rule bases and backward/forward chaining.

Task #6

Convert each formula below into CNF.

1. $\forall x\,(\mathrm{P}(x) \Leftrightarrow \mathrm{Q}(x))$
2. $\forall x\,\big(\texttt{Bird}(x) \longrightarrow \exists y\,(\texttt{Nest}(y) \wedge \texttt{BuildsIn}(x, y))\big)$
3. $\forall x\,\Big(\texttt{Student}(x) \longrightarrow \neg\forall y\,(\texttt{Course}(y) \longrightarrow \texttt{Failed}(x, y))\Big)$
4. $\forall x\,\big(\texttt{Cat}(x) \longrightarrow \texttt{Animal}(x)\big) \wedge \forall x\,\big(\texttt{Animal}(x) \longrightarrow \exists y\,\texttt{Likes}(x, y)\big)$

   (Look carefully at how many times $\forall x$ appears before you start.)

Reminder: the full conversion algorithm

1. Eliminate $\Leftrightarrow$: $A \Leftrightarrow B \equiv (A \longrightarrow B) \wedge (B \longrightarrow A)$
2. Eliminate $\longrightarrow$: $A \longrightarrow B \equiv \neg A \vee B$
3. Push $\neg$ inward (De Morgan, quantifier negation, double negation) to reach NNF.
4. **Standardize variables apart**: rename bound variables so no two quantifiers share a name.
5. **Skolemize**: replace each existentially quantified variable with a fresh function of the universally quantified variables enclosing it, then drop the $\exists$.
6. Drop all remaining $\forall$ (implicit in CNF).
7. Distribute $\vee$ over $\wedge$ to flatten into clauses.

❶ **Eliminate ⇔:**

$$\forall x \left((P(x) \rightarrow Q(x)) \wedge (Q(x) \rightarrow P(x)) \right)$$

❷ **Eliminate →:**

$$\forall x \left((\neg P(x) \vee Q(x)) \wedge (\neg Q(x) \vee P(x)) \right)$$

❸ **Push ¬ inward:** no change needed (no negated compound sub-formula).

❹ **Standardize variables apart:** only one variable, x; no change needed.

❺ **Skolemize:** no $\exists$ present; no change needed.

❻ **Drop ∀:**

$$(\neg P(x) \vee Q(x)) \wedge (\neg Q(x) \vee P(x))$$

❼ **Distribute:** already a conjunction of two clauses; no change needed.

Final CNF:

$$(\neg P(x) \vee Q(x)) \wedge (\neg Q(x) \vee P(x))$$

$$\forall x \left(\neg \texttt{Bird}(x) \vee \exists y \left(\texttt{Nest}(y) \wedge \texttt{BuildsIn}(x, y) \right) \right)$$

Skolemize: $\exists y$ is inside the scope of $\forall x$, so y depends on x. Replace y by a fresh Skolem function $f(x)$ and drop $\exists y$ ($f(x)$ *intuitively denotes "the nest that bird* x *builds in."*):

$$\forall x \left(\neg \texttt{Bird}(x) \vee \left(\texttt{Nest}(\texttt{f}(x)) \wedge \texttt{BuildsIn}(x, \texttt{f}(x)) \right) \right)$$

$$\neg \texttt{Bird}(x) \vee \left(\texttt{Nest}(\texttt{f}(x)) \wedge \texttt{BuildsIn}(x, \texttt{f}(x)) \right)$$

$$\left(\neg \texttt{Bird}(x) \vee \texttt{Nest}(\texttt{f}(x)) \right) \wedge \left(\neg \texttt{Bird}(x) \vee \texttt{BuildsIn}(x, \texttt{f}(x)) \right)$$

$$\left(\neg \texttt{Bird}(x) \vee \texttt{Nest}(\texttt{f}(x)) \right) \wedge \left(\neg \texttt{Bird}(x) \vee \texttt{BuildsIn}(x, \texttt{f}(x)) \right)$$

$$\forall x \left(\neg \texttt{Student}(x) \vee \neg \forall y \left(\neg \texttt{Course}(y) \vee \texttt{Failed}(x, y) \right) \right)$$

$$\begin{aligned} \neg \forall y \left(\neg \texttt{Course}(y) \vee \texttt{Failed}(x, y) \right) &\equiv \exists y \neg (\neg \texttt{Course}(y) \vee \texttt{Failed}(x, y)) \\ &\equiv \exists y \left(\texttt{Course}(y) \wedge \neg \texttt{Failed}(x, y) \right) \end{aligned}$$

So the formula becomes:

$$\forall x \left(\neg \texttt{Student}(x) \vee \exists y \left(\texttt{Course}(y) \wedge \neg \texttt{Failed}(x, y) \right) \right)$$

Skolemize: $\exists y$ is inside the scope of $\forall x$, so replace y with a Skolem function $g(x)$ and drop $\exists y$ ($g(x)$ *intuitively denotes "a course that student x passed."*):

$$\forall x \left(\neg \texttt{Student}(x) \vee \left(\texttt{Course}(g(x)) \wedge \neg \texttt{Failed}(x, g(x)) \right) \right)$$

Drop $\forall$:

$$\neg \texttt{Student}(x) \vee \left(\texttt{Course}(g(x)) \wedge \neg \texttt{Failed}(x, g(x)) \right)$$

Distribute:

$$(\neg \texttt{Student}(x) \vee \texttt{Course}(g(x))) \wedge (\neg \texttt{Student}(x) \vee \neg \texttt{Failed}(x, g(x)))$$

Eliminate $\rightarrow$ (in both conjuncts):

$$\forall x\,(\neg \texttt{Cat}(x) \vee \texttt{Animal}(x)) \wedge \forall x\,(\neg \texttt{Animal}(x) \vee \exists y\,\texttt{Likes}(x, y))$$

$$\forall x\,(\neg \texttt{Cat}(x) \vee \texttt{Animal}(x)) \wedge \forall z\,(\neg \texttt{Animal}(z) \vee \exists y\,\texttt{Likes}(z, y))$$

Skolemize. $\exists y$ is inside the scope of $\forall z$ only (not $\forall x$), so replace y with a Skolem function of z, i.e. $h(z)$:

$$\forall x\,(\neg \texttt{Cat}(x) \vee \texttt{Animal}(x)) \wedge \forall z\,(\neg \texttt{Animal}(z) \vee \texttt{Likes}(z, h(z)))$$

$$(\neg \texttt{Cat}(x) \vee \texttt{Animal}(x)) \wedge (\neg \texttt{Animal}(z) \vee \texttt{Likes}(z, h(z)))$$

Final CNF:

$$(\neg \texttt{Cat}(x) \vee \texttt{Animal}(x)) \wedge (\neg \texttt{Animal}(z) \vee \texttt{Likes}(z, h(z)))$$

Anatomy of a Rule-Based System

- **Rule base (knowledge base)**: the full collection of IF-THEN rules encoding expert knowledge
- **Working memory (fact base)**: the set of facts currently known to be true
- **Inference engine**: the mechanism that matches facts against rules and decides what to do next

Rule Base (static, authored by expert) $\xrightarrow{\textit{matches}}$ Working Memory (dynamic, grows during reasoning)

The **inference engine** is what connects these two, and *how* it connects them is exactly the choice between forward and backward chaining.

Rules with Compound Conditions

- A rule's IF-part can combine multiple conditions with **AND** / **OR**
- All conditions (for AND) must match known facts before the rule can fire

Compound Rule

$$\texttt{bird}(x) \land \texttt{cannot_fly}(x) \longrightarrow \texttt{penguin}(x)$$

IF `x is a bird` *AND* `x cannot fly` *THEN* `x is a penguin`

- Rules can also **chain**: the conclusion of one rule becomes a condition used by another
- This chaining is what allows an inference engine to derive facts that were never explicitly stated

Example

Knowing only has_feathers and cannot_fly, a chain of rules can derive lives_in_cold_climate.

From Rules to Reasoning: Two Strategies

- A rule base alone does nothing. It needs a strategy to decide **which rules to apply, and in what order**
- Two fundamentally different strategies exist for traversing the same rule base:

Forward Chaining

Start from **facts**, fire matching rules, derive new facts

Backward Chaining

Start from a **goal**, find rules that could prove it, recurse on subgoals

Same rules, same knowledge → different direction of travel.

Forward Chaining

Core Idea

Forward chaining is a **data-driven** inference strategy: reasoning starts from known facts and moves toward conclusions.

- Start with the **working memory** (facts currently known)
- Scan the **rule base** for rules whose conditions (IF-part) are satisfied
- Fire a matching rule $\longrightarrow$ add its conclusion (THEN-part) as a new fact
- Repeat until no new facts can be derived

"What can I conclude from what I currently know?"

The Inference Cycle

1. **Match**: find all rules whose premises are satisfied by the current facts
2. **Conflict Resolution**: if several rules match, choose which one(s) to fire
3. **Act**: execute the chosen rule, updating working memory

Conflict Resolution Strategies

- Rule ordering (priority)
- Specificity (prefer more specific rules)
- Recency (prefer rules using most recently added facts)

The cycle repeats until **no new facts** are generated *(fixed point reached)*.

Example & Use Cases

Simple Rule Base

$$\mathfrak{R}_1 : \texttt{bird}(x) \land \texttt{cannot_fly}(x) \longrightarrow \texttt{penguin}(x)$$

$$\mathfrak{R}_2 : \texttt{penguin}(x) \longrightarrow \texttt{lives_in_cold_climate}(x)$$

$$\mathfrak{R}_3 : \texttt{has_feathers}(x) \longrightarrow \texttt{bird}(x)$$

Facts: `has_feathers(tux)`, `cannot_fly(tux)`

$\longrightarrow$ Fires $\mathfrak{R}_3$, then $\mathfrak{R}_1$, then $\mathfrak{R}_2$ $\longrightarrow$ derives `lives_in_cold_climate(tux)`

Typical Applications

- Monitoring and control systems *(sensor data $\longrightarrow$ alerts)*
- Situations requiring **all** derivable conclusions

Conflict Resolution (1/3)

Why It's Needed

The Problem

In forward chaining, more than one rule may have its conditions satisfied **at the same time**. This set of matching rules is called the conflict set.

- The inference engine cannot fire all matching rules simultaneously *(order matters, side effects matter)*
- A **conflict resolution strategy** decides *which* rule fires next
- This choice can change the reasoning path, and probably the final outcomes

Same facts, same rules but different firing order can lead to different results.

Conflict Resolution (2/3)

Common Strategies

- **Rule ordering / priority**: rules have fixed precedence *(e.g., numbered, or explicit priority tags)*; highest-priority match fires first
- **Specificity**: prefer the rule with more conditions *(more specific to the current situation)* over a more general rule
- **Recency**: prefer rules that reference the most recently added facts *(keeps reasoning focused on new information)*
- **Refractoriness**: a rule that has already fired on a given set of facts is not allowed to fire again on the same facts *(prevents infinite loops)*

Conflict Resolution (3/3)

Example

Conflict Set

Given facts: `bird(tux)`, `has_feathers(tux)`, `cannot_fly(tux)`

Suppose **two** rules match at once:

$$\mathfrak{R}_a : \texttt{bird}(x) \rightarrow \texttt{lays_eggs}(x)$$

$$\mathfrak{R}_b : \texttt{bird}(x) \wedge \texttt{cannot_fly}(x) \rightarrow \texttt{penguin}(x)$$

- By **specificity**: $\mathfrak{R}_b$ fires first (more conditions matched)
- By **rule ordering**: whichever rule is listed first fires first, regardless of specificity

Takeaway

The conflict resolution strategy is part of the **engine's design**, not the knowledge base. Same rules can behave differently under different engines.

Exercise: Forward Chaining Inference (1/6)

Task #7

Consider a rule-based expert system with the following knowledge base, made up of seven production rules $\mathfrak{R}_1$ to $\mathfrak{R}_7$, operating over ten Boolean facts $A, B, C, D, E, F, G, H, I, J$.

Rule base

Rule	Statement		
$\mathfrak{R}_1$	$A \wedge B$	$\longrightarrow$	C
$\mathfrak{R}_2$	C	$\longrightarrow$	D
$\mathfrak{R}_3$	$A \wedge E$	$\longrightarrow$	F
$\mathfrak{R}_4$	$D \wedge F$	$\longrightarrow$	G
$\mathfrak{R}_5$	G	$\longrightarrow$	H
$\mathfrak{R}_6$	$B \wedge E$	$\longrightarrow$	I
$\mathfrak{R}_7$	$I \wedge C$	$\longrightarrow$	J

Initial working memory

The following facts are initially *known* to be **true**:

A, B, E

All other facts (C, D, F, G, H, I, J) are initially *unknown*.

Exercise: Forward Chaining Inference (2/6)

(a) Apply the **forward chaining** algorithm to this rule base, starting from the initial working memory. At each cycle, list:

- which rule(s) have all their premises satisfied by the current working memory,
- which new fact(s) get added as a result,

until no further rule can fire.

(b) List the complete set of facts contained in working memory once the algorithm terminates.

(c) Can the fact H be derived from the initial facts A, B, E? If so, give the ordered sequence of rules that were fired to reach it.

(d) Rule $\mathfrak{R}_7$ requires both I and C to be known. Explain, using conflict resolution, why $\mathfrak{R}_1$ and $\mathfrak{R}_6$ are eligible to fire in the same cycle, and whether the order in which they fire changes the final result of the inference process.

(e) Suppose fact E were *not* part of the initial working memory *(i.e. only A and B are given)*. Which facts, if any, can no longer be derived? Justify the answer by identifying the rule(s) that become permanently blocked.

Exercise: Forward Chaining Inference (3/6)

(a) Forward chaining trace

Working memory (cycle 0): $\{A, B, E\}$

- **Cycle 1:** Scan all rules against $\{A, B, E\}$.
  - $\mathfrak{R}_1$: IF $A \wedge B$ THEN $C \longrightarrow$ premises satisfied (A, B known) $\longrightarrow$ fire, add C.
  - $\mathfrak{R}_3$: IF $A \wedge E$ THEN $F \longrightarrow$ premises satisfied (A, E known) $\longrightarrow$ fire, add F.
  - $\mathfrak{R}_6$: IF $B \wedge E$ THEN $I \longrightarrow$ premises satisfied (B, E known) $\longrightarrow$ fire, add I.
  - $\mathfrak{R}_2, \mathfrak{R}_4, \mathfrak{R}_5, \mathfrak{R}_7$ cannot fire yet (their premises are not all known).

  New working memory {A, B, E, C, F, I}
- **Cycle 2:** Scan all rules against $\{A, B, E, C, F, I\}$.
  - $\mathfrak{R}_2$: IF C THEN $D \longrightarrow$ premise satisfied (C known) $\longrightarrow$ fire, add D.
  - $\mathfrak{R}_7$: IF $I \wedge C$ THEN $J \longrightarrow$ premises satisfied (I, C known) $\longrightarrow$ fire, add J.
  - $\mathfrak{R}_4$ cannot fire yet (F known, but D not yet known at the start of this cycle).

  New working memory {A, B, E, C, F, I, D, J}

Exercise: Forward Chaining Inference (4/6)

- **Cycle 3:** Scan all rules against $\{A, B, C, D, E, F, I, J\}$.
  - $\mathfrak{R}_4$: IF $D \wedge F$ THEN $G \longrightarrow$ premises satisfied (D, F known) $\longrightarrow$ fire, add G.

  New working memory {A, B, C, D, E, F, G, I, J}
- **Cycle 4:** Scan all rules against $\{A, B, C, D, E, F, G, I, J\}$.
  - $\mathfrak{R}_5$: IF G THEN $H \longrightarrow$ premise satisfied (G known) $\longrightarrow$ fire, add H.

  New working memory {A, B, C, D, E, F, G, H, I, J}
- **Cycle 5:** No rule's premises are unsatisfied among the remaining candidates; every consequent has already been derived. Algorithm terminates.

(b) Final working memory

{A, B, C, D, E, F, G, H, I, J}

All ten facts end up being derivable. The rule base is fully saturated from the given initial facts.

Exercise: Forward Chaining Inference (5/6)

(c) Can H be derived?

Yes. The ordered sequence of rule firings that leads to H is:

$$\mathfrak{R}_1 \ (\longrightarrow \mathrm{C}), \quad \mathfrak{R}_3 \ (\longrightarrow \mathrm{F}), \quad \mathfrak{R}_2 \ (\longrightarrow \mathrm{D}), \quad \mathfrak{R}_4 \ (\longrightarrow \mathrm{G}), \quad \mathfrak{R}_5 \ (\longrightarrow \mathrm{H})$$

($\mathfrak{R}_6$ and $\mathfrak{R}_7$ also fire along the way but are not on the minimal path required to reach H; only $\mathfrak{R}_1, \mathfrak{R}_2, \mathfrak{R}_3, \mathfrak{R}_4, \mathfrak{R}_5$ are strictly needed.)

(d) Conflict resolution between $\mathfrak{R}_1$ and $\mathfrak{R}_6$

At cycle 1, both $\mathfrak{R}_1$ (needs A, B) and $\mathfrak{R}_6$ (needs B, E) have all their premises already present in working memory (A, B, E are all known from the start), so both belong to the *conflict set*. Since $\mathfrak{R}_1$ and $\mathfrak{R}_6$ do not share a consequent and neither rule's premises depend on the other's conclusion, they are independent: firing $\mathfrak{R}_1$ before $\mathfrak{R}_6$, or $\mathfrak{R}_6$ before $\mathfrak{R}_1$ produces the same final working memory.

Exercise: Forward Chaining Inference (6/6)

(e) Effect of removing E from the initial facts

If working memory starts as $\{A, B\}$ only:

- $\mathfrak{R}_1$: IF $A \wedge B$ THEN C still fires $\longrightarrow$ C is added.
- $\mathfrak{R}_3$: IF $A \wedge E$ THEN F **cannot** fire, since E is never known $\longrightarrow$ F is permanently blocked.
- $\mathfrak{R}_6$: IF $B \wedge E$ THEN I **cannot** fire, since E is never known $\longrightarrow$ I is permanently blocked.
- $\mathfrak{R}_2$: IF C THEN D still fires (since C was derived) $\longrightarrow$ D is added.
- $\mathfrak{R}_4$: IF $D \wedge F$ THEN G **cannot** fire, since F is never derived $\longrightarrow$ G is permanently blocked.
- $\mathfrak{R}_5$: IF G THEN H **cannot** fire, since G is never derived $\longrightarrow$ H is permanently blocked.
- $\mathfrak{R}_7$: IF $I \wedge C$ THEN J **cannot** fire, since I is never derived $\longrightarrow$ J is permanently blocked.

Final working memory in this case: $\{A, B, C, D\}$ only.



The code is available @ github.com/a-mhamdi/jlai ⟶ *Codes* ⟶ *Julia* ⟶ *Part-1*

⟶ *Pluto* ⟶ *forward-chaining.jl*

Pluto.jl

⟶ *Jupyter* ⟶ *forward-chaining.ipynb*



Backward Chaining

Core Idea

Backward chaining is a **goal-driven** inference strategy: reasoning starts from a hypothesis (goal) and works backward to see if known facts support it.

- Start with a **goal** to prove *(e.g., is this true?)*
- Find rules whose **conclusion** (THEN-part) matches the goal
- Treat the rule's **conditions** (IF-part) as new **subgoals**
- Recursively prove each subgoal, until subgoals match known facts

"Is this specific thing true, given what I know?"

The Backward Chaining Mechanism

Select goal: pick a hypothesis to verify

Match: find rule(s) whose conclusion matches the goal

Subgoal: push the rule's premises onto a **goal stack**

Recurse: attempt to prove each subgoal *(as a fact or via further rules)*

Backtrack: if a subgoal fails, try an alternative rule *(if any)*

Success Condition

The original goal is proven true when all its subgoals are reduced to facts already present in working memory.

Example & Use Cases

Same Rule Base as Forward Chaining

$$\mathfrak{R}_1 : \texttt{bird}(x) \land \texttt{cannot_fly}(x) \longrightarrow \texttt{penguin}(x)$$

$$\mathfrak{R}_2 : \texttt{penguin}(x) \longrightarrow \texttt{lives_in_cold_climate}(x)$$

$$\mathfrak{R}_3 : \texttt{has_feathers}(x) \longrightarrow \texttt{bird}(x)$$

Goal: `lives_in_cold_climate(tux)`?

$\longrightarrow$ Needs $\mathfrak{R}_2 \longrightarrow$ subgoal `penguin(tux)` $\longrightarrow$ Needs $\mathfrak{R}_1 \longrightarrow$ subgoals `bird(tux)`, `cannot_fly(tux)` $\longrightarrow$ `bird(tux)` needs $\mathfrak{R}_3 \longrightarrow$ subgoal `has_feathers(tux)` $\longrightarrow$ Both facts known $\longrightarrow$ **goal proven**

Typical Applications

- Diagnostic / expert consultation systems *(e.g., **MYCIN**)*
- Situations testing a **specific hypothesis**, not deriving everything

Conflict Resolution in Backward Chaining

The Problem

When proving a goal, **multiple rules** may have a conclusion (THEN-part) that matches the current goal.

- Unlike forward chaining, there's no single "conflict set" fired in one step
- Instead, the engine must decide **which candidate rule to try first**
- If that rule's subgoals fail, the engine must **backtrack** and try the next candidate

The question shifts from "which rule fires?" to "which rule do I try first, and what if it fails?"

Rule Selection & Backtracking Strategies

- **Rule ordering**: try candidate rules in a fixed sequence *(e.g., as listed in the knowledge base)*
- **Specificity**: prefer rules with more restrictive conditions, on the assumption they are more likely correct for the situation
- **Most-recently-used facts first**: order subgoal attempts to reuse facts already proven in this search, avoiding redundant work
- **Depth-first with backtracking**: if a chosen rule's subgoal fails, undo and try the next matching rule for that same goal

Backtracking: Example

Two Rules, Same Conclusion

$$\mathfrak{R}_a : \texttt{has_feathers}(x) \rightarrow \texttt{bird}(x)$$

$$\mathfrak{R}_b : \texttt{lays_eggs}(x) \wedge \texttt{has_beak}(x) \rightarrow \texttt{bird}(x)$$

Goal: `bird(tux)`?

Known facts: `lays_eggs(tux)`, `has_beak(tux)`

- Engine tries $\mathfrak{R}_a$ first *(rule ordering)* $\rightarrow$ subgoal `has_feathers(tux)` $\rightarrow$ not known $\rightarrow$ fails
- **Backtrack**: engine tries $\mathfrak{R}_b$ $\rightarrow$ subgoals `lays_eggs(tux)`, `has_beak(tux)` $\rightarrow$ both known $\rightarrow$ **succeeds**

Takeaway

Backtracking lets backward chaining recover from a failed rule choice. The cost is potentially re-exploring parts of the search space.

Exercise on Backward Chaining (1/7)

Task #8

The expert system below identifies an animal from observable characteristics. The rule base $\mathcal{R} = \{\mathfrak{R}_1, \ldots, \mathfrak{R}_{11}\}$ is:

$$\mathfrak{R}_1 : \texttt{has_hair}(x) \longrightarrow \texttt{is_mammal}(x)$$

$$\mathfrak{R}_2 : \texttt{gives_milk}(x) \longrightarrow \texttt{is_mammal}(x)$$

$$\mathfrak{R}_3 : \texttt{has_feathers}(x) \longrightarrow \texttt{is_bird}(x)$$

$$\mathfrak{R}_4 : \texttt{flies}(x) \wedge \texttt{lays_eggs}(x) \longrightarrow \texttt{is_bird}(x)$$

$$\mathfrak{R}_5 : \texttt{is_mammal}(x) \wedge \texttt{eats_meat}(x) \longrightarrow \texttt{is_carnivore}(x)$$

$$\mathfrak{R}_6 : \texttt{is_mammal}(x) \wedge \texttt{has_pointed_teeth}(x) \wedge \texttt{has_claws}(x) \wedge \texttt{has_forward_eyes}(x) \longrightarrow \texttt{is_carnivore}(x)$$

$$\mathfrak{R}_7 : \texttt{is_carnivore}(x) \wedge \texttt{has_tawny_color}(x) \wedge \texttt{has_dark_spots}(x) \longrightarrow \texttt{is_cheetah}(x)$$

$$\mathfrak{R}_8 : \texttt{is_carnivore}(x) \wedge \texttt{has_tawny_color}(x) \wedge \texttt{has_black_stripes}(x) \longrightarrow \texttt{is_tiger}(x)$$

$$\mathfrak{R}_9 : \texttt{is_bird}(x) \wedge \neg\texttt{flies}(x) \wedge \texttt{has_long_neck}(x) \wedge \texttt{has_long_legs}(x) \longrightarrow \texttt{is_ostrich}(x)$$

$$\mathfrak{R}_{10} : \texttt{is_bird}(x) \wedge \neg\texttt{flies}(x) \wedge \texttt{swims}(x) \wedge \texttt{has_black_white_color}(x) \longrightarrow \texttt{is_penguin}(x)$$

$$\mathfrak{R}_{11} : \texttt{is_bird}(x) \wedge \texttt{is_good_flyer}(x) \longrightarrow \texttt{is_albatross}(x)$$

Exercise on Backward Chaining (2/7)

Rules are tried by the engine in the order in which they are written whenever several rules can conclude the same predicate *(e.g. $\mathfrak{R}_5$ is always attempted before $\mathfrak{R}_6$ to prove* `is_carnivore`*)*.

Working memory *(known facts)* for the current animal *a*

Fact	Truth value
`has_hair`(a)	**true**
`has_tawny_color`(a)	**true**
`has_dark_spots`(a)	**true**

Every other primitive (leaf) predicate is *unknown* and, if needed, must be asked to the user. Assume the user answers:

Question asked	User's answer
`eats_meat`(a)?	"I don't know" (unprovable)
`has_pointed_teeth`(a)?	yes
`has_claws`(a)?	yes
`has_forward_eyes`(a)?	yes

Exercise on Backward Chaining (3/7)

The system must determine whether

Goal: `is_cheetah(a)` ?

(a) Draw the **AND/OR** search tree that backward chaining builds while trying to prove `is_cheetah(a)`. Mark **OR**-branches *(alternative rules for the same goal)* and **AND**-branches *(conjuncts of one rule)*.

(b) Simulate the backward-chaining algorithm step by step. Show the evolution of the goal stack *(or goal list)*, state which rule is selected at each choice point, and show explicitly where the engine *backtracks*.

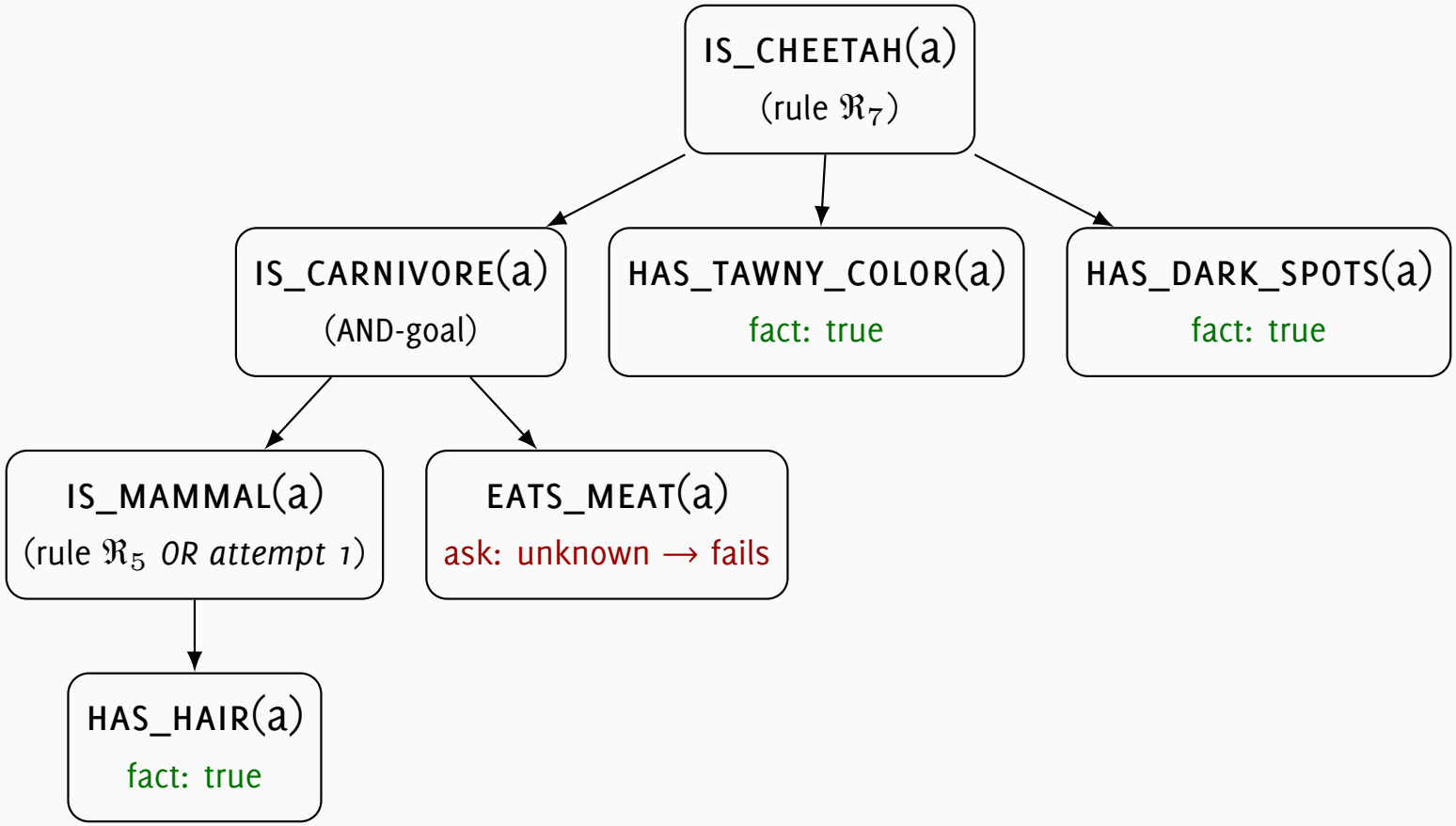
(c) List, in the order they would actually be asked, every question the engine puts to the user.

(d) Give the final answer to the goal, with the sequence of rules that constitute the justification *(the "explanation")* of the conclusion.

Exercise on Backward Chaining (4/7)

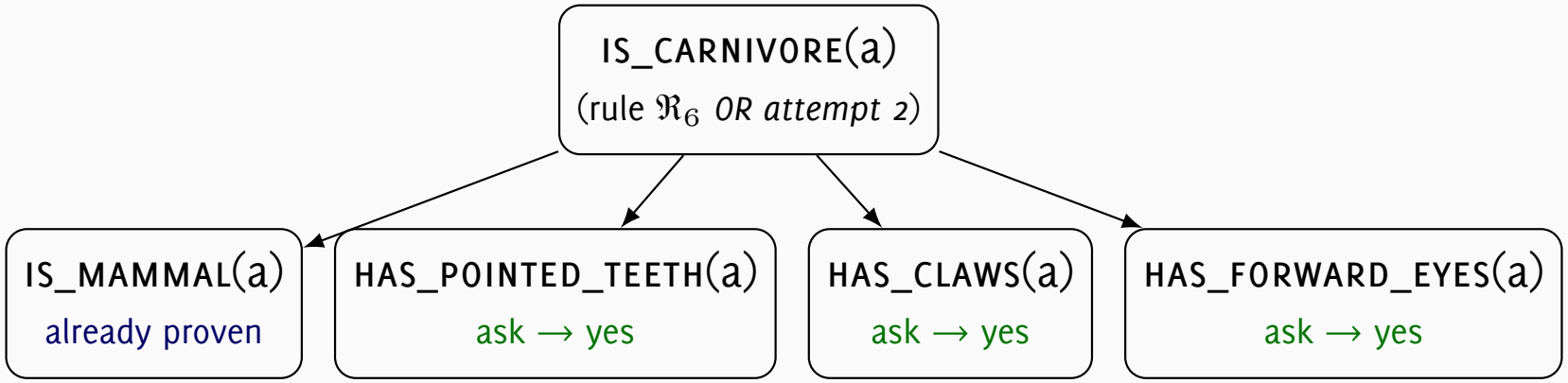
Solution

(a) AND/OR tree



Exercise on Backward Chaining (5/7)

Since the **AND**-branch through $\mathfrak{R}_5$ fails (`eats_meat`(a) cannot be proven), the engine backtracks and retries `is_carnivore`(a) with the next rule, $\mathfrak{R}_6$:



(b) Step-by-step trace of the goal stack

Notation

The stack is written left *(top, next goal to solve)* to right.
① "Push" adds sub-goals of the chosen rule;
② "Pop" removes a goal once it is proven.

★ *Abbreviations used in the stack column: carn = is_carnivore, mammal = is_mammal, hair = has_hair, tawny = has_tawny_color, spots = has_dark_spots, pt = has_pointed_teeth, cl = has_claws, fe = has_forward_eyes.*

Exercise on Backward Chaining (6/7)

Step	Current top goal	Action	Goal stack after action
1	is_cheetah(a)	Try $\mathfrak{R}7$ (only rule concluding is_cheetah); push its 3 premises	[carn(a), tawny(a), spots(a)]
2	is_carnivore(a)	Try $\mathfrak{R}_5$ (first rule for is_carnivore); push premises	[mammal(a), eats_meat(a), tawny(a), spots(a)]
3	is_mammal(a)	Try $\mathfrak{R}_1$; push premise	[hair(a), eats_meat(a), tawny(a), spots(a)]
4	has_hair(a)	Found in WM = true $\longrightarrow$ **Pop**; is_mammal(a) proven via $\mathfrak{R}_1$ (cached)	[eats_meat(a), tawny(a), spots(a)]
5	eats_meat(a)	Not in WM, no rule concludes it $\longrightarrow$ **ask user**; answer "unknown" $\longrightarrow$ premise fails	–
6	–	**Backtrack** to step 2: $\mathfrak{R}_5$ failed, try next rule $\mathfrak{R}_6$ for is_carnivore	[mammal(a), pt(a), cl(a), fe(a), tawny(a), spots(a)]
7	is_mammal(a)	Already proven true (cached) $\longrightarrow$ **Pop**	[pt(a), cl(a), fe(a), tawny(a), spots(a)]
8	has_pointed_teeth(a)	Ask user $\longrightarrow$ yes $\longrightarrow$ **Pop**	[cl(a), fe(a), tawny(a), spots(a)]
9	has_claws(a)	Ask user $\longrightarrow$ yes $\longrightarrow$ **Pop**	[fe(a), tawny(a), spots(a)]
10	has_forward_eyes(a)	Ask user $\longrightarrow$ yes $\longrightarrow$ **Pop**; all 4 premises of $\mathfrak{R}_6$ true $\longrightarrow$ is_carnivore(a) proven (cached)	[tawny(a), spots(a)]
11	has_tawny_color(a)	In WM = true $\longrightarrow$ **Pop**	[spots(a)]
12	has_dark_spots(a)	In WM = true $\longrightarrow$ **Pop**	[ ] (empty)
13	–	All premises of $\mathfrak{R}_7$ proven $\longrightarrow$ is_cheetah(a) **TRUE**	–

Exercise on Backward Chaining (7/7)

(c) Questions actually asked to the user, in order

1. `eats_meat(a)?` ⟶ "unknown"
2. `has_pointed_teeth(a)?` ⟶ yes
3. `has_claws(a)?` ⟶ yes
4. `has_forward_eyes(a)?` ⟶ yes

Note

`has_hair`, `has_tawny_color` and `has_dark_spots` are *not* asked, because they are already present in working memory: the engine only asks for facts it cannot find in the WM and cannot derive with a rule.

(d) Explanation trace

`is_cheetah(a)` = **TRUE**

$\texttt{has_hair}(a) \xrightarrow{\mathfrak{R}_1} \texttt{is_mammal}(a)$; together with $\texttt{has_pointed_teeth}(a)$, $\texttt{has_claws}(a)$, $\texttt{has_forward_eyes}(a) \xrightarrow{\mathfrak{R}_6} \texttt{is_carnivore}(a)$; together with $\texttt{has_tawny_color}(a)$ and $\texttt{has_dark_spots}(a) \xrightarrow{\mathfrak{R}_7} \texttt{is_cheetah}(a)$.



The code is available @ github.com/a-mhamdi/jlai ⟶ *Codes* ⟶ *Julia* ⟶ *Part-1*

⟶ *Pluto* ⟶ *backward-chaining.jl*

Pluto.jl

⟶ *Jupyter* ⟶ *backward-chaining.ipynb*

jupyter

Forward vs. Backward Chaining

	Forward Chaining	**Backward Chaining**
Direction	Facts ⟶ Conclusions	Goal ⟶ Facts
Driven by	Data	Hypothesis
Guiding question	"What can I conclude?"	"Is this true?"
Best suited for	Deriving *all* possible conclusions	Verifying a *specific* hypothesis
Search style	Bottom-up, breadth over facts	Top-down, depth over subgoals
Typical systems	Monitoring systems *(CLIPS)*	Diagnostic systems *(MYCIN)*

Same knowledge base ⟶ two different traversal strategies.

Fuzzy Logic

What is Fuzzy Logic? (1/3)

"There are many misconceptions about fuzzy logic. To begin with, fuzzy logic is not fuzzy. Basically, fuzzy logic is a precise logic of imprecision. [...] fuzzy logic is designed to deal with imperfect information. Imperfect information is information which in one or more aspects is imprecise, uncertain, incomplete, unreliable, vague or partially true. In the real world, such information is the norm rather than exception."

Lotfi Zadeh, *WCECS 2014*



What is Fuzzy Logic? (2/3)

> " Fuzzy Logic, in computer science, is a form of logic used in some expert systems and other artificial-intelligence applications in which variables can have degrees of truthfulness or falsehood represented by a range of values between 1 (true) and 0 (false). With fuzzy logic, the outcome of an operation can be expressed as a probability rather than as a certainty. For example, in addition to being either true or false, an outcome might have such meanings as probably true, possibly true, possibly false, and probably false."

Fuzzy Logic, Microsoft® Encarta® Online Encyclopedia 2009
https://www.refseek.com/data/cache/en/1/Fuzzy_Logic.html

What is Fuzzy Logic? (3/3)

In fuzzy logic, an element can *partially* belong to a set with a membership degree $\mu_{\mathcal{A}}(x) \in [0, 1]$

- Extension of classical Boolean logic
- Deals with **degrees of truth** rather than absolute true/false
- Variables can have values between 0 and 1
- Mimics human reasoning and decision-making

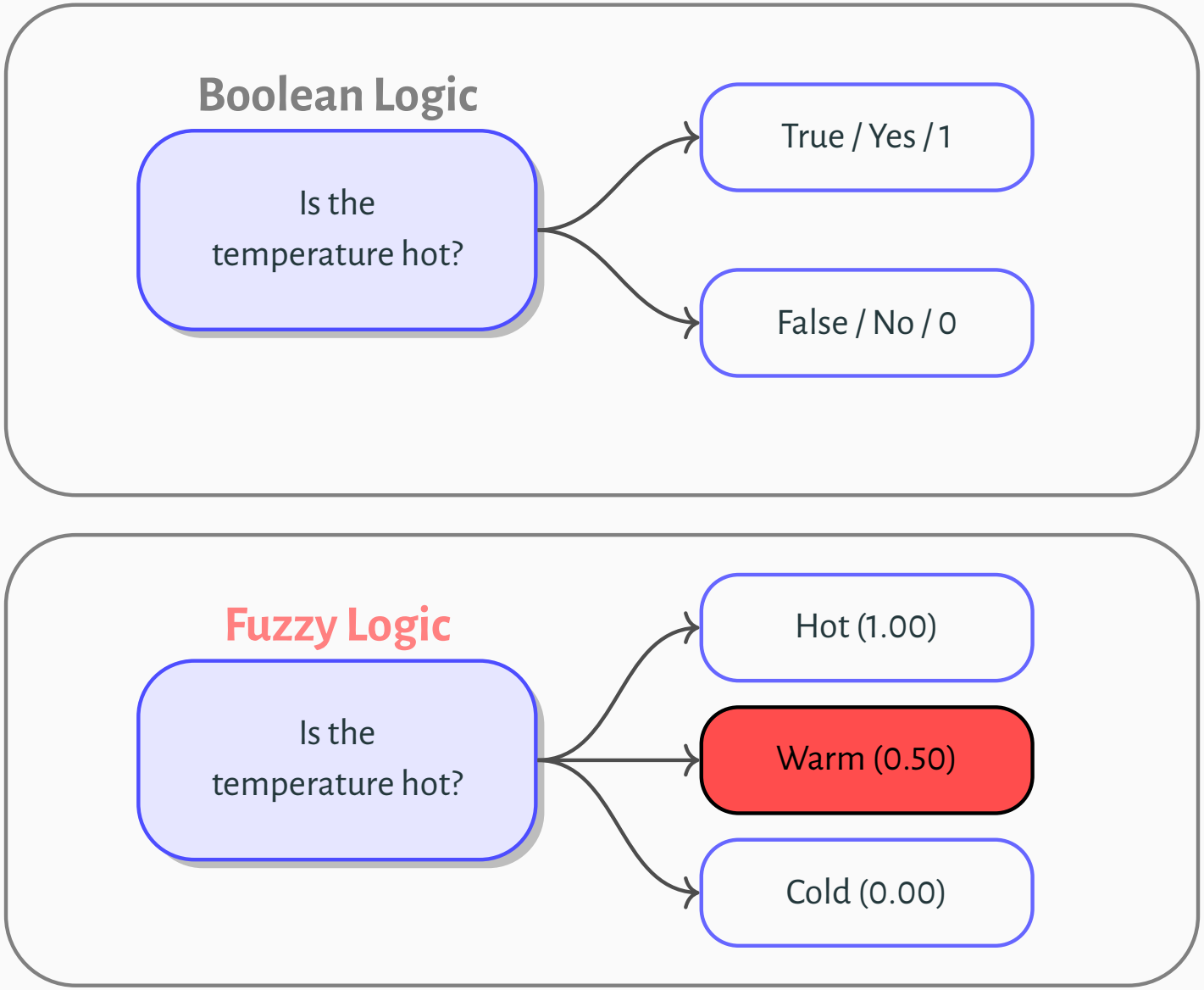
Crisp Set (Classical)

- Binary membership: 0 or 1
- Sharp boundaries
- Example: Age $\geq$ 65 = OLD

Fuzzy Set

- Gradual membership: $[0, 1]$
- Smooth transitions
- Example: Age 60 = 0.5 OLD

Fuzzy logic as an extension of the Boolean Logic

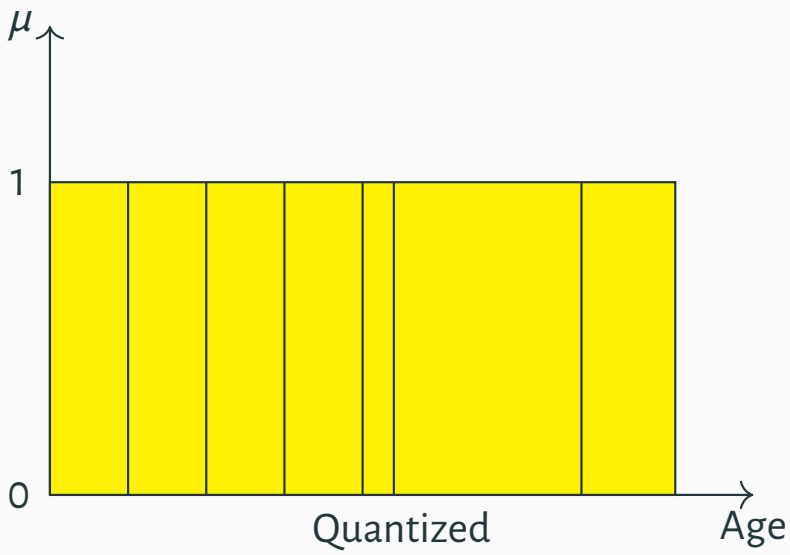


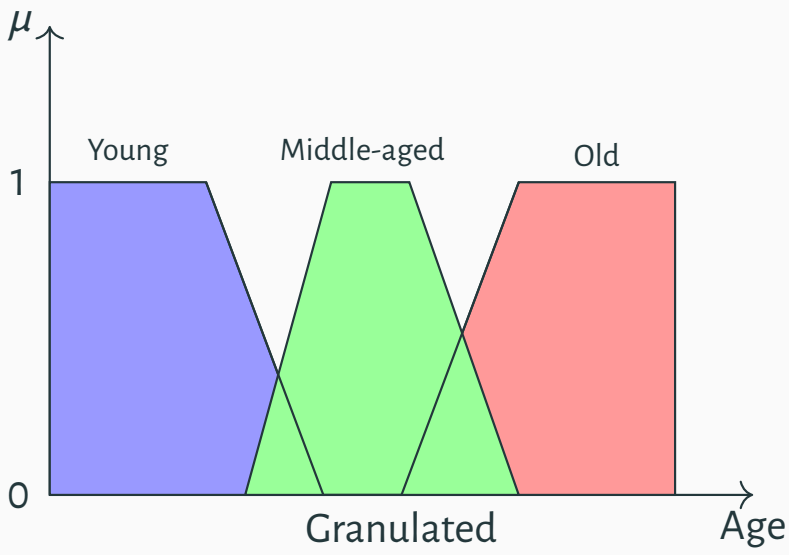
What does Fuzzy Logic have to offer?

Fuzzy Logic aims at formalizing/mechanizing two noticeable human capabilities:

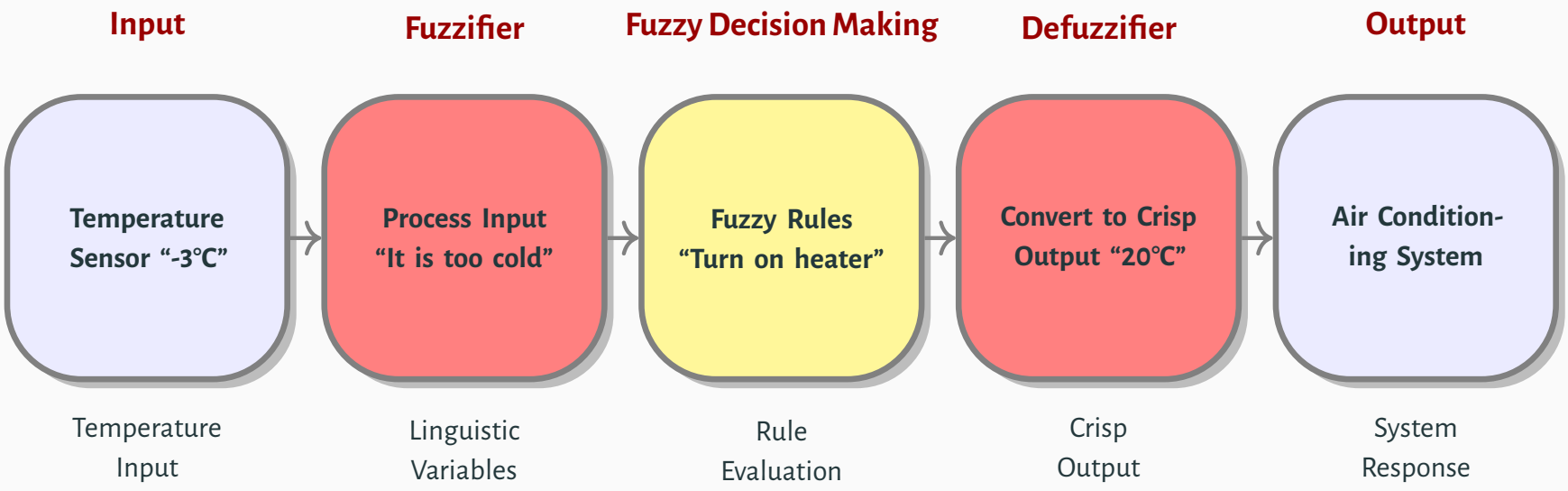
1. communicating, reasoning and rational decision making
   (in presence of imprecision, uncertainty & partiality of truth)
2. performing a wide variety of tasks
   (w/o measurements or computations)

Continuous ⟶ Quantized ⟶ Granulated





Example of a Fuzzy Control System



Architecture

Rule Base is provided by experts. It contains the set of rules to govern the decision making.

Fuzzification converts crisp numbers to fuzzy sets.

Inference Engine decides which rules to be fired matching degree of the current fuzzy inputs.

Defuzzification converts the fuzzy sets delivered by the inference engine into some crisp value

Fuzzification Process

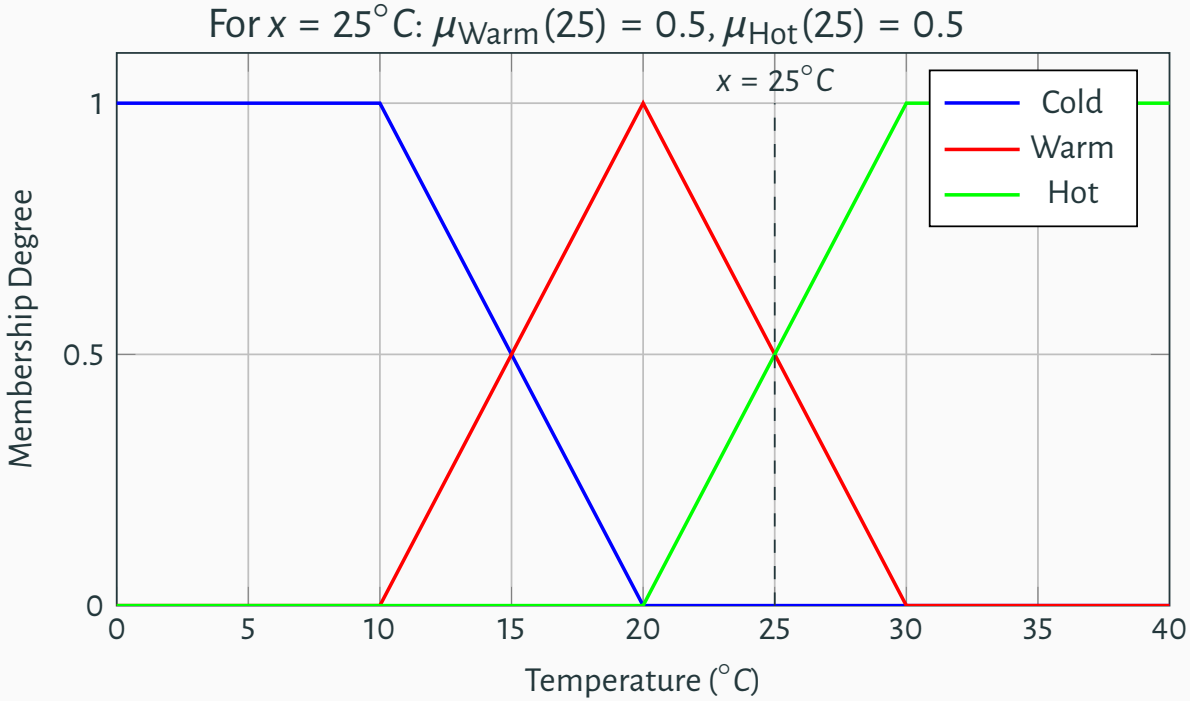
Fuzzification transforms crisp input values into fuzzy membership degrees using membership functions.

Steps:

1. Define linguistic variables (e.g., temperature: Cold, Warm, Hot)
2. Choose membership functions for each linguistic term
3. Calculate membership degrees for input values

Common types of membership functions:

- **Triangular**
- **Trapezoidal**
- **Gaussian**
- **Sigmoid**



Fuzzy Rule Base

If-then Rules

Rules express expert knowledge in linguistic form:

General Form:

$$\text{If } x \text{ is } \mathcal{X} \text{ and } y \text{ is } \mathcal{Y} \text{ then } z \text{ is } \mathcal{Z}$$

Example Rules:

1. If temperature is Cold and humidity is High then fan speed is Low
2. If temperature is Hot and humidity is Low then fan speed is High
3. If temperature is Warm then fan speed is Medium

Inference Methods

Mamdani Inference:

- Most common method
- Output is a fuzzy set
- Uses min-max composition

Sugeno Inference:

- Output is a linear function or constant
- Computationally efficient
- Common in adaptive systems

Tsukamoto Inference:

- Output membership functions must be monotonic
- Direct crisp output calculation

Defuzzification Methods

A fuzzy value can be defuzzified through multiple ways.

Common Methods:

1. Centroid (Center of Area - **COA**, aka Center of Gravity - **COG**)
2. Bisector of Area (**BOA**)
3. Mean of Maximum (**MOM**)
4. Smallest/Largest of Maximum (**SOM** and **LOM**)
5. Weighted Average

Centroid Method (COA | COG)

Most widely used method

$$z^* = \frac{\int \mu_Z(z) \cdot z \, dz}{\int \mu_Z(z) \, dz}$$

For discrete case:

$$z^* = \frac{\sum_{i=1}^{n} \mu_Z(z_i) \cdot z_i}{\sum_{i=1}^{n} \mu_Z(z_i)}$$

- Finds the center of gravity of the fuzzy set
- Considers entire output distribution
- Smooth output variation

Crisp Value in Fuzzy Notation (1/2)

Boolean Values as Fuzzy Sets

- $0 \longrightarrow \{0\}$ (completely non-membership)
- $1 \longrightarrow \{1\}$ (completely membership)
- In between 0 and 1, *e.g.*, $0.5 \longrightarrow \{0.5\}$ (partial membership)

Crisp Set Notation as Fuzzy Set Notation

- A crisp set $\mathbf{A} = \{x \mid P(x)\}$ can be represented as a fuzzy set $\mathcal{A}$ with membership function:

$$\mu_{\mathcal{A}}(x) = \begin{cases} 1 & \text{if } x \in \mathbf{A} \\ 0 & \text{if } x \notin \mathbf{A} \end{cases}$$

- Alternatively, a fuzzy set $\mathcal{A}$ can be defined with a membership function:

$$\mu_{\mathcal{A}}(x) = \begin{cases} 1 & \text{if } x \text{ is } \underline{\text{completely in}}\ \mathcal{A} \\ 0.5 & \text{if } x \text{ is } \underline{\text{partially in}}\ \mathcal{A} \\ 0 & \text{if } x \text{ is } \underline{\text{completely out}} \text{ of } \mathcal{A} \end{cases}$$

Crisp Value in Fuzzy Notation (2/2)

Fuzzy Notation for Crisp Set Elements

A crisp element $x \in \mathbf{U}$ can be represented as a fuzzy set $\{x\}$ with membership function:

$$\mu_{\{x\}}(x') = \delta(x, x')$$

where δ is the Kronecker delta.

Fuzzy Representation of Crisp Set Operations

For fuzzy sets $\mathcal{A}$ and $\mathcal{B}$:

- Union ($\mathcal{A} \cup \mathcal{B}$):

$$\mu_{\mathcal{A} \cup \mathcal{B}}(x) = \max(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x))$$

- Intersection ($\mathcal{A} \cap \mathcal{B}$):

$$\mu_{\mathcal{A} \cap \mathcal{B}}(x) = \min(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x))$$

- Complement ($\neg \mathcal{A}$):

$$\mu_{\neg \mathcal{A}}(x) = 1 - \mu_{\mathcal{A}}(x)$$

Fuzzy Operators

AND Operation (min)

$$\mu_{\mathcal{A}\cap\mathcal{B}}(x) = \min(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x))$$

NOT Operation

$$\mu_{\neg\mathcal{A}}(x) = 1 - \mu_{\mathcal{A}}(x)$$

OR Operation (max)

$$\mu_{\mathcal{A}\cup\mathcal{B}}(x) = \max(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x))$$

Implication

$$\mu_{\mathcal{A}\to\mathcal{B}}(x, y) = \min(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(y))$$

Example

If temp is Hot (0.7) AND humidity is High (0.8) $\Rightarrow$ Activation = $\min(0.7, 0.8) = 0.7$

Fuzzy Numbers (1/6)

★ Represent imprecise numbers: number & linguistic modifier *(e.g., nearly, around, etc.)*

- approximately five kilos
- about 12 pm

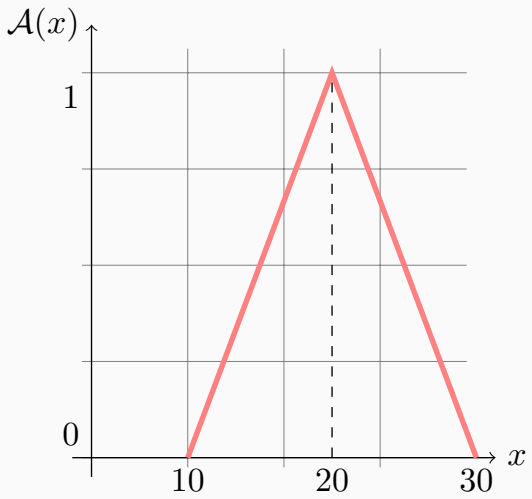
★ Play an important role in decision making, approximate reasoning, statistics with imprecise probabilities and fuzzy control.

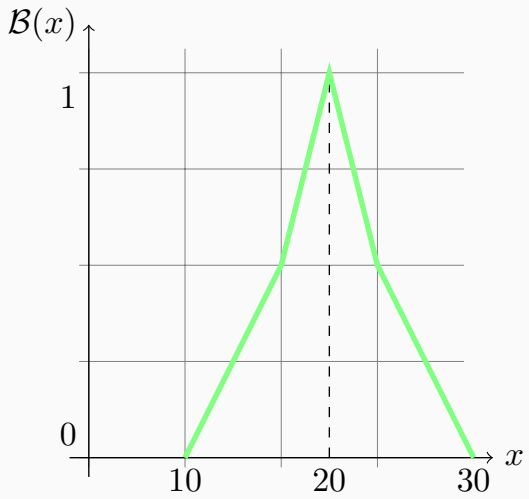
We need to perform arithmetic operations on fuzzy numbers *(e.g., calculate a ratio of some fuzzy output over some fuzzy input)*

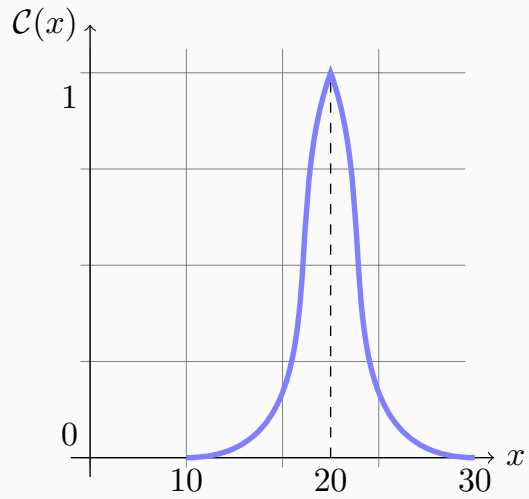
"around 20"

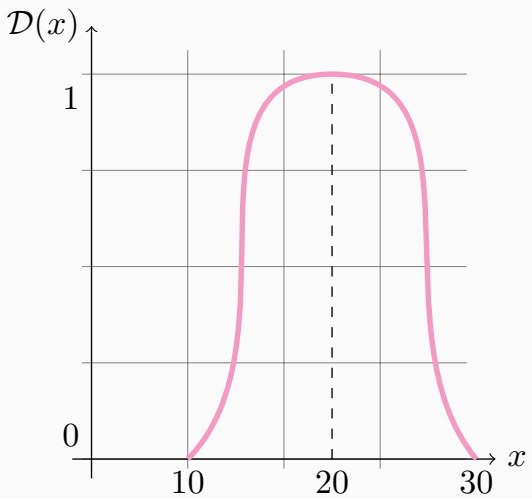
- includes some number values on either side of the central value of 20
- Central value is fully compatible with concept
- Number around central value are compatible with it to lesser degrees
- Degree of compatibility represented by fuzzy set; Membership value decreases from 1.0 to 0.0 on both sides of central value = fuzzy number.

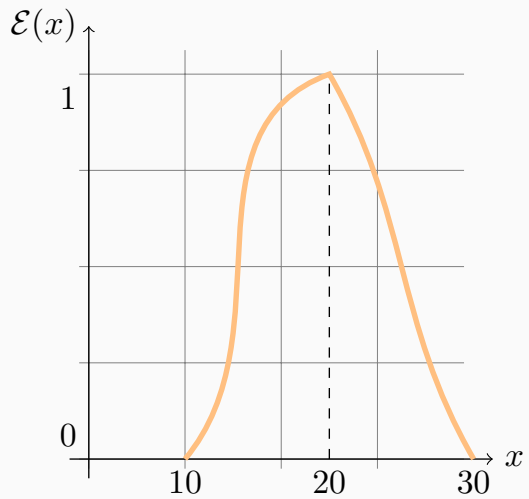
Fuzzy Numbers (2/6)

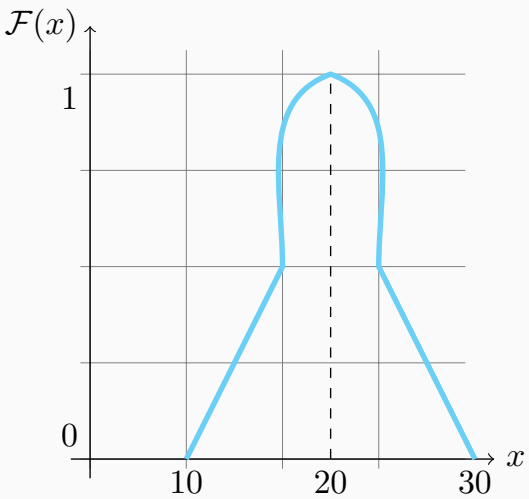












Fuzzy Numbers (3/6)

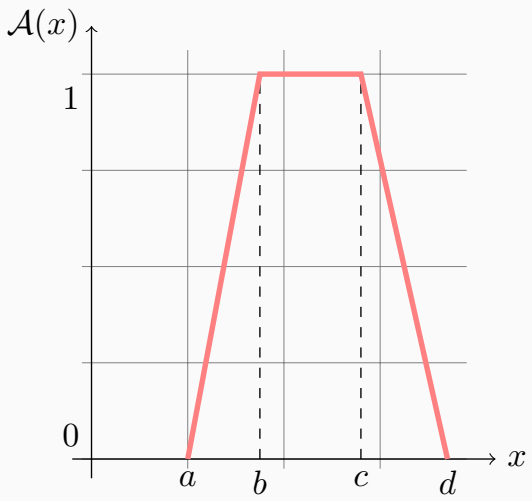
For a fuzzy membership function to qualify as a fuzzy number, it must capture our intuitive concept of a set of numbers around a given real number or interval of real numbers

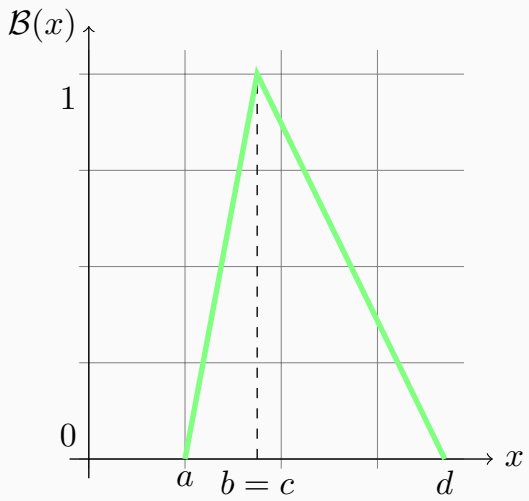
$$\mathcal{A}(x) = \begin{cases} f(x) & \text{for} \quad x \in [a,\ b] \\ 1 & \text{for} \quad x \in [b,\ c] \\ g(x) & \text{for} \quad x \in [c,\ d] \\ 0 & \text{for} \quad x < a \text{ or } x > d \end{cases} \tag{1}$$

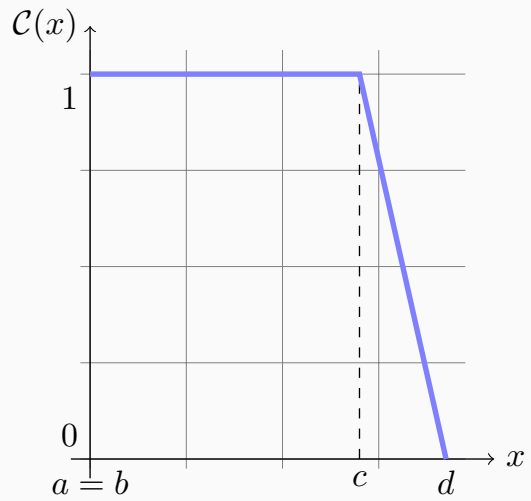
Common shapes of Fuzzy Numbers

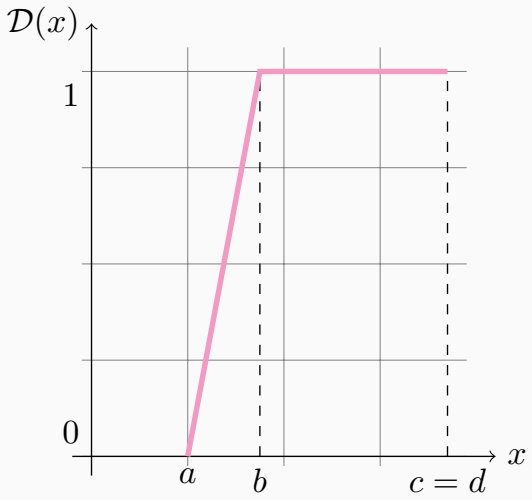
- Most common membership functions are trapezoidal and triangular *(easy to construct and manipulate)*
- Choice of a, b, c and d is important and is highly context-dependant
- Most applications not significantly affected by shapes of functions *(i.e., use linear shapes)*
- When some of real numbers (a, b, c, d) are equal, get degenerated forms of fuzzy numbers

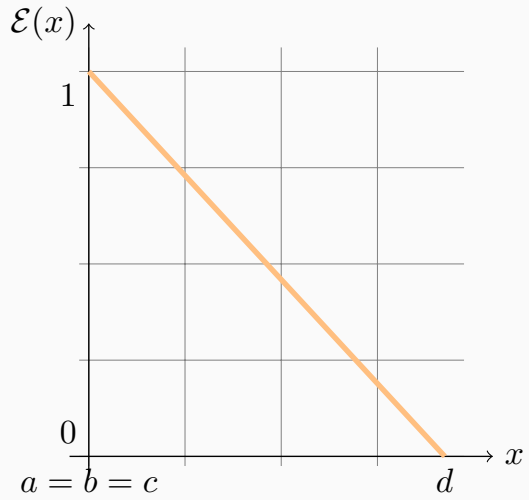
Fuzzy Numbers (4/6)

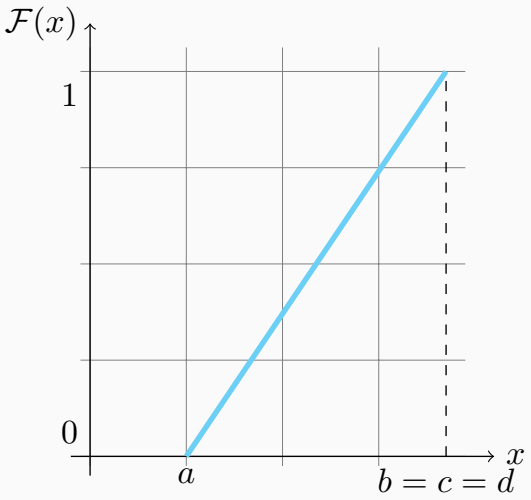






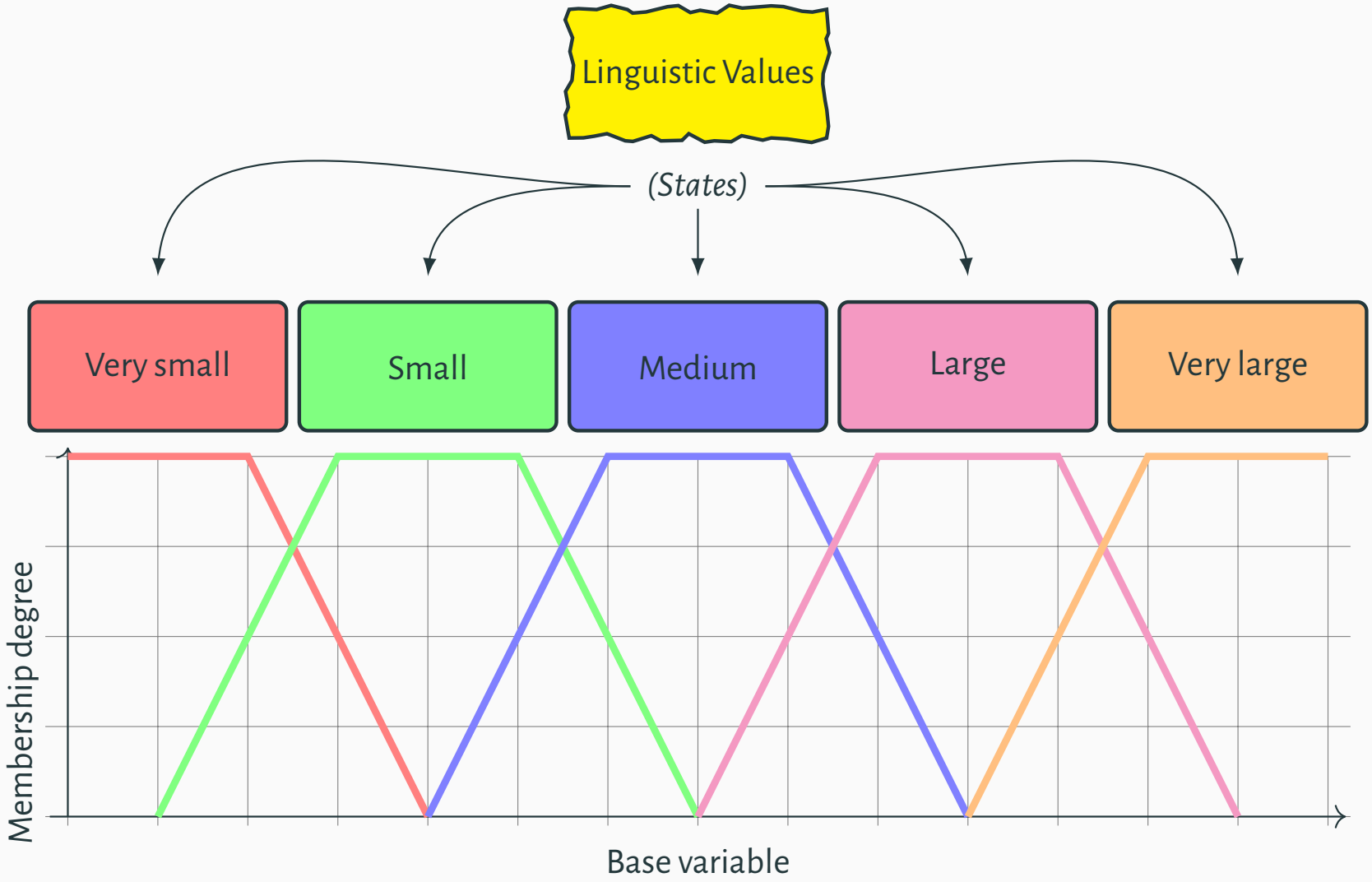






Fuzzy Numbers (5/6)

States are fuzzy numbers which represent linguistic concepts



Fuzzy Numbers (6/6)

1. Fuzzy numbers are normal fuzzy sets (height=1)
2. Fuzzy numbers are convex fuzzy sets
3. Support of every fuzzy number is open interval (a, d) of real numbers (support must be bounded)
4. Interval analysis can be used to define arithmetic operations on fuzzy numbers

Basic arithmetic operations:

- addition $[a,\ b] + [c,\ d] = [a + c,\ b + d]$
- Subtraction $[a,\ b] - [c,\ d] = [a - d,\ b - c]$
- Multiplication $[a,\ b] \times [c,\ d] = [\min(ac, ad, bc, bd),\ \max(ac, ad, bc, bd)]$
- Division[2] $[a,\ b] \div [c,\ d] = [a,\ b] \times [1/d,\ 1/c]$

[2]Interval division assumes that the number 0 is not one of the elements in the divisor interval [c, d].

Constructing Fuzzy Sets (1/5)



How would you assess today's temperature?

We can describe a parameter describing a phenomena *(e.g., Temperature for environment or Error for distance measurement)* using a finite, small number of descriptors, referred to as linguistic variables of parameter.

Temperature (T) {Cold, Average, Warm}

Error (E) {Small, Medium, Large}



The number of linguistic variables should be kept small (7 ± 2) due to our limited capacity to distinguish more. Commonly 3 to 5 linguistics variables are used in describing parameters.

Constructing Fuzzy Sets (2/5)



- Fuzzy sets offer an important and unique approach to describe linguistic variables
- Membership functions

$$\mathcal{A}(x) = \mathcal{X} \rightarrow [0, 1]$$

  are mathematical functions that are used to describe fuzzy sets
- Choosing membership functions require understanding of:
  - nature of the problem and parameter at hand
  - Level of details to be captured
  - Context of application

Prerequistes

- Concepts and linguistic values *(e.g., cold temperature)*
- Numerical measurements and/or linguistic assessments *(e.g., degrees Celsius)*
- Given context
- Data or Expert

Constructing Fuzzy Sets (3/5)

To construct fuzzy sets:

Expert-Driven Using developer, user, decision-maker, etc.

1. Direct methods
   - Answers to questions that explicitly pertain to the constructed membership function
   - Single or multiple experts
2. indirect methods
   - Simpler questions, easier to answer, less sensitive to subjective biases, pertain to membership function only implicitly
   - Single or multiple experts

Data-Driven Form data to fuzzy sets

Constructing Fuzzy Sets (4/5)

Direct Methods with Multiple Experts

Example

n experts were asked to validate the proposition "x **belongs to** A" as either true or false

True $a_i(x) = 1$

False $a_i(x) = 0$

where $i \in \{1 \cdots n\}$ denotes the ith expert.

$$A(x) = \frac{1}{n} \sum_{i=1}^{n} a_i(x)$$



Can also distinguish degrees of competence c_i of individual experts:

$$A(x) = \sum_{i=1}^{n} c_i a_i(x), \qquad \text{where} \quad \sum_{i=1}^{n} c_i = 1$$

Constructing Fuzzy Sets (5/5)

- Given 5 labourers $\{\mathfrak{L}1, \mathfrak{L}2, \mathfrak{L}3, \mathfrak{L}4, \mathfrak{L}5\}$
- Need to determine membership function **"A"** that captures linguistic term **"Excellent Labourer"**
- Ask 10 superintendents if particular person is excellent labourer *(answer either yes (1) or no (0))*
- For each labourer, calculate membership grade of belonging to fuzzy set **"A"** by taking ratio of total number of yes (1) to total number of responses.

	$\mathfrak{L}1$	$\mathfrak{L}2$	$\mathfrak{L}3$	$\mathfrak{L}4$	$\mathfrak{L}5$
$\mathfrak{E}$#1	1	1	1	1	1
$\mathfrak{E}$#2	0	0	1	1	1
$\mathfrak{E}$#3	0	1	0	1	0
$\mathfrak{E}$#4	1	0	1	1	1
$\mathfrak{E}$#5	0	0	1	1	1
$\mathfrak{E}$#6	0	1	1	1	1
$\mathfrak{E}$#7	0	0	0	0	0
$\mathfrak{E}$#8	1	1	1	1	1
$\mathfrak{E}$#9	0	0	0	1	0
$\mathfrak{E}$#10	0	0	0	1	0

$\Longrightarrow$ Opinions of individual experts must be aggregated

The resulting set would be: $A = 0.3/\mathfrak{L}1 + 0.4/\mathfrak{L}2 + 0.6/\mathfrak{L}3 + 0.9/\mathfrak{L}4 + 0.6/\mathfrak{L}5$

Tipping Problem

What should be the TIP at a restaurant, given the quality of FOOD and of SERVICE. These latter are represented by some scores ranging from 0 *(poor)* to 10 *(excellent)*.

Rule Base

1. FOOD is rancid $\vee$ SERVICE is poor $\longrightarrow$ TIP is cheap;
2. SERVICE is good $\longrightarrow$ TIP is average;
3. FOOD is delicious $\vee$ SERVICE is excellent $\longrightarrow$ TIP is generous.

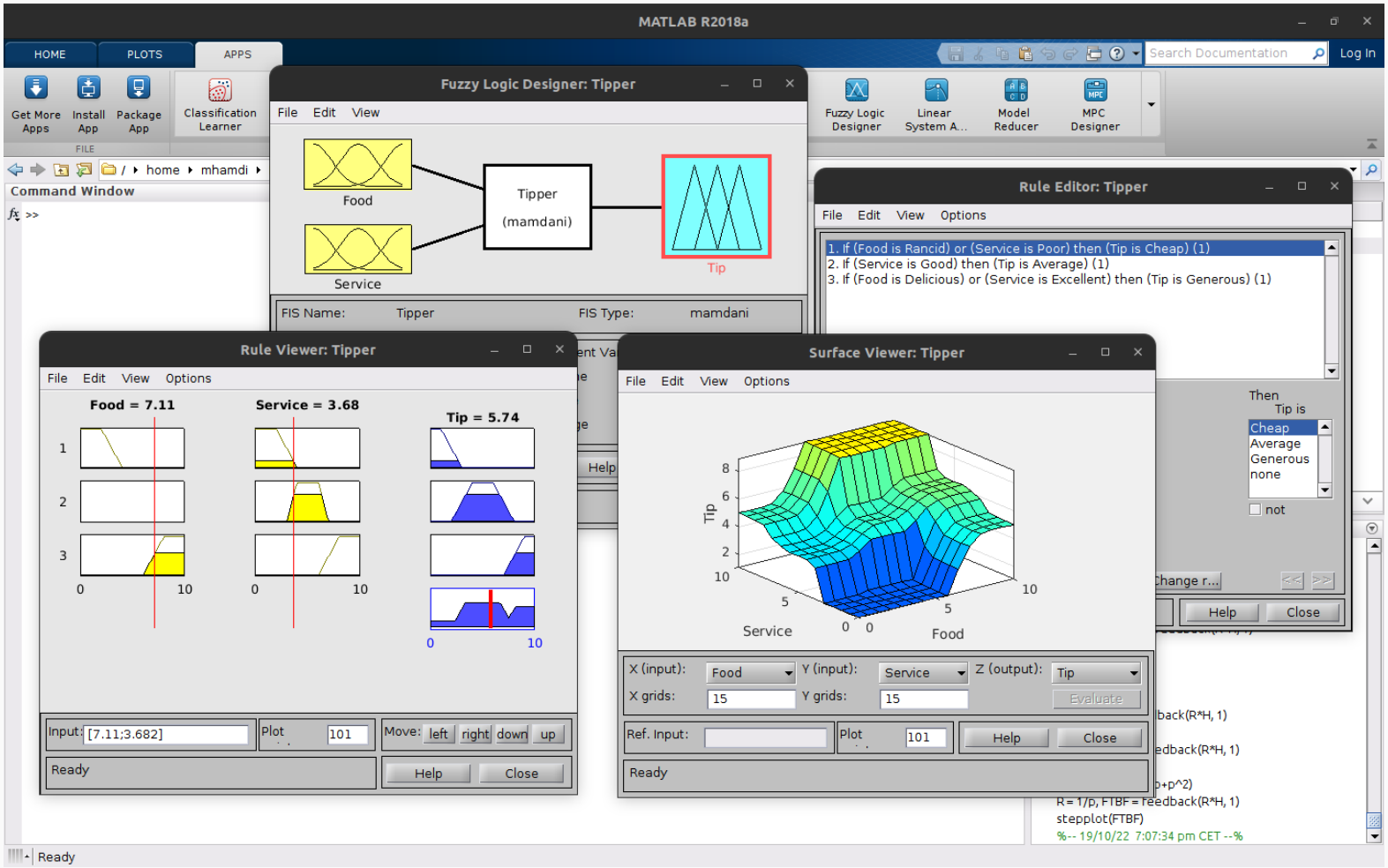
Tipping Problem

What should be the TIP at a restaurant, given the quality of FOOD and of SERVICE. These latter are represented by some scores ranging from 0 *(poor)* to 10 *(excellent)*.

Rule Base

1. FOOD is rancid $\vee$ SERVICE is poor $\longrightarrow$ TIP is cheap;
2. SERVICE is good $\longrightarrow$ TIP is average;
3. FOOD is delicious $\vee$ SERVICE is excellent $\longrightarrow$ TIP is generous.

Using Fuzzy Logic Toolbox



Code is available at https://github.com/a-mhamdi/cosnip/
⟶ *Matlab* ⟶ *Fuzzy* ⟶ *Tipper.fis*

Using Fuzzy.jl Package



The code is available @ github.com/a-mhamdi/jlai ⟶ *Codes* ⟶ *Julia* ⟶ *Part-1*

⟶ *Pluto* ⟶ *tipper.jl*

⟶ *Jupyter* ⟶ *tipper.ipynb*

Mamdani FIS

A fuzzy logic controller that uses linguistic rules to map input values to output values.

- was introduced by Ebrahim (Abe) H. Mamdani in 1975
- works using rules of linguistics, style like human concepts
  (more intuitive and easier to understand)
- well suited to applications where rules are inspired from human expert knowledge.

Example

We want to control the temperature of a living room using a thermostat.

Inputs
1. ε difference between the desired and measured temperatures
2. $\delta\varepsilon$ rate of change of the temperature error

Output heating/cooling command(H/C) is the recommended action to take *(e.g., heat, cool or no action)*

Fuzzy Rule Base:

$\mathfrak{R}_1$: If ε is "cold" AND $\delta\varepsilon$ is "rapidly decreasing" THEN H/C is "heat"

$\mathfrak{R}_2$: If ε is "hot" AND $\delta\varepsilon$ is "rapidly increasing" THEN H/C is "cool"

$\mathfrak{R}_3$: If ε is "near target" AND $\delta\varepsilon$ is "slow" THEN H/C is "no action"

Task #9

Consider a fuzzy logic system with two inputs u, v and an output w. We suppose that each variable ranges from $0 \longrightarrow 10$. w changes by a unit step. The membership functions of the fuzzy variables are described below.

- u can be:

  Negative (N) $\mathcal{L}(2,\ 4)$
  Zero (Z) $\Delta(3,\ 6,\ 9)$
  Positive (P) $\Gamma(6,\ 8)$.

- v can be:

  Negative (N) $\mathcal{L}(2,\ 5)$
  Zero (Z) $\Pi(2,\ 4,\ 6,\ 8)$
  Positive (P) $\Gamma(6,\ 8)$.

- w can be:

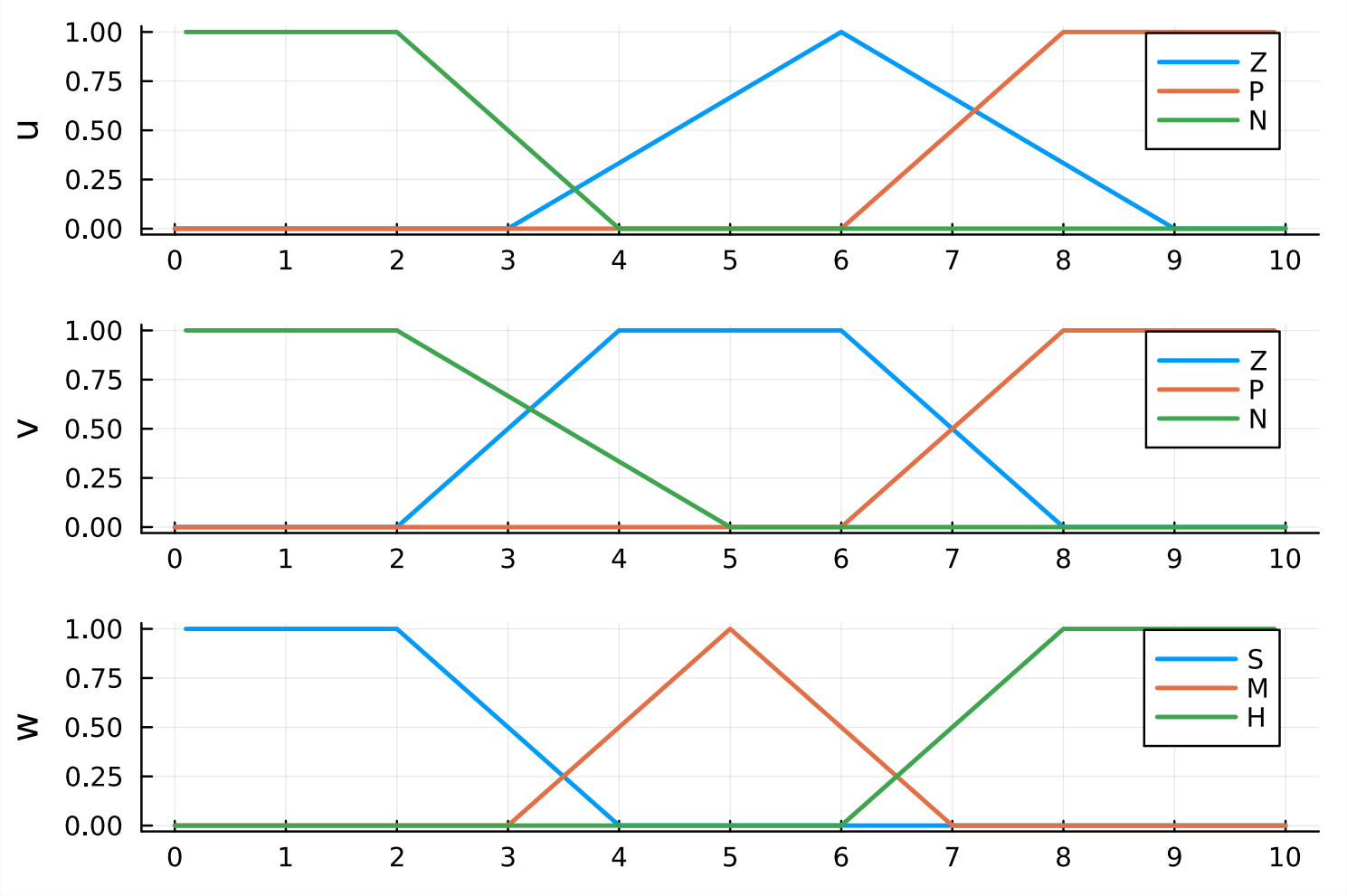
  Small (S) $\mathcal{L}(2,\ 4)$
  Medium (M) $\Delta(3,\ 5,\ 7)$
  High (H) $\Gamma(6,\ 8)$.

Rule Base - case of $\wedge$

v \ u	N	Z	P
N	S	S	M
Z	S	M	H
P	M	H	H



Evaluate
w if $u = 4$ & $v = 6$.



$$\begin{cases} \mu(4) = \{(\mathbf{N}, 0), (\mathbf{Z}, 1/3), (\mathbf{P}, 0)\} \\ \mu(6) = \{(\mathbf{N}, 0), (\mathbf{Z}, 1), (\mathbf{P}, 0)\} \end{cases} \Longrightarrow w^\star = \frac{3 \times 0 + 4 \times 1/3 + 5 \times 1/3 + 6 \times 1/3 + 7 \times 0}{0 + 1/3 + 1/3 + 1/3 + 0} = 5$$

Task #10[3]

Design a fuzzy lighting controller system, in which the control system dims the bulb light automatically according to the environmental light. Assume that the inputs to the system are the environmental light x_1 and the changing rate of the environmental light x_2. The output y represents the control value of the dimmer.

- x_1 ranges between 120 and 220 lumens. x_1 can be:

  Dark (D) $\mathcal{L}(130, 150)$
  Ambient (A) $\Pi(130, 150, 190, 210)$
  Light (L) $\Gamma(190, 210)$.

- x_2 ranges between –10 and +10. x_2 can be:

  Negative-Small (NS) $\mathcal{L}(-10, 0)$
  Zero (Z) $\Delta(-10, 0, 10)$
  Positive-Small (PS) $\Gamma(0, 10)$.

- y ranges between 0 and +10. y can be:

  Very-Small (VS) $\mathcal{L}(2, 4)$
  Small (S) $\Delta(2, 4, 6)$
  Big (B) $\Delta(4, 6, 8)$
  Very-Big (VB) $\Gamma(6, 8)$.

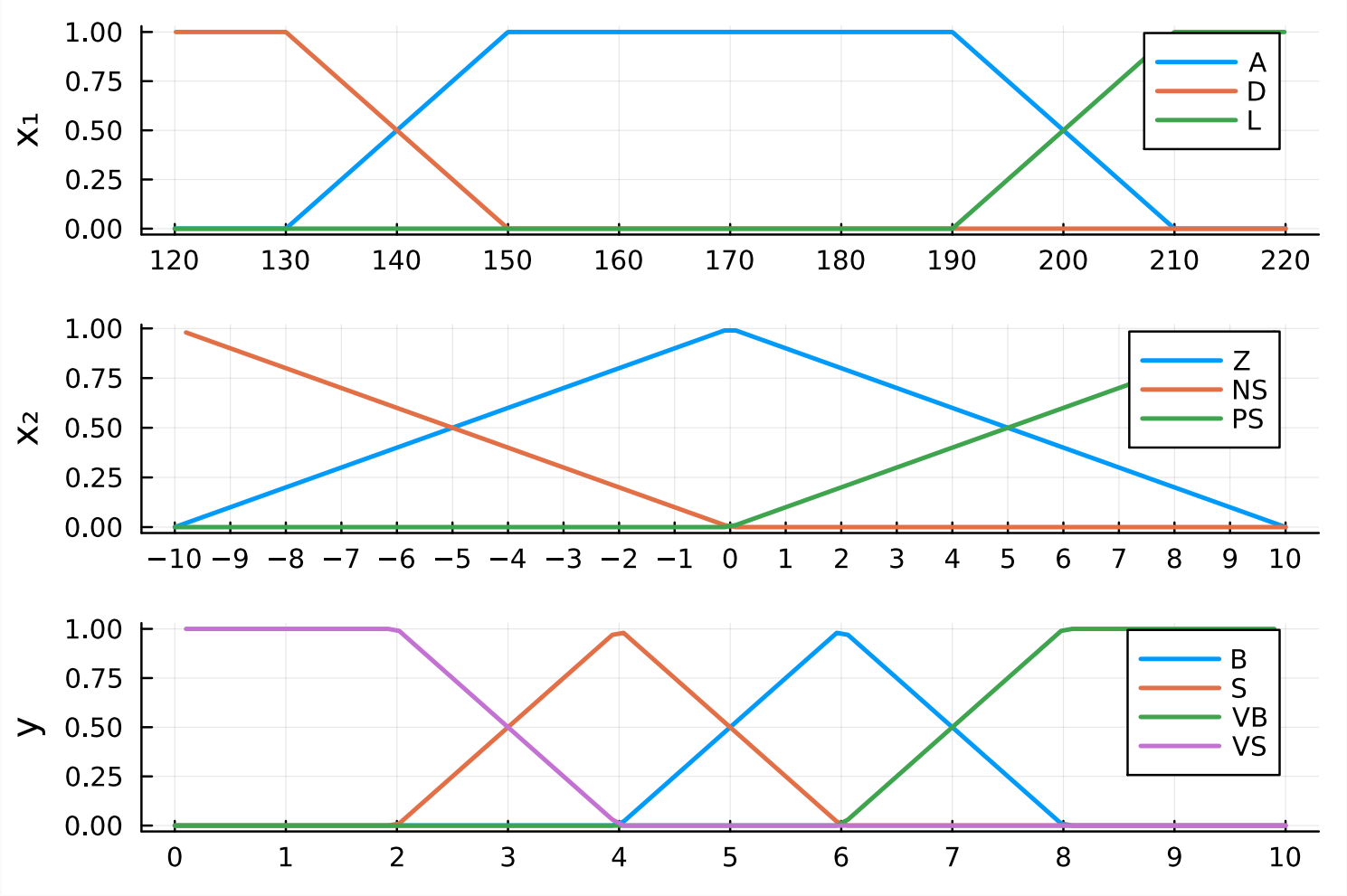
Rule Base - case of $\wedge$

x_2 \ x_1	D	A	L
NS	VB	B	B
Z	B	B	S
PS	B	S	VS



Evaluate
y if $x_1 = 125$ & $x_2 = -6$.

[3]Credit: Dr. Mohammed A. T.



$$\mu(125) = \{(\mathbf{D}, 1), (\mathbf{A}, 0), (\mathbf{L}, 0)\} \quad \text{and} \quad \mu(-6) = \{(\mathbf{NS}, 3/5), (\mathbf{Z}, 2/5), (\mathbf{PS}, 0)\}$$

$$\Longrightarrow y^\star = \frac{5 \times 2/5 + 6 \times 2/5 + 7 \times 1/2 + 8 \times 3/5 + 9 \times 3/5 + 10 \times 3/5}{2/5 + 2/5 + 3/5 + 1/2 + 3/5 + 3/5 + 3/5} \approx 7.775$$

Takagi-Sugeno-Kang (TSK) FIS

Each fuzzy rule has the following structure: "If (antecedent or premise) then (consequent)"

Antecedent composed by fuzzy sets defined by MF

Consequent represented by a polynomial function of the fuzzy inputs

Example

We want to design a TSK FIS to control the temperature of a room. Our system accepts two inputs: room temperature (T) and outdoor temperature (To). The output is the heating or cooling action (H/C).

$\mathfrak{R}_1$: If T is "cold" and To is "cold" then $H/C = \frac{1}{2}T + \frac{1}{3}To$

$\mathfrak{R}_2$: If T is "cold" and To is "hot" then $H/C = \frac{1}{5}T + \frac{3}{5}To$

$\mathfrak{R}_3$: If T is "hot" and To is "cold" then $H/C = -\frac{4}{5}T + \frac{1}{4}To$

- ▲ Simple and efficient defuzzification
- ▲ Interpolation *(incomplete or sparse rule bases)*
- ▲ Smooth and continuous output
- ▼ Limited rule complexity
- ▼ Tuning *(careful tuning of the fuzzy rule base)*

Task #11

Suppose we have three fuzzy predicates: $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ described by these trapezoidal fuzzy sets:

$$\mathcal{A} \quad \Pi\left(0,\ 2,\ 5,\ 9\right)$$
$$\mathcal{B} \quad \Pi\left(2,\ 8,\ 13,\ 16\right)$$
$$\mathcal{C} \quad \Pi\left(11,\ 16,\ 19,\ 19\right)$$

x and y are fuzzy variables, each one ranges between 0 and 19. Given the following three rules:

$$\mathfrak{R}_1 \quad (x \text{ is } \mathcal{A}) \wedge (y \text{ is } \mathcal{C}) \longrightarrow u = 10$$
$$\mathfrak{R}_2 \quad \neg(x \text{ is } \mathcal{A}) \vee (y \text{ is } \mathcal{B}) \longrightarrow u = 2$$
$$\mathfrak{R}_3 \quad (x \text{ is } \mathcal{B}) \wedge \neg(y \text{ is } \mathcal{C}) \longrightarrow u = 5$$

Compute the degree of satisfaction for each case:

① $x_1 = 5 \,\&\, y_1 = 12$ ② $x_2 = 0 \,\&\, y_2 = 15$ ③ $x_3 = 7 \,\&\, y_3 = 13$

$x_1 = 5 \;\&\; y_1 = 12$

$\mathfrak{R}_1$

$\mu_{\mathcal{A}}(x_1) \min \mu_C(y_1)$

$1 \min 1/5 = 1/5$

$\mathfrak{R}_2$

$\left[1 - \mu_{\mathcal{A}}(x_1)\right] \max \mu_{\mathcal{B}}(y_1)$

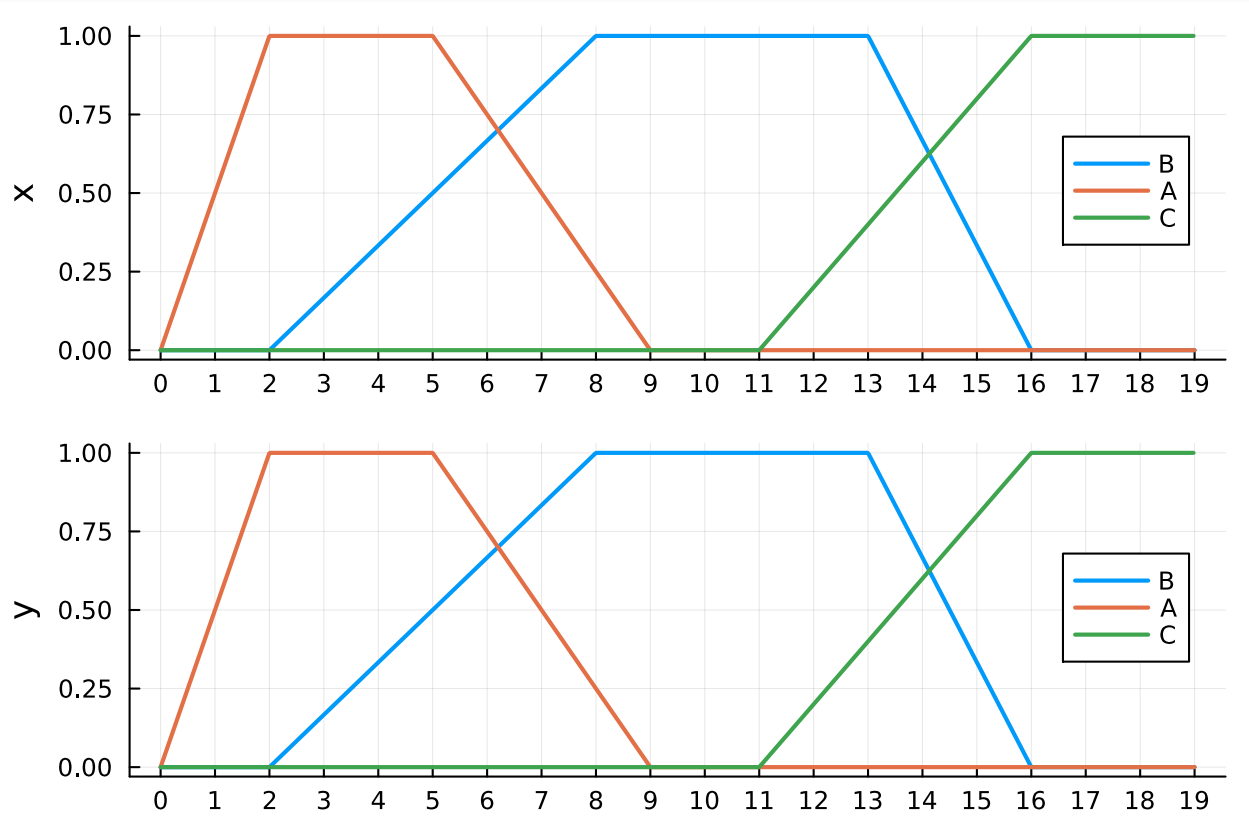
$0 \max 1 = 1$

$\mathfrak{R}_3$

$\mu_{\mathcal{B}}(x_1) \min \left[1 - \mu_C(y_1)\right]$

$1/2 \min 4/5 = 1/2$

$u_1 = 3.82$



$x_2 = 0 \;\&\; y_2 = 15$

$\Re_1$

$\mu_{\mathcal{A}}(x_2) \min \mu_C(y_2)$

$0 \min _ = 0$

$\Re_2$

$\left[1 - \mu_{\mathcal{A}}(x_2)\right] \max \mu_{\mathcal{B}}(y_2)$

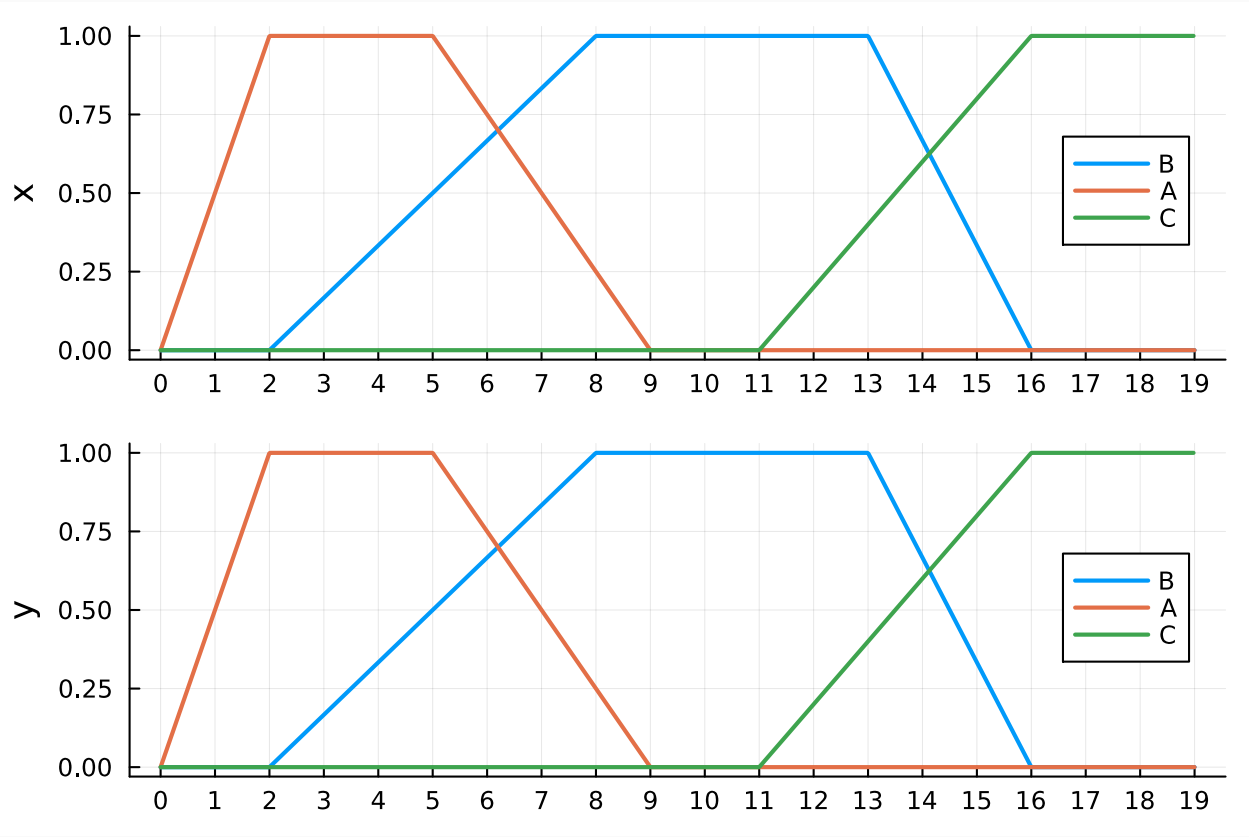
$1 \max _ = 1$

$\Re_3$

$\mu_{\mathcal{B}}(x_2) \min \left[1 - \mu_C(y_2)\right]$

$0 \min _ = 0$

$u_2 = 2$



$x_3 = 7 \;\&\; y_3 = 13$

$\Re_1$

$\mu_{\mathcal{A}}(x_3) \min \mu_C(y_3)$

$1/2 \min 2/5 = 2/5$

$\Re_2$

$\left[1 - \mu_{\mathcal{A}}(x_3)\right] \max \mu_{\mathcal{B}}(y_3)$

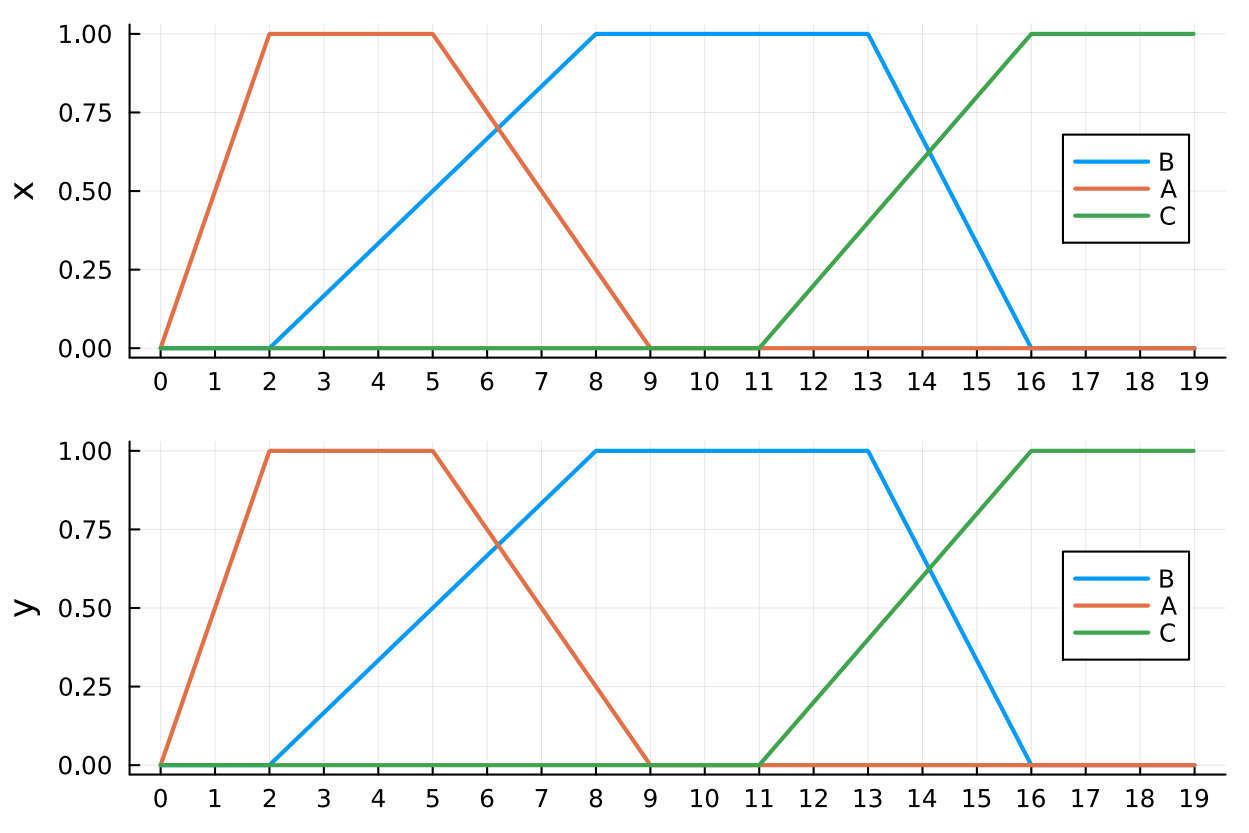
$1/2 \max 1 = 1$

$\Re_3$

$\mu_{\mathcal{B}}(x_3) \min \left[1 - \mu_C(y_3)\right]$

$5/6 \min 3/5 = 3/5$

$u_3 = 4.5$



Task #12[4]

Let us consider a TSK fuzzy model with the input $(x_1,\ x_2)$ and a single output y. Evaluate $y^\star$ for $(x_1,\ x_2) = (1.5,\ 0.5)$. The rules are:

$$\mathfrak{R}_1 \quad \text{IF } x_1 \text{ is } S \text{ AND } x_2 \text{ is } S \text{ THEN } y = -x_1 + x_2 + 1$$

$$\mathfrak{R}_2 \quad \text{IF } x_1 \text{ is } S \text{ AND } x_2 \text{ is } L \text{ THEN } y = -x_2 + 3$$

$$\mathfrak{R}_3 \quad \text{IF } x_1 \text{ is } L \text{ AND } x_2 \text{ is } S \text{ THEN } y = -x_1 + 3$$

$$\mathfrak{R}_4 \quad \text{IF } x_1 \text{ is } L \text{ AND } x_2 \text{ is } L \text{ THEN } y = -x_1 + x_2 + 2$$

The fuzzy predicates S and L are described by these membership functions:

S $\mathcal{L}\,(-2,\ 2)$ $\qquad$ L $\Gamma\,(-2,\ 2)$

[4] Credit: Dr. Hashim A. H.

$x_1 = 1.5 \;\&\; x_2 = 0.5$

$\Re_1$

$\mu_{\mathcal{S}}(x_1) \min \mu_{\mathcal{S}}(x_2)$

$0.125 \min 0.375 = 0.125$

$(y = 0)$

$\Re_2$

$\mu_{\mathcal{S}}(x_1) \min \mu_{\mathcal{L}}(x_2)$

$0.125 \min 0.625 = 0.125$

$(y = 2.5)$

$\Re_3$

$\mu_{\mathcal{L}}(x_1) \min \mu_{\mathcal{S}}(x_2)$

$0.875 \min 0.375 = 0.375$

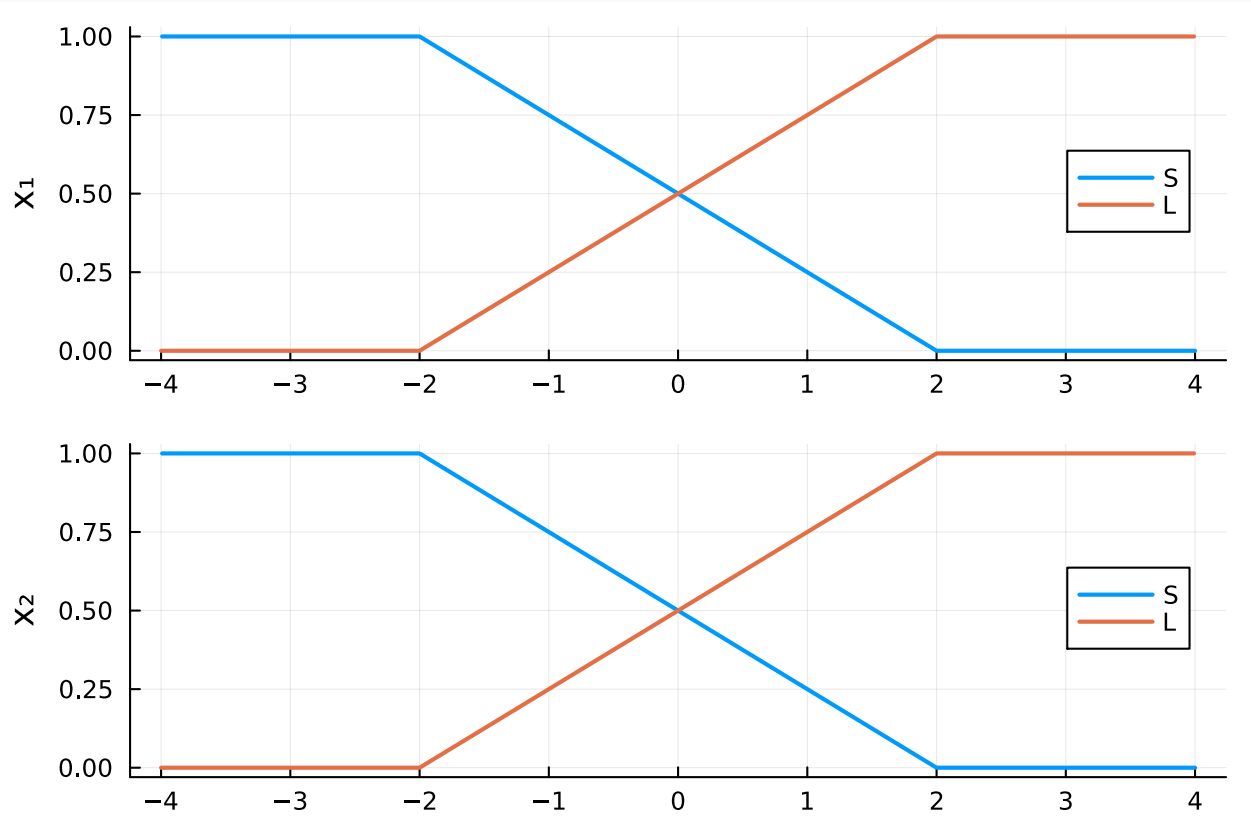
$(y = 1.5)$

$\Re_4$

$\mu_{\mathcal{L}}(x_1) \min \mu_{\mathcal{L}}(x_2)$

$0.875 \min 0.625 = 0.625$

$(y = 1)$



$$y^\star = \frac{0 \times 0.125 + 2.5 \times 0.125 + 1.5 \times 0.375 + 1 \times 0.625}{0.125 + 0.125 + 0.375 + 0.625} = 1.2$$

Tsukamoto FIS

A type of fuzzy logic-based system developed by Tsukamoto in the 1970s. The system would compute the crisp output for each rule based on the firing strength and then take the weighted average to produce the final performance evaluation.

In this method, for each fired rule i with firing strength α_i, the representative output value $y_i(\alpha_i)$ is determined as the value where the consequent membership function $\mu_{y_i}(y) = \alpha_i$.

Due to the **monotonicity** of μ_{y_i}, there exists a unique $y_i(\alpha_i)$.

The crisp output $y^\star$ is then computed as the weighted average of each rule's output:

$$y^\star = \frac{\sum_{i=1}^{n} \alpha_i \, y_i(\alpha_i)}{\sum_{i=1}^{n} \alpha_i}$$

- ▲ Simplified Rule Base
- ▲ Faster Computation
- ▼ Limited Transparency
- ▼ Limited Flexibility

Example

We want to evaluate employee performance based on two criteria:

① job satisfaction and ② productivity

We can define three fuzzy sets for each criterion: "low" (**L**), "medium" (**M**), and "high" (**H**)

Fuzzy Rule Base:

$\mathfrak{R}_1$: If job satisfaction is **L**, then performance is **L**.

$\mathfrak{R}_2$: If job satisfaction is **M** and productivity is **L**, then performance is **L**.

$\mathfrak{R}_3$: If job satisfaction is **M** and productivity is **M**, then performance is **H**.

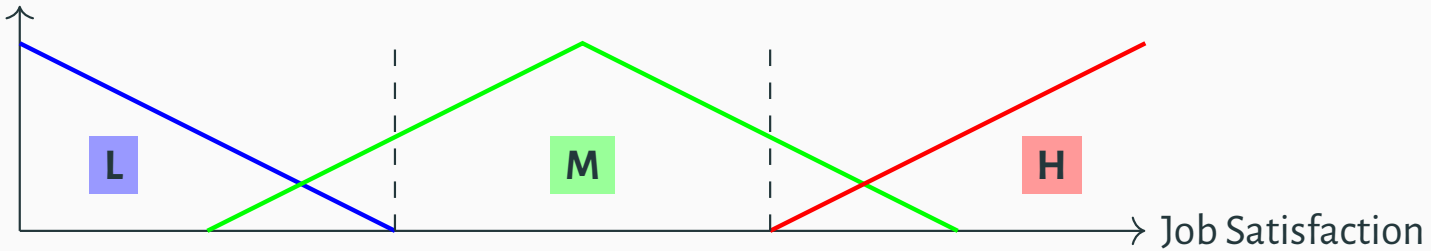
$\mathfrak{R}_4$: If job satisfaction is **M** and productivity is **H**, then performance is **H**.

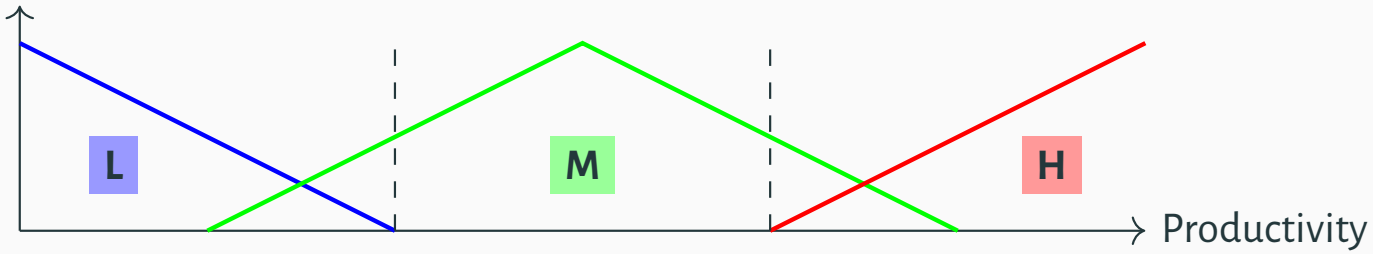
$\mathfrak{R}_5$: If job satisfaction is **H** and productivity is **L**, then performance is **L**.

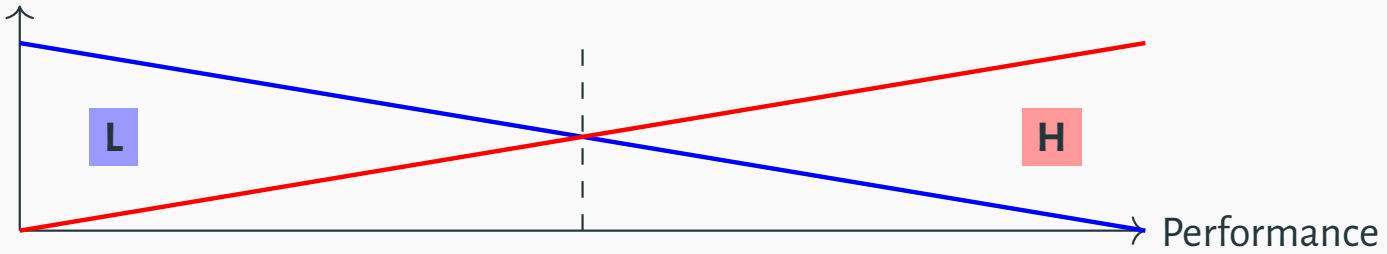
$\mathfrak{R}_6$: If job satisfaction is **H** and productivity is **M**, then performance is **H**.

$\mathfrak{R}_7$: If job satisfaction is **H** and productivity is **H**, then performance is **H**.

1. Each rule will compute a firing strength (minimum of input memberships), and then use the inverse of the output membership function to get a crisp value.
2. The final output is a weighted average of all rule outputs.







The method assumes that each consequent fuzzy set $\mathcal{Y}_i$ has a monotonic membership function (either decreasing or increasing), ensuring a one-to-one correspondence between the firing level α_i and the output value $y_i(\alpha_i)$.

Task #13

Consider a simple fuzzy logic controller for a heater system. The input is the **error** (difference between desired and actual temperature, in °C), ranging from –10 to 10. The output is the **heating power** (in %), ranging from 0 to 100.

Input Fuzzy Sets (for Error):

- **Negative ($\mathcal{N}$)**: Left open shoulder membership function with peaks at
  $$-10\ (\mu = 1), -5\ (\mu = 1), 0\ (\mu = 0).$$
- **Zero ($\mathcal{Z}$)**: Triangular membership function with peaks at
  $$-5\ (\mu = 0), 0\ (\mu = 1), 5\ (\mu = 0).$$
- **Positive ($\mathcal{P}$)**: Right open shoulder membership function with peaks at
  $$0\ (\mu = 0), 5\ (\mu = 1), 10\ (\mu = 1).$$

Output Fuzzy Sets (for Heating Power, y):

Each is a singleton (crisp value) for Tsukamoto's method, assuming monotonicity:

$\mathfrak{R}_1$ If error is $\mathcal{N}$, then y is Low Heat $\left(y_1 = \mu_1^{(-1)}(\alpha_1) = 25 - 25 \times \alpha_1\right)$.

$\mathfrak{R}_2$ If error is $\mathcal{Z}$, then y is Medium Heat $\left(y_2 = \mu_2^{(-1)}(\alpha_2) = 30 + 50 \times \alpha_2\right)$.

$\mathfrak{R}_3$ If error is $\mathcal{P}$, then y is High Heat $\left(y_3 = \mu_3^{(-1)}(\alpha_3) = 60 + 40 \times \alpha_3\right)$.

Given Crisp Input: error $= -3^\circ$C.

(a) Compute the membership degrees for the input fuzzy sets: $\mu_{\mathcal{N}}(-3), \mu_{\mathcal{Z}}(-3), \mu_{\mathcal{P}}(-3)$.

(b) Determine the fired rules and their output membership strengths (α_i).

(c) Apply Tsukamoto's defuzzification to compute the crisp output heating power $y^\star$.

Tsukamoto's formula:

$$y^\star = \frac{\sum \alpha_i \cdot y_i}{\sum \alpha_i},$$

where y_i is the crisp output for rule i, and α_i is the strength of rule i (min of antecedents).

Part (a): Membership Degrees For an error $= -3^\circ$C:

- $\mu_{\mathcal{N}}(-3) = 0.6$
- $\mu_{\mathcal{Z}}(-3) = 0.4$
- $\mu_{\mathcal{P}}(-3) = 0$

Part (b): Fired Rules and Strengths (α_i)

$\mathfrak{R}_1$ IF $\mathcal{N}, \alpha_1 = \mu_{\mathcal{N}}(-3) = 0.6 \longrightarrow y_1 = 10.$

$\mathfrak{R}_2$ IF $\mathcal{Z}, \alpha_2 = \mu_{\mathcal{Z}}(-3) = 0.4 \longrightarrow y_2 = 50.$

$\mathfrak{R}_3$ IF $\mathcal{P}, \alpha_3 = \mu_{\mathcal{P}}(-3) = 0 \longrightarrow$ Not fired.

Part (c): TSUKAMOTO's Defuzzification

$$y^* = \frac{\alpha_1 \times y_1 + \alpha_2 \times y_2}{\alpha_1 + \alpha_2} = \frac{0.6 \times 10 + 0.4 \times 50}{0.6 + 0.4} = \frac{6 + 20}{1} = 26\%.$$

The crisp heating power output is 26%.

A slightly negative error suggests low heat, weighted toward 10% but pulled up by the $\mathcal{Z}$ membership.

Applications of Fuzzy Logic

Control Systems:

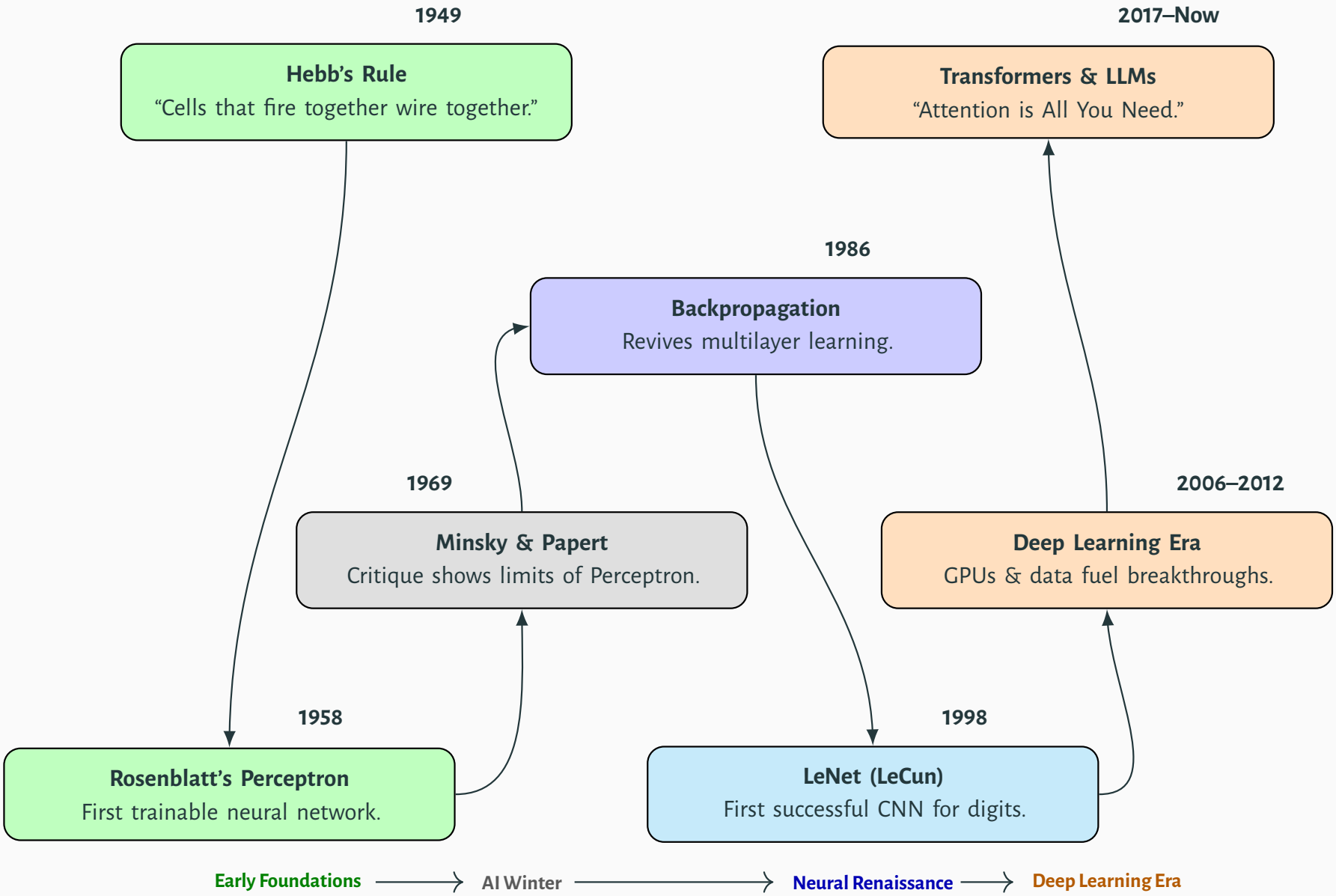
- Air conditioning
- Washing machines
- Automotive systems
- Robotics

Decision Support:

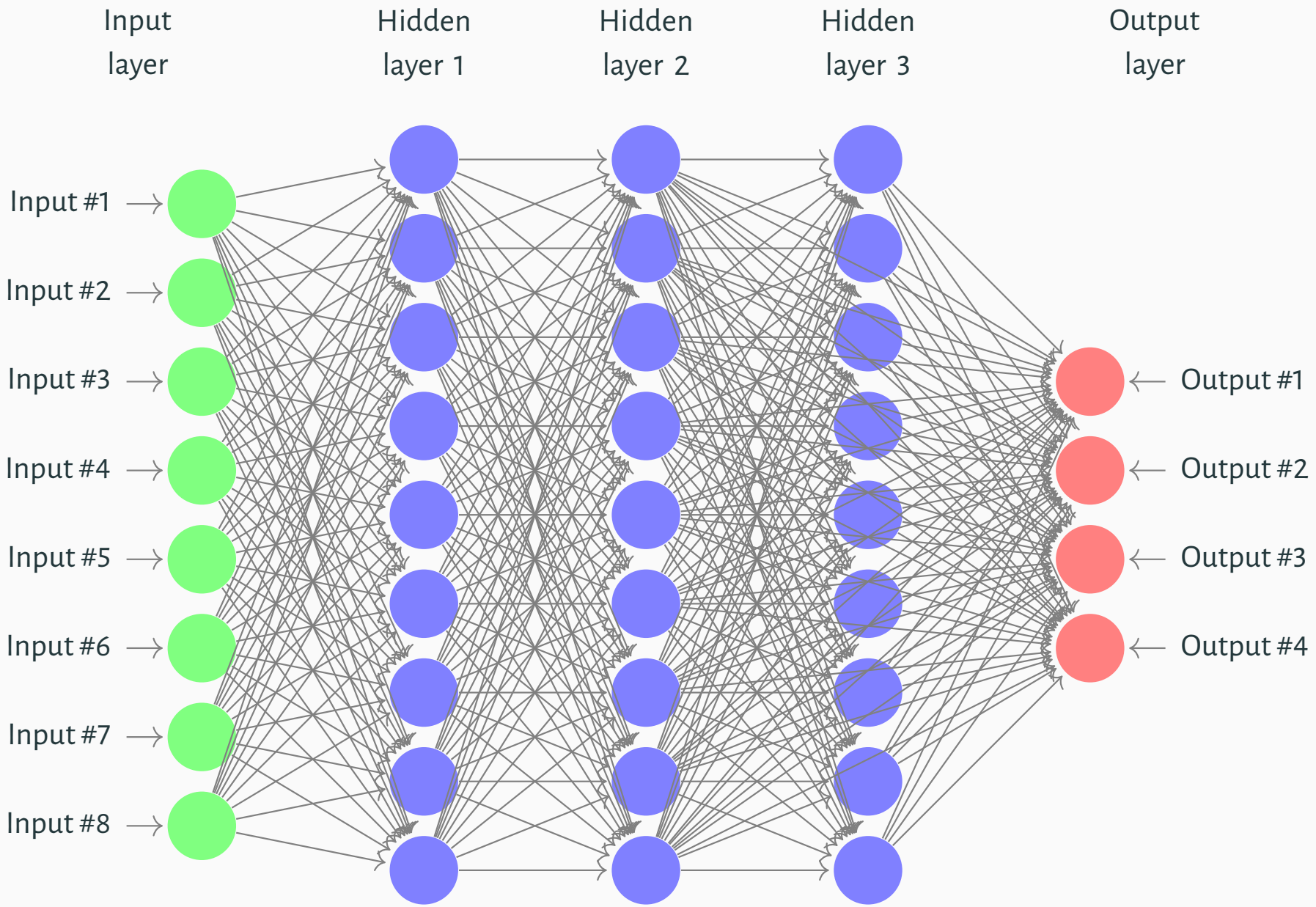
- Medical diagnosis
- Financial analysis
- Pattern recognition
- Expert systems

Neural Networks

Timeline of Neural Network Milestones

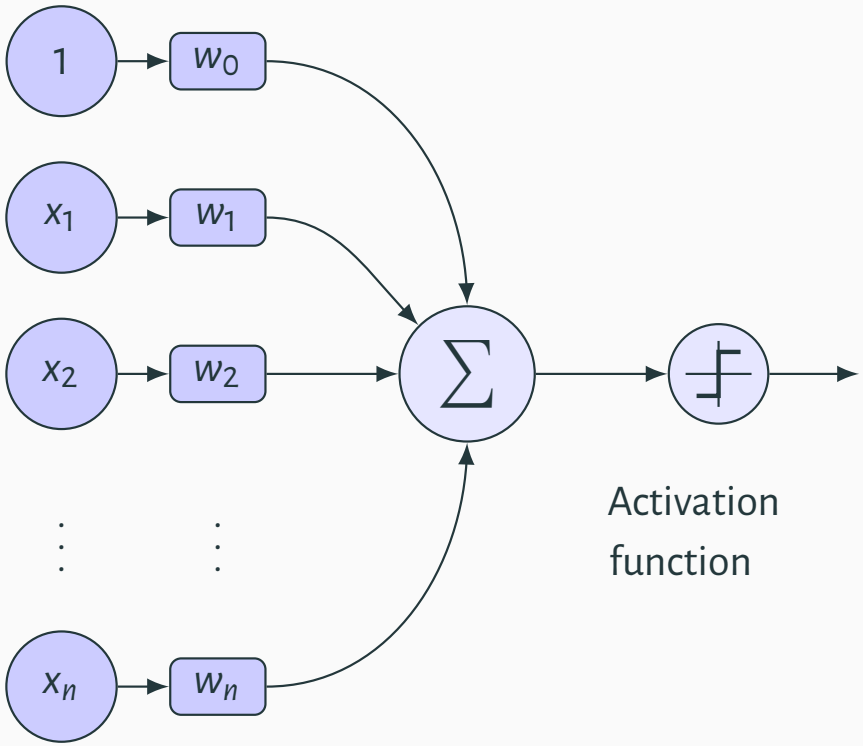


General Architecture

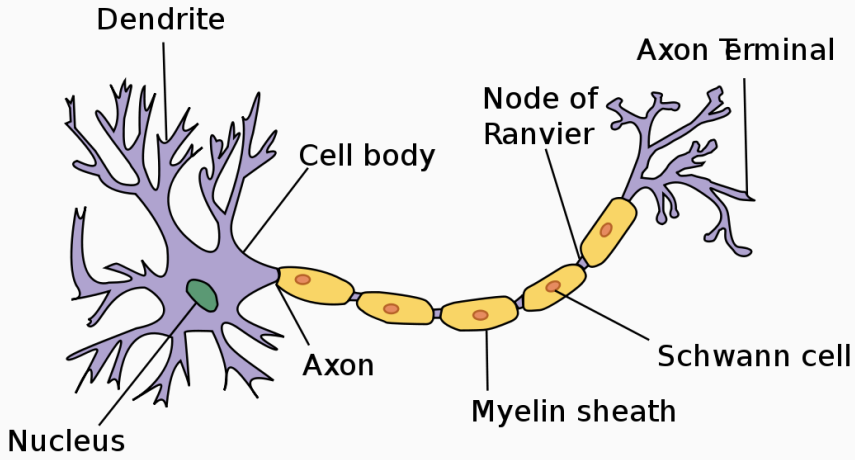


Fundamental unit of a neural network (1/4)

Artificial neuron



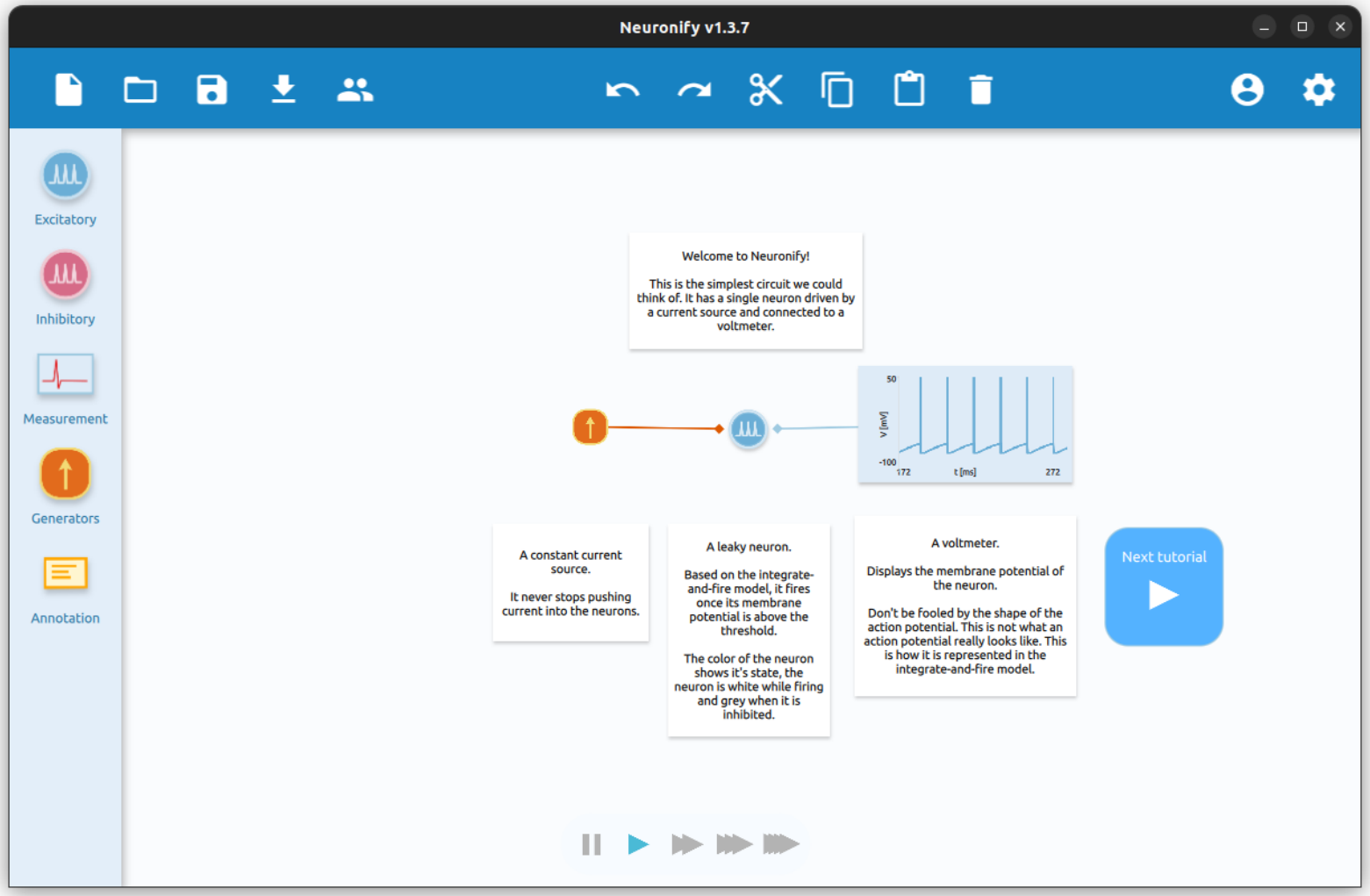
Biological neuron



https://id.wikipedia.org/wiki/Sel_saraf

Fundamental unit of a neural network (2/4)

Neural simulation



http://ovilab.net/neuronify/

Fundamental unit of a neural network (3/4)

Origin

- Proposed by **Warren McCulloch** (neuroscientist) and **Walter Pitts** (logician) in 1943;
- Published in the paper *"A Logical Calculus of the Ideas Immanent in Nervous Activity"*;
- Marked the beginning of computational neuroscience.

Mechanism

$$y = \begin{cases} 1 & \text{if } \sum_i w_i x_i \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

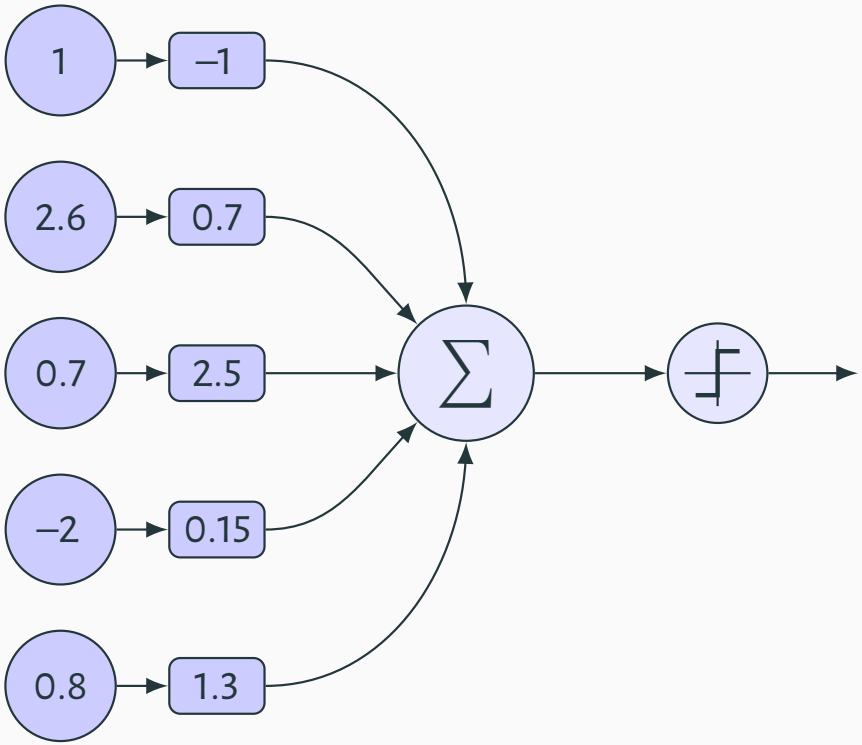
Properties

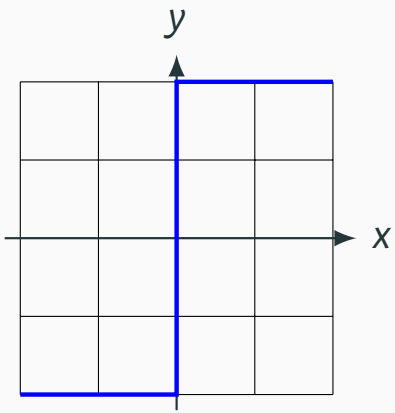
- ⋆ A simple mathematical model of a neuron;
- ⋆ Processes binary inputs (0 or 1) to produce a binary output;
- ⋆ Uses weighted summation of inputs;
- ⋆ Activates (outputs 1) if the sum meets or exceeds a threshold (θ).

McCulloch, W. S. and Pitts, W. (1943) *A logical calculus of the ideas immanent in nervous activity.* **The bulletin of mathematical biophysics.** 5 pp. 115–133.

Fundamental unit of a neural network (4/4)

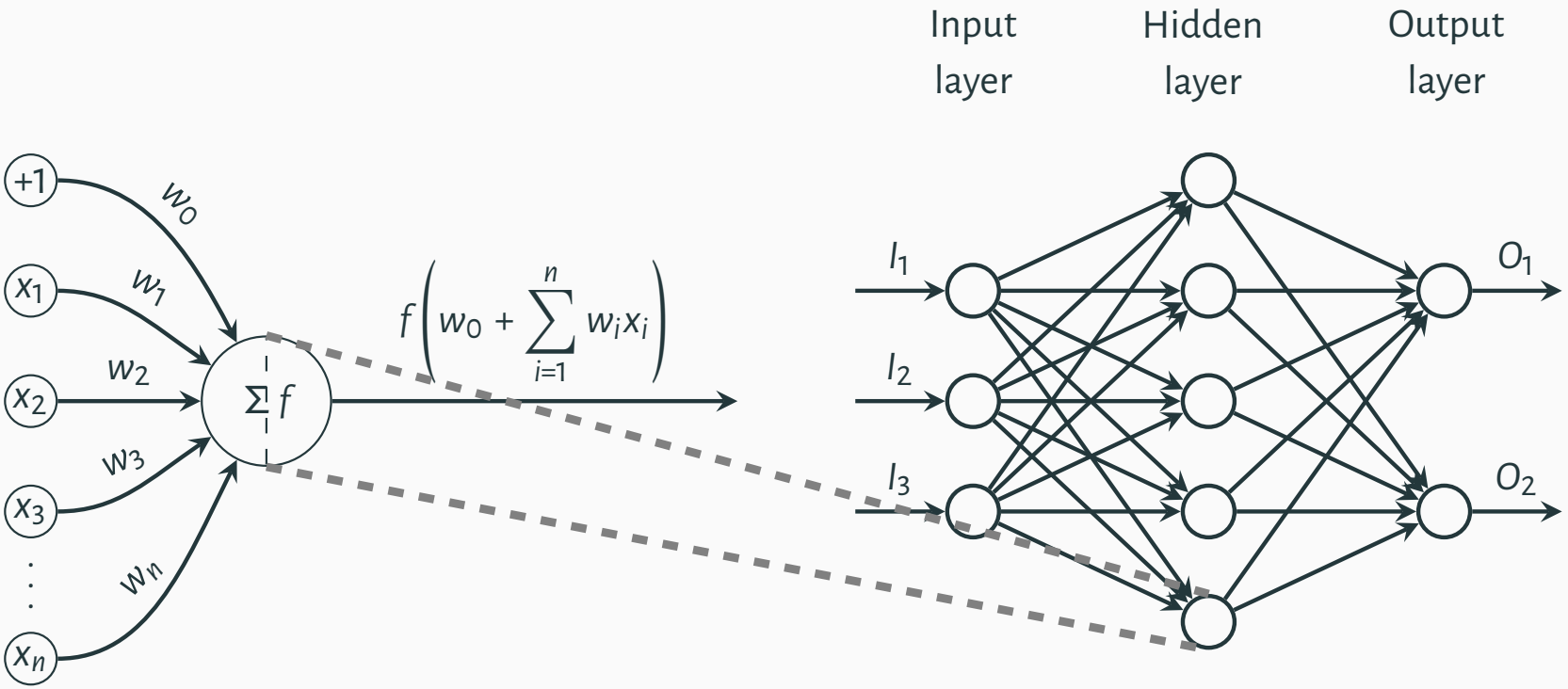
Task #14
Compute the output of the following neuron.





$$y = \text{sign}(1 \times -1 + 2.6 \times 0.7 + 0.7 \times 2.5 - 2 \times 0.15 + 0.8 \times 1.3) = 1$$

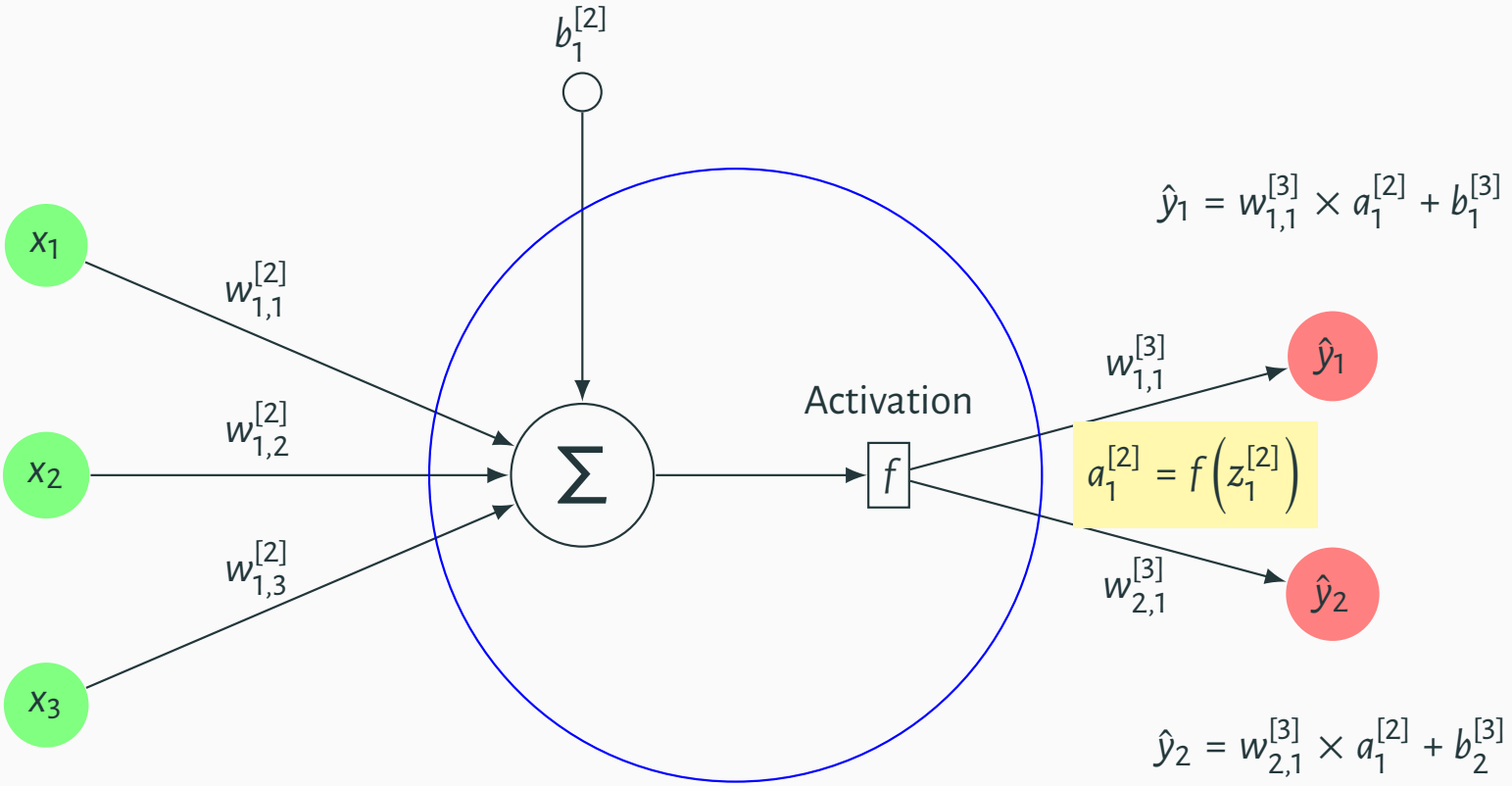
Multilayer Perceptron (MLP) (1/2)



Task #15
For the above structure, determine how many parameters are to be adjusted.

$$\#\,\text{params} \;=\; 5 \times 3 + 5 + 2 \times 5 + 2 \;=\; 32$$

Multilayer Perceptron (MLP) (2/2)



$$z_1^{[2]} = \underbrace{w_{1,1}^{[2]} \times x_1 + w_{1,2}^{[2]} \times x_2 + w_{1,3}^{[2]} \times x_3}_{\mathbf{w}_1^{[2]T}\mathbf{x}} + b_1^{[2]}$$

Task #16

Given the following weight matrices and biases vectors. Draw the corresponding neural network architecture. *(Place the values of the synaptic weights and biases on the arrows.)*

$$\mathcal{W}^{[1]} = \begin{bmatrix} 0 & 7.14 & 8 & 4 \\ 1 & -5 & 5.12 & -1 \\ -1 & 0.35 & -2 & 0.15 \end{bmatrix} \quad \text{and} \quad \mathbf{b}^{[1]} = \begin{bmatrix} -1.2 \\ 3 \\ 0.9 \end{bmatrix}$$

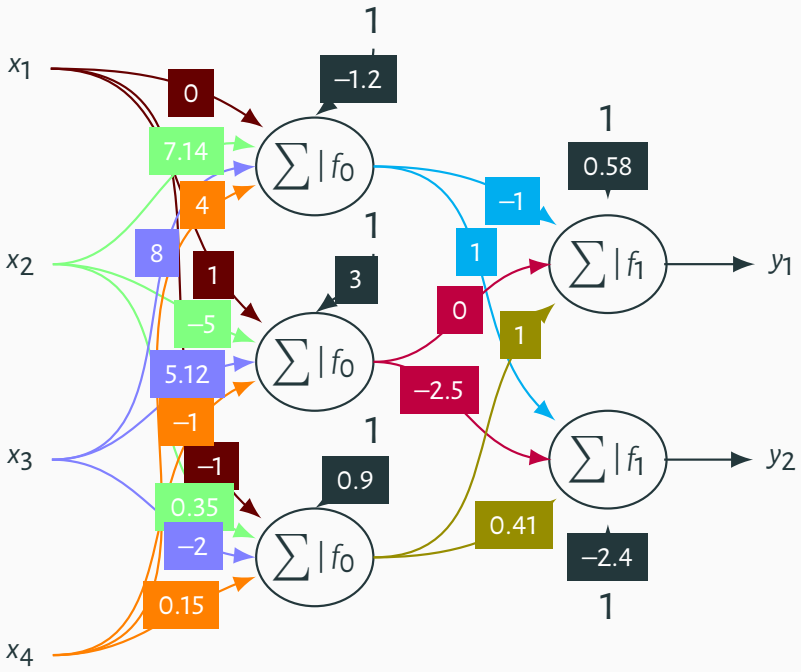
$$\mathcal{W}^{[2]} = \begin{bmatrix} -1 & 0 & 1 \\ 1 & -2.5 & 0.41 \end{bmatrix} \quad \text{and} \quad \mathbf{b}^{[2]} = \begin{bmatrix} 0.58 \\ -2.4 \end{bmatrix}$$

Task #16

Given the following weight matrices and biases vectors. Draw the corresponding neural network architecture. *(Place the values of the synaptic weights and biases on the arrows.)*

$$\mathcal{W}^{[1]} = \begin{bmatrix} 0 & 7.14 & 8 & 4 \\ 1 & -5 & 5.12 & -1 \\ -1 & 0.35 & -2 & 0.15 \end{bmatrix} \quad \text{and} \quad \mathbf{b}^{[1]} = \begin{bmatrix} -1.2 \\ 3 \\ 0.9 \end{bmatrix}$$

$$\mathcal{W}^{[2]} = \begin{bmatrix} -1 & 0 & 1 \\ 1 & -2.5 & 0.41 \end{bmatrix} \quad \text{and} \quad \mathbf{b}^{[2]} = \begin{bmatrix} 0.58 \\ -2.4 \end{bmatrix}$$



Hebbian Learning Rule

Origin Proposed by **Donald Hebb** (1949), summarized as:

"Cells that fire together wire together."

Mechanism

$$\Delta w_{ij} = \eta \cdot x_i \cdot y_j$$

where:

- Δw_{ij}: Change in the weight between neurons i and j
- η: Learning rate
- x_i: Activation of neurons i
- y_j: True label

Properties

- ⋆ Biologically inspired and unsupervised;
- ⋆ Encourages associative memory and pattern recognition;
- ⋆ No normalization mechanism; weights can grow unbounded.

Hebb D. (1949). *The organisation of behaviour: A Neuropsychological Theory* **Psychology Press.**

Rosenblatt Learning Rule (Perceptron Learning Rule)

Origin Introduced by **Frank Rosenblatt** (1958) with the Perceptron model.

Mechanism

$$\Delta w_{ij} = \eta \cdot x_i \cdot (y_j - \hat{y}_j)$$

where:

- Δw_{ij}: Change in the weight for feature i and output j
- η: Learning rate
- x_i: Input feature value
- y_j: True label (target output)
- $\hat{y}_j$: Predicted label

Properties

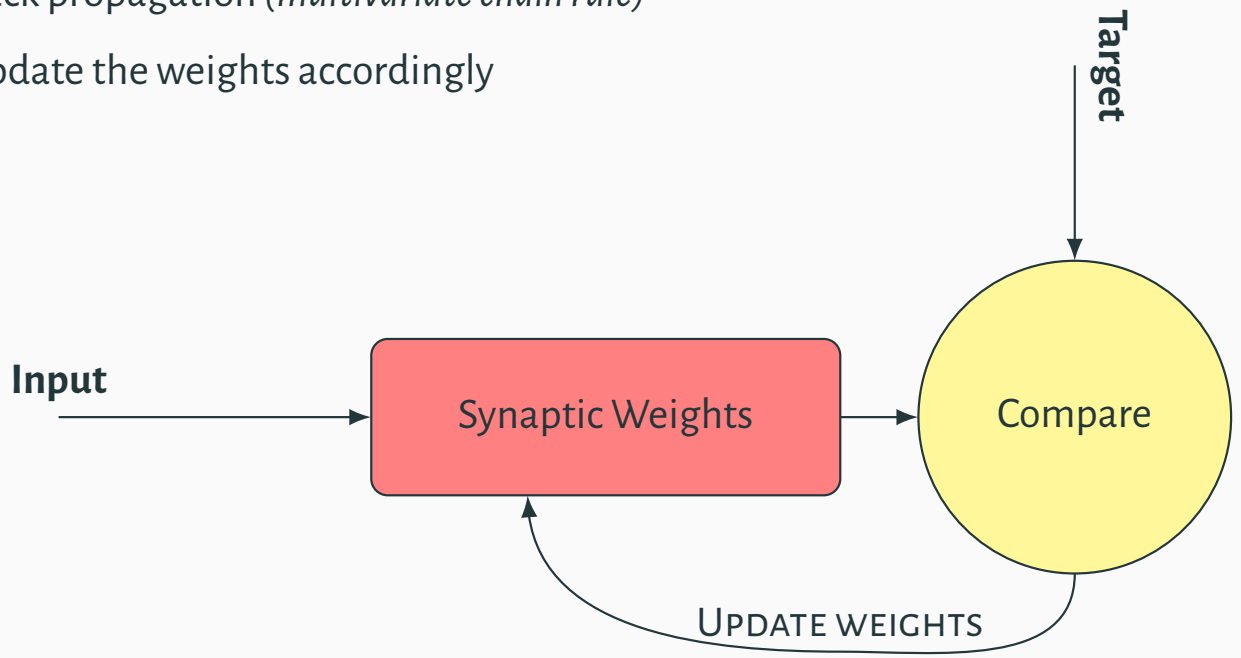
- ⋆ Supervised learning rule for binary classification;
- ⋆ Guarantees convergence if data is linearly separable;
- ⋆ Limited to linear decision boundaries.

Rosenblatt, F. (1958). *The perceptron: A probabilistic model for information storage and organization in the brain*. **Psychological Review.** 65(6), 386–408. doi 10.1037/h0042519

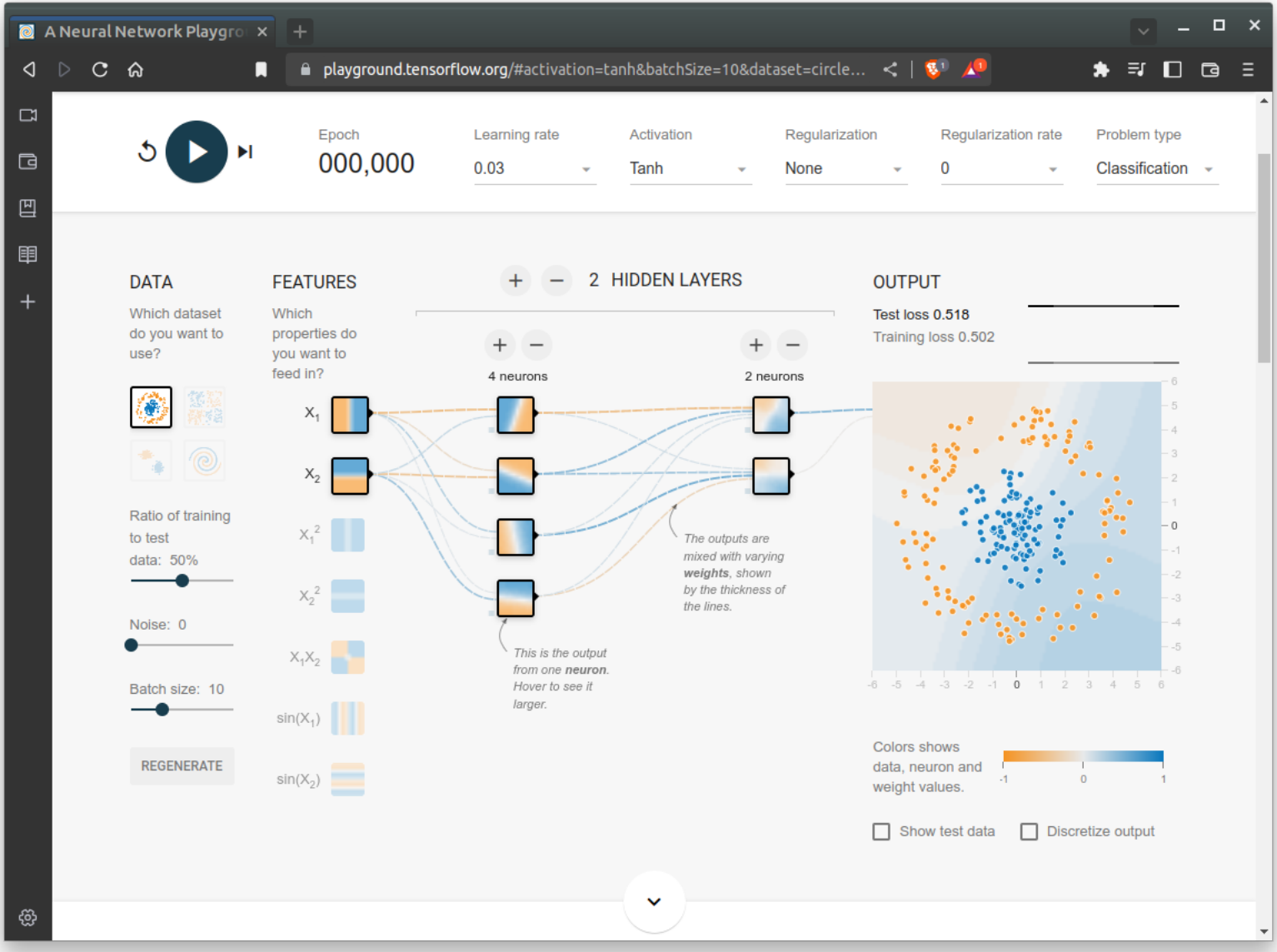
Key Differences

Aspect	Hebbian Learning Rule	Rosenblatt Learning Rule
Type of Learning	Unsupervised	Supervised
Primary Goal	Strengthen correlated activations	Minimize classification errors
Weight Update	Based on correlation of inputs	Based on prediction errors
Applications	Associative memory and pattern recognition	Training perceptrons and related linear classifiers.
Biological Basis	Inspired by neuroscience	Abstract, focused on AI

- ✓ Design a structure
- ✓ Specify a loss function to minimize
- ✓ Optimize using gradient descent
  1. Feedforward propagation *(matrix multiplication and point-wise activation)*
  2. Back propagation *(multivariate chain rule)*
  3. Update the weights accordingly



Tinker with a neural network



https://playground.tensorflow.org/

Regression vs Classification

Regression

- Predicts continuous values
- Output: Real numbers
- Examples:
  - House prices
  - Temperature
  - Stock prices
  - Age prediction

$y \in \mathbb{R}$ (*e.g.*, \$250,000)

Classification

- Predicts discrete categories
- Output: Class labels
- Examples:
  - Spam/Not spam
  - Disease diagnosis
  - Image recognition
  - Sentiment analysis

$y \in \{C_1, C_2, \ldots, C_n\}$ (*e.g.*, {Yes, No})

Loss vs Cost Function

y denotes true labels, $\hat{y}$ denotes predictions and $\varepsilon = y - \hat{y}$ is the prediction error.

Loss Function

- Error on a *single* training example

$$\mathcal{L}_i^{(j)} = \frac{1}{2}\left(y_i^{(j)} - \hat{y}_i^{(j)}\right)^2 = \frac{1}{2}\left(\varepsilon_i^{(j)}\right)^2,$$

where $y_i^{(j)}$ is the i-th label of the j-th sample.

Cost Function

- Aggregate error across the *entire* dataset

$$\mathcal{J} = \frac{1}{n}\sum_j \sum_i \mathcal{L}_i^{(j)}$$

Loss $\longrightarrow$ one example | Cost $\longrightarrow$ all examples

Loss Functions and Corresponding Derivatives (1/2)

Mean Squared Error *(MSE/L2)*

commonly used for regression

$$\mathcal{L} = \frac{1}{2} \sum_i \left(y_i - \hat{y}_i\right)^2$$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = -\left(y_i - \hat{y}_i\right)$$

Mean Absolute Error *(MAE/L1)*

robust regression alternative

$$\mathcal{L} = \sum_i \left|y_i - \hat{y}_i\right|$$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = -\text{sign}(y_i - \hat{y}_i)$$

Loss Functions and Corresponding Derivatives (2/2)

Binary Cross-Entropy *(Log Loss)*

for binary classification

$$\mathcal{L} = -y \log(\hat{y}) - (1-y) \log\left(1-\hat{y}\right)$$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right)$$

Categorical Cross-Entropy

for multi-class classification

$$\mathcal{L} = -\sum_i y_i \log(\hat{y}_i) = -\log\left(\prod_i \hat{y}_i^{y_i}\right)$$

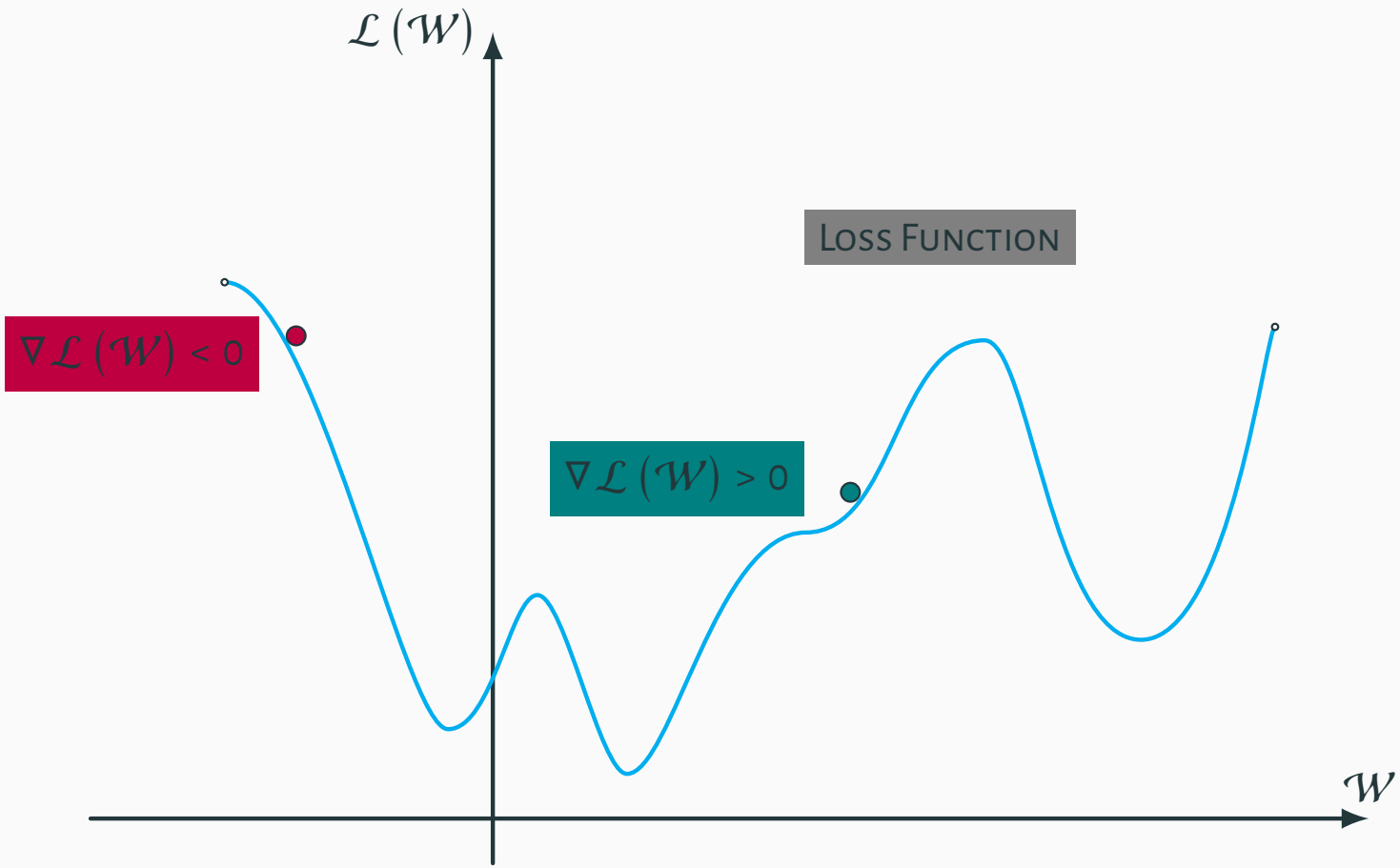
$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i}$$

Lift. Learn. Grow.



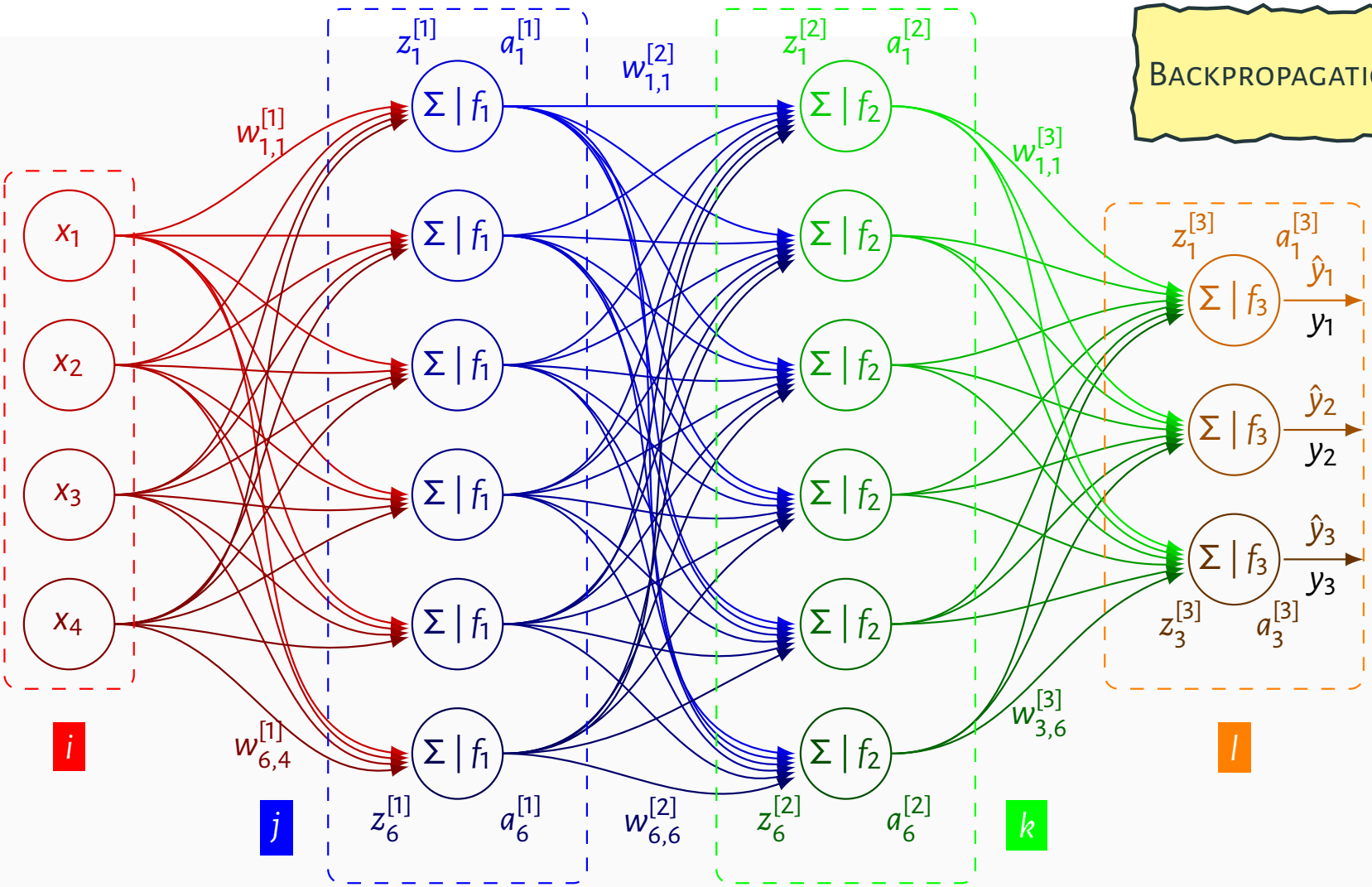
Visual created using Gemini 3

Gradient Descent



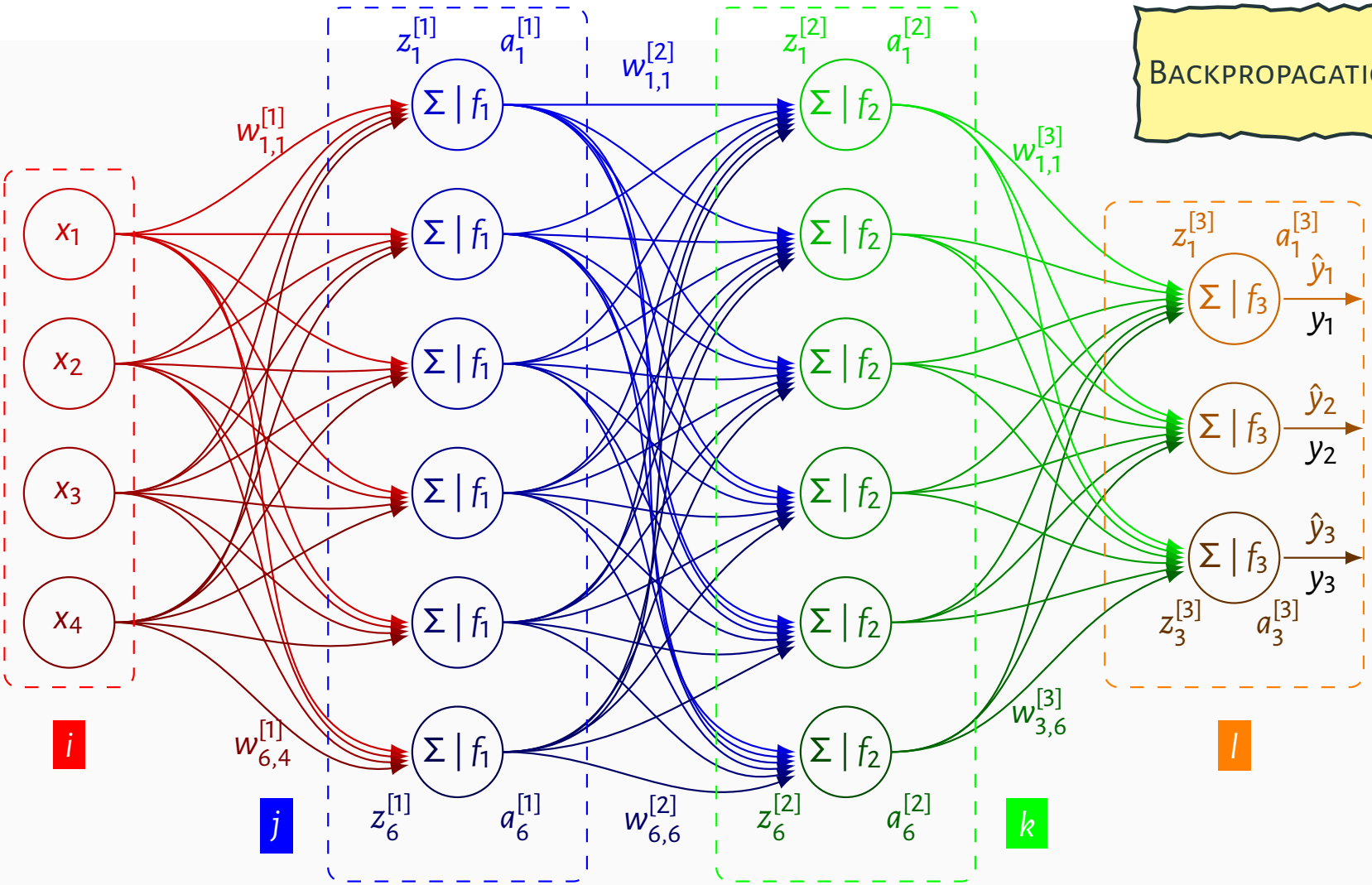
$$\underbrace{\mathcal{W}}_{\text{New weight}} \triangleq \underbrace{\mathcal{W}}_{\text{Old weight}} - \overbrace{\eta}^{\text{Weighting factor}} \underbrace{\nabla\mathcal{L}(\mathcal{W})}_{\text{Gradient term is steepest ascent}}$$

Backpropagation



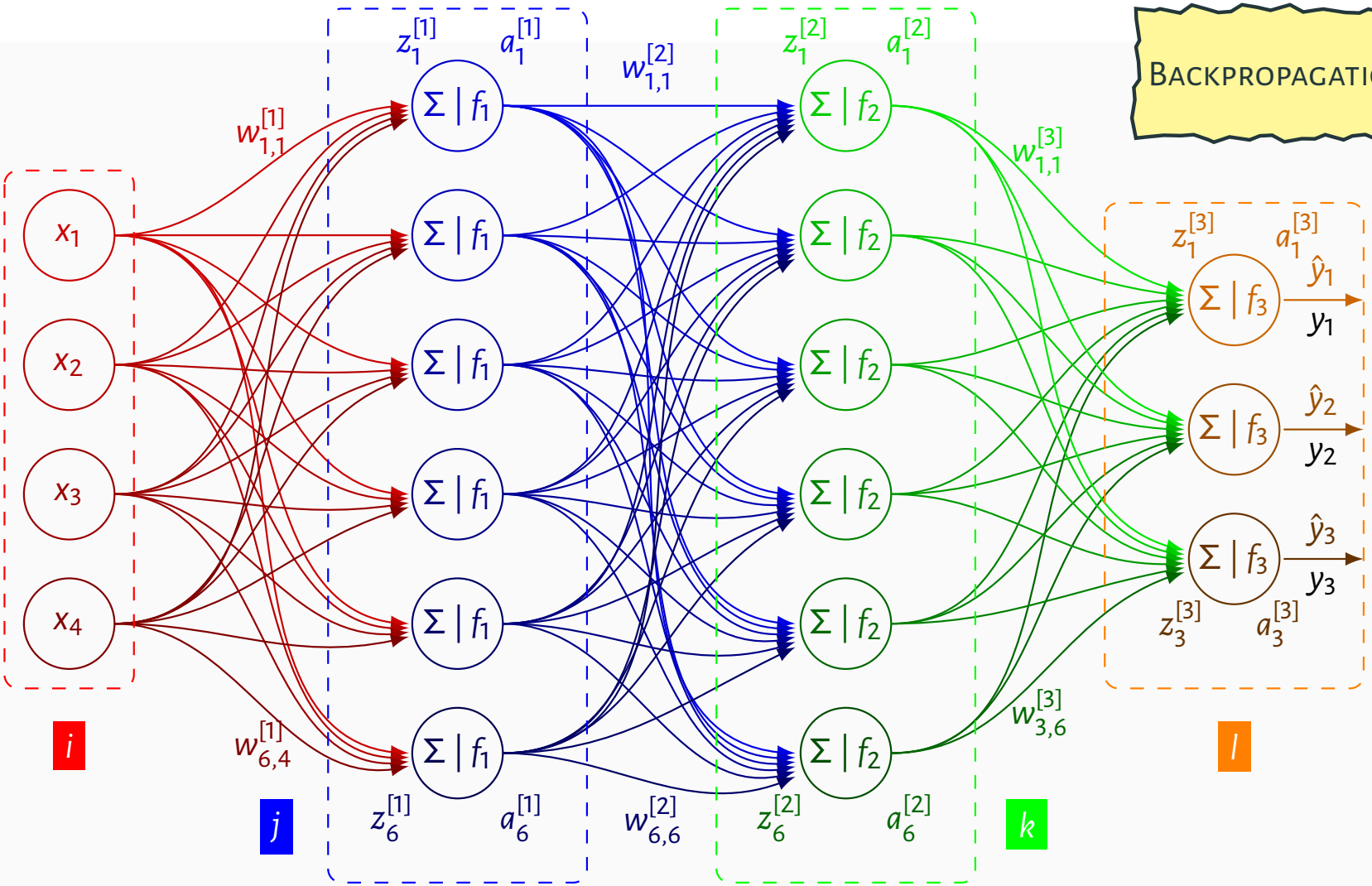
$$\mathcal{L}(W) = \frac{1}{2}\sum_l (y_l - \hat{y}_l)^2 \implies \frac{\partial \mathcal{L}}{\partial w^{[3]}_{l,k}} = -\underbrace{(y_l - \hat{y}_l)\, \dot{f}_3\left(z^{[3]}_l\right)}_{\delta^{[3]}_l} a^{[2]}_k$$

Backpropagation

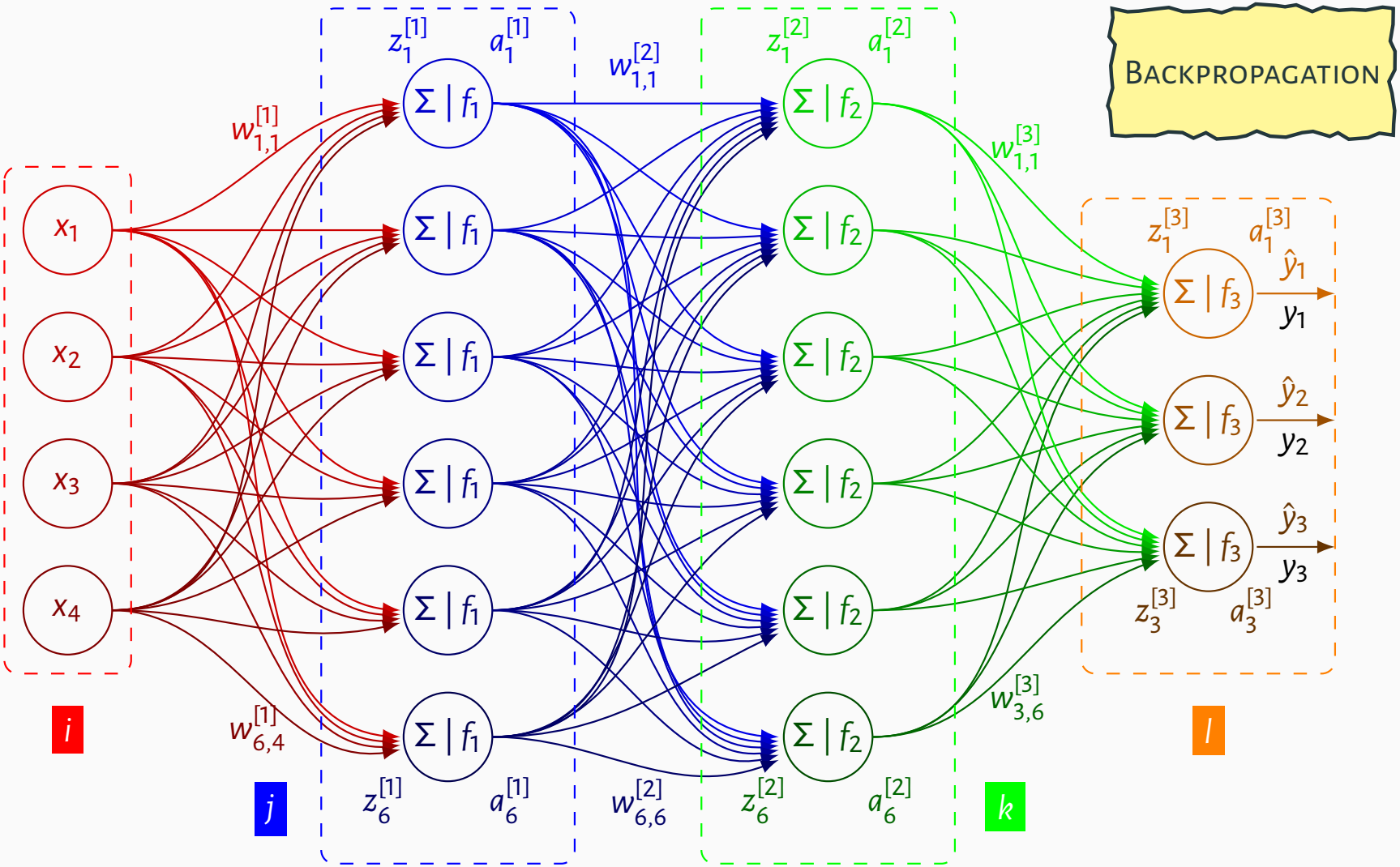


$$\frac{\partial \mathcal{L}}{\partial w^{[2]}_{k,j}} = -\sum_l \left(y_l - \hat{y}_l\right) \dot{f}_3\left(z^{[3]}_l\right) w^{[3]}_{l,k} \dot{f}_2\left(z^{[2]}_k\right) a^{[1]}_j = -\underbrace{\sum_l \delta^{[3]}_l w^{[3]}_{l,k} \dot{f}_2\left(z^{[2]}_k\right)}_{\delta^{[2]}_k} a^{[1]}_j$$

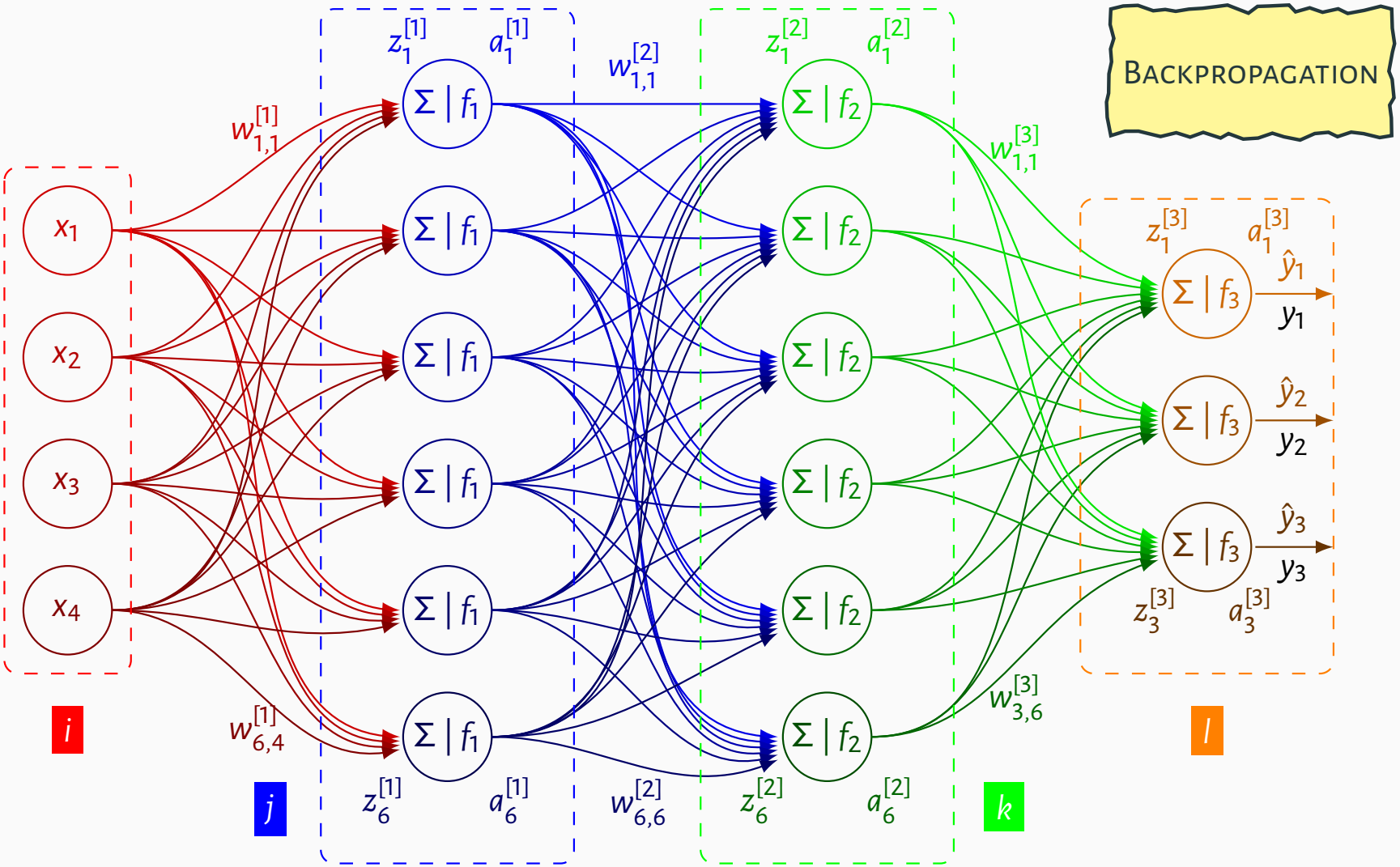
Backpropagation



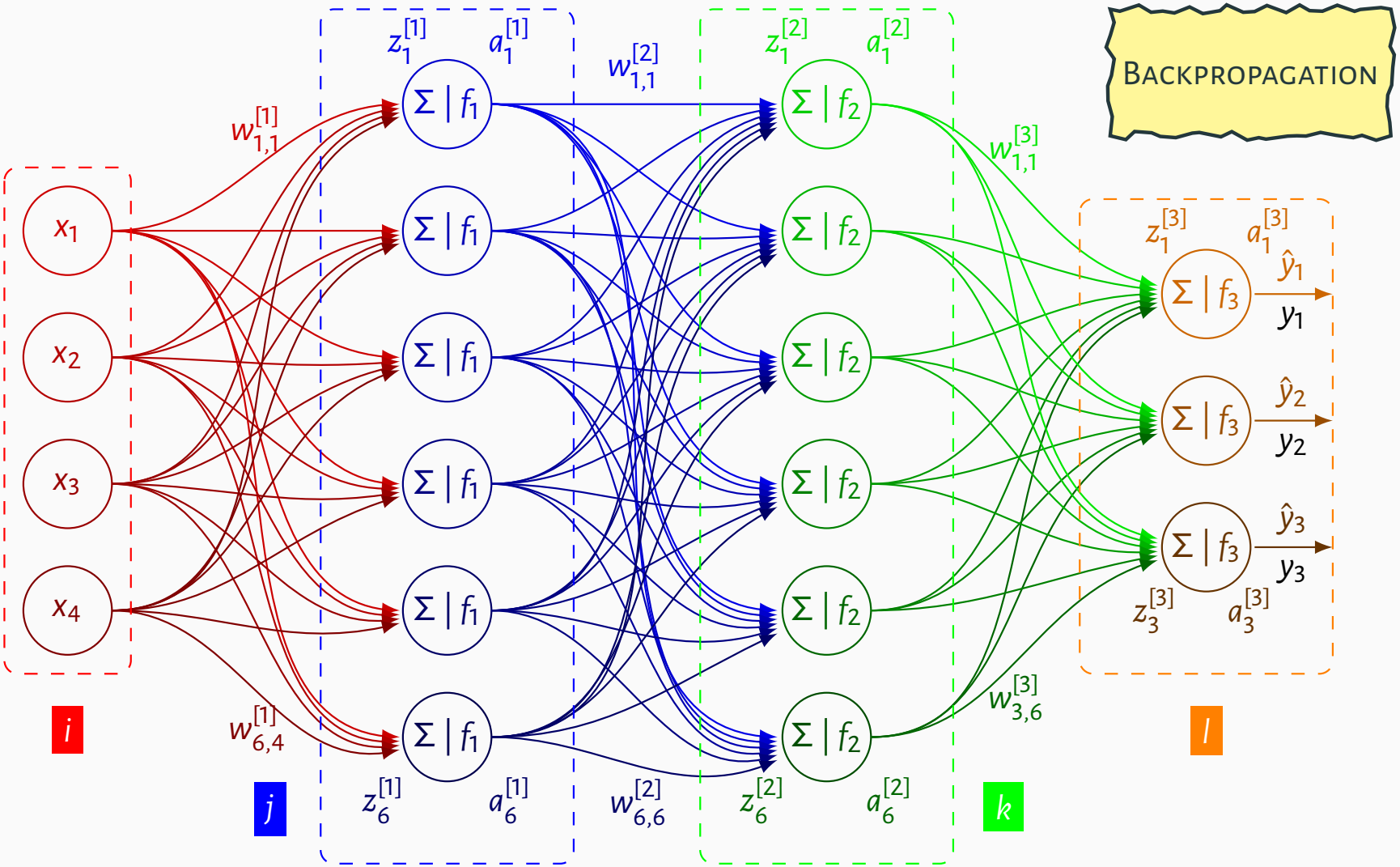
$$\frac{\partial \mathcal{L}}{\partial w^{[1]}_{j,i}} = -\sum_l (y_l - \hat{y}_l)\, \dot{f}_3\left(z^{[3]}_l\right) \sum_k w^{[3]}_{l,k} \dot{f}_2\left(z^{[2]}_k\right) w^{[2]}_{k,j} \dot{f}_1\left(z^{[1]}_j\right) x_i = -\underbrace{\sum_k \delta^{[2]}_k w^{[2]}_{k,j} \dot{f}_1\left(z^{[1]}_j\right)}_{\delta^{[1]}_j} x_i$$



$$\delta^{[3]}_l = (y_l - \hat{y}_l) \times \dot{f}_3\left(z^{[3]}_l\right) \implies \Delta w^{[3]}_{l,k} = \eta \delta^{[3]}_l \times a^{[2]}_k$$



$$\delta^{[2]}_k = \left(\delta^{[3]}_1 w^{[2]}_{1,k} + \delta^{[3]}_2 w^{[2]}_{2,k} + \delta^{[3]}_3 w^{[2]}_{3,k}\right) \times \dot{f}_2\left(z^{[2]}_k\right) \Longrightarrow \Delta w^{[2]}_{k,j} = \eta \delta^{[2]}_k \times a^{[1]}_j$$



$$\delta^{[1]}_j = \left(\delta^{[2]}_1 w^{[1]}_{1,j} + \cdots + \delta^{[2]}_6 w^{[1]}_{6,j}\right) \times \dot{f}_2\left(z^{[2]}_j\right) \Longrightarrow \Delta w^{[1]}_{j,i} = \eta \delta^{[1]}_j \times x_i$$

Multivariate chain rule

Output layer $\longrightarrow$ hidden layer #2

$$\frac{\partial \hat{y}_l}{\partial w^{[3]}_{l,k}} = \underbrace{\frac{\partial \hat{y}_l}{\partial z^{[3]}_l}}_{\dot{f}_3\left(z^{[3]}_l\right)} \underbrace{\frac{\partial z^{[3]}_l}{\partial w^{[3]}_{l,k}}}_{a^{[2]}_k}$$

Output layer $\longrightarrow$ hidden layer #1

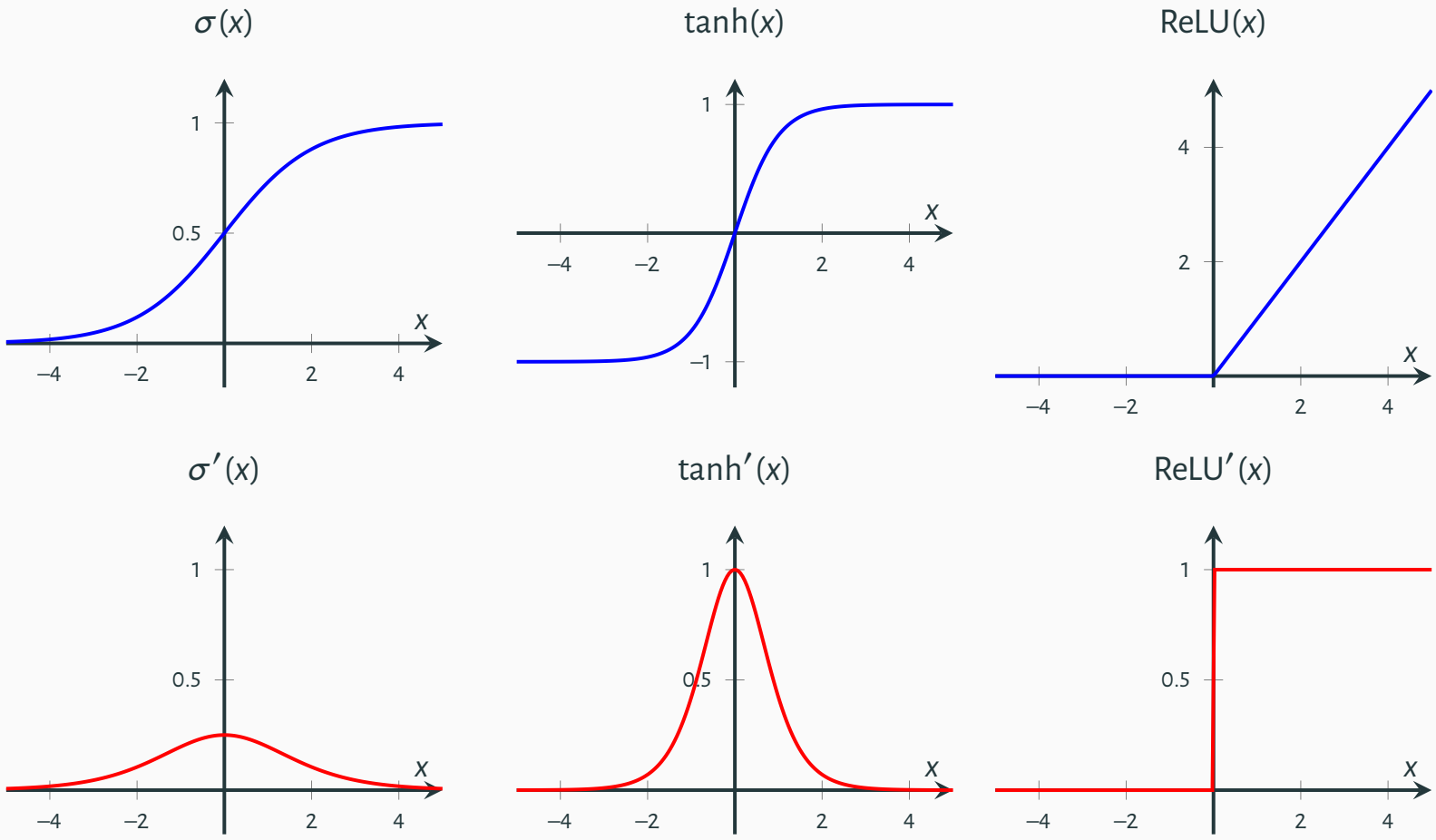
$$\frac{\partial \hat{y}_l}{\partial w^{[2]}_{k,j}} = \underbrace{\frac{\partial \hat{y}_l}{\partial z^{[3]}_l}}_{\dot{f}_3\left(z^{[3]}_l\right)} \underbrace{\frac{\partial z^{[3]}_l}{\partial a^{[2]}_k}}_{w^{[3]}_{l,k}} \underbrace{\frac{\partial a^{[2]}_k}{\partial z^{[2]}_k}}_{\dot{f}_2\left(z^{[2]}_k\right)} \underbrace{\frac{\partial z^{[2]}_k}{\partial w^{[2]}_{k,j}}}_{a^{[1]}_j}$$

Output layer $\longrightarrow$ input layer

$$\frac{\partial \hat{y}_l}{\partial w^{[1]}_{j,i}} = \underbrace{\frac{\partial \hat{y}_l}{\partial z^{[3]}_l}}_{\dot{f}_3\left(z^{[3]}_l\right)} \underbrace{\frac{\partial z^{[3]}_l}{\partial a^{[2]}_k}}_{w^{[3]}_{l,k}} \underbrace{\frac{\partial a^{[2]}_k}{\partial z^{[2]}_k}}_{\dot{f}_2\left(z^{[2]}_k\right)} \underbrace{\frac{\partial z^{[2]}_k}{\partial a^{[1]}_j}}_{w^{[2]}_{k,j}} \underbrace{\frac{\partial a^{[1]}_j}{\partial z^{[1]}_j}}_{\dot{f}_1\left(z^{[1]}_j\right)} \underbrace{\frac{\partial z^{[1]}_j}{\partial w^{[1]}_{j,i}}}_{x_i}$$

Activation Functions and Their Derivatives (1/2)

Each function (blue) and its derivative (red) are shown.



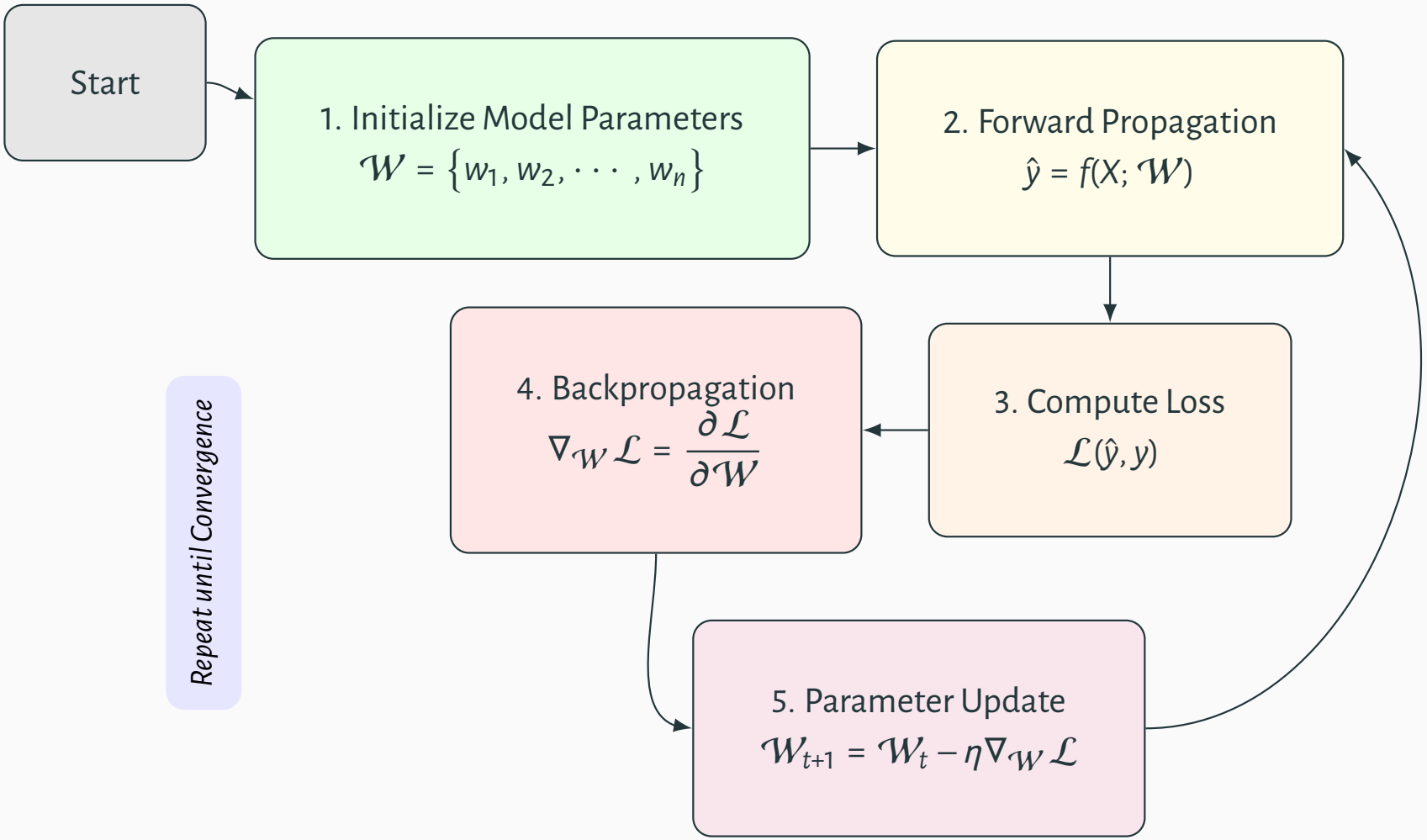
σ and tanh saturate for large $|x|$, while ReLU avoids this but is non-smooth at $x = 0$.

Activation Functions and Their Derivatives (2/2)

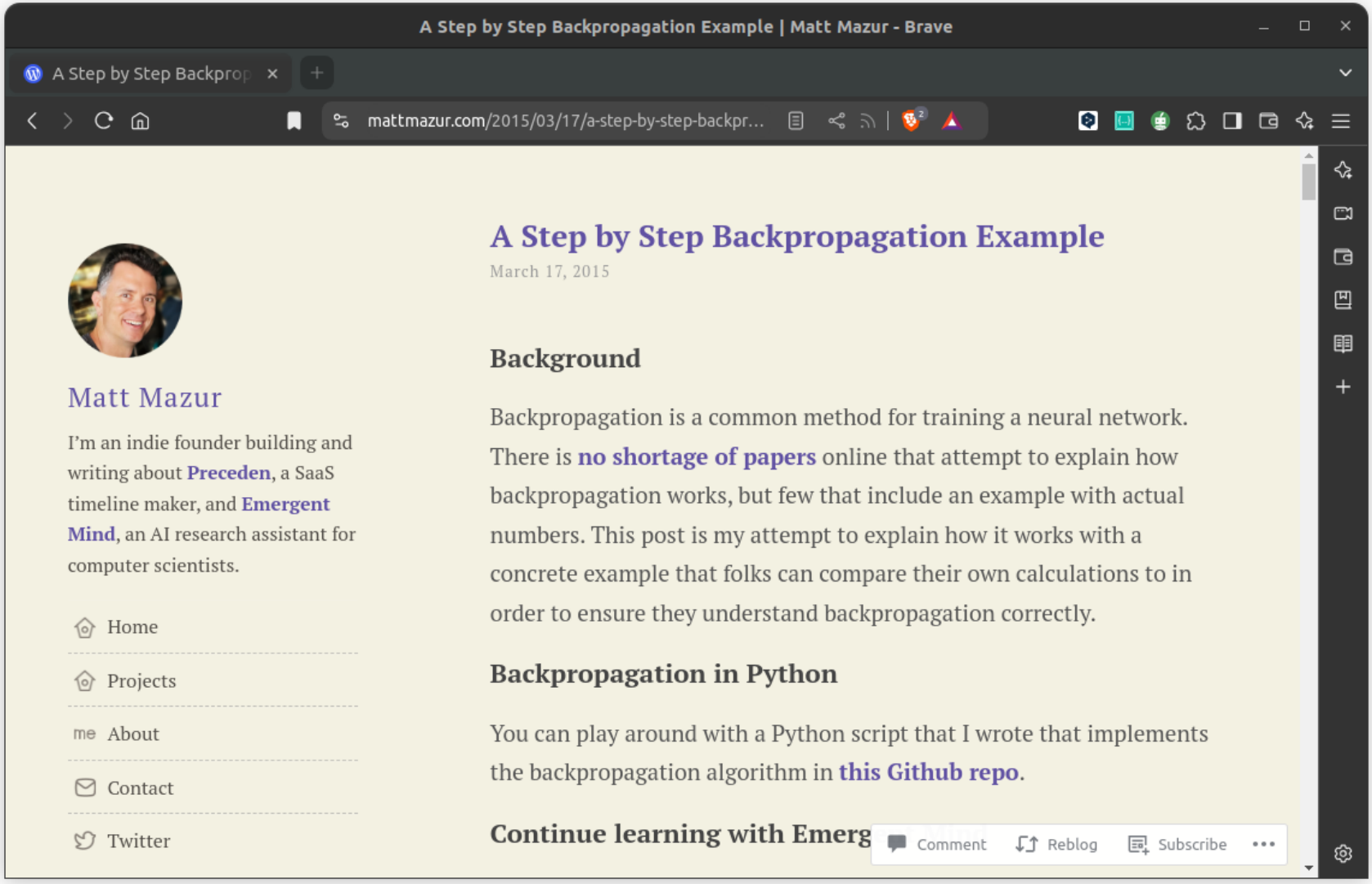
Function	Formula	Derivative	Notes
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma'(x) = \sigma(x)\big(1-\sigma(x)\big)$	Smooth, bounded
Tanh	$\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	$\tanh'(x) = 1-\tanh^{2}(x)$	Zero-centered
ReLU	$\mathrm{ReLU}(x) = \max(0,x)$	$\mathrm{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$	Sparse gradients

Activation functions introduce non-linearity, enabling neural networks to approximate complex mappings.

Neural Network Training



Backpropagation by hand



https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

List of available optimizers (1/3)

These are a few typical optimizers for artificial neural networks:

$$\Delta\hat{\mathcal{W}} \triangleq \mathcal{F}\left(\nabla \underbrace{\mathcal{L}\left(\hat{\mathcal{W}}\right)}_{\text{Loss Function}}\right) \equiv \hat{\mathcal{W}} \triangleq \hat{\mathcal{W}} + \mathcal{F}\left(\nabla\mathcal{L}\left(\hat{\mathcal{W}}\right)\right) \qquad \nabla\mathcal{L}\left(\hat{\mathcal{W}}\right) = \begin{bmatrix} \frac{\partial\mathcal{L}}{\partial\hat{w}_0} \\ \vdots \\ \frac{\partial\mathcal{L}}{\partial\hat{w}_n} \end{bmatrix}$$

Stochastic Gradient Descent (SGD)

$$\hat{\mathcal{W}} \triangleq \hat{\mathcal{W}} - \eta\nabla\mathcal{L}\left(\hat{\mathcal{W}}\right)$$

Mini-batch Gradient Descent

$$\hat{\mathcal{W}} \triangleq \hat{\mathcal{W}} - \frac{\eta}{m}\nabla\sum_{i=1}^{m}\mathcal{L}\left(\hat{\mathcal{W}}\right) \qquad \longleftarrow \text{ m denotes the size of the mini-batch}$$

List of available optimizers (2/3)

Momentum

$$\hat{\mathcal{W}} \triangleq \hat{\mathcal{W}} - \mathcal{V}, \quad \text{where} \quad \mathcal{V} \triangleq \alpha\mathcal{V} + \eta\nabla\mathcal{J}\left(\hat{\mathcal{W}}\right)$$

Nesterov Accelerated Gradient (NAG)

$$\hat{\mathcal{W}} \triangleq \hat{\mathcal{W}} - \mathcal{V} \quad \text{where} \quad \mathcal{V} \triangleq \alpha\mathcal{V} + \eta\nabla\mathcal{J}\left(\hat{\mathcal{W}} - \alpha\mathcal{V}\right)$$

AdaGrad

$$\hat{\mathcal{W}} \triangleq \hat{\mathcal{W}} - \frac{\eta}{\sqrt{\mathcal{G} + \epsilon}}\nabla\mathcal{J}\left(\hat{\mathcal{W}}\right) \quad \text{where} \quad \mathcal{G} \triangleq \mathcal{G} + \left(\nabla\mathcal{J}\left(\hat{\mathcal{W}}\right)\right)^2$$

RMSProp

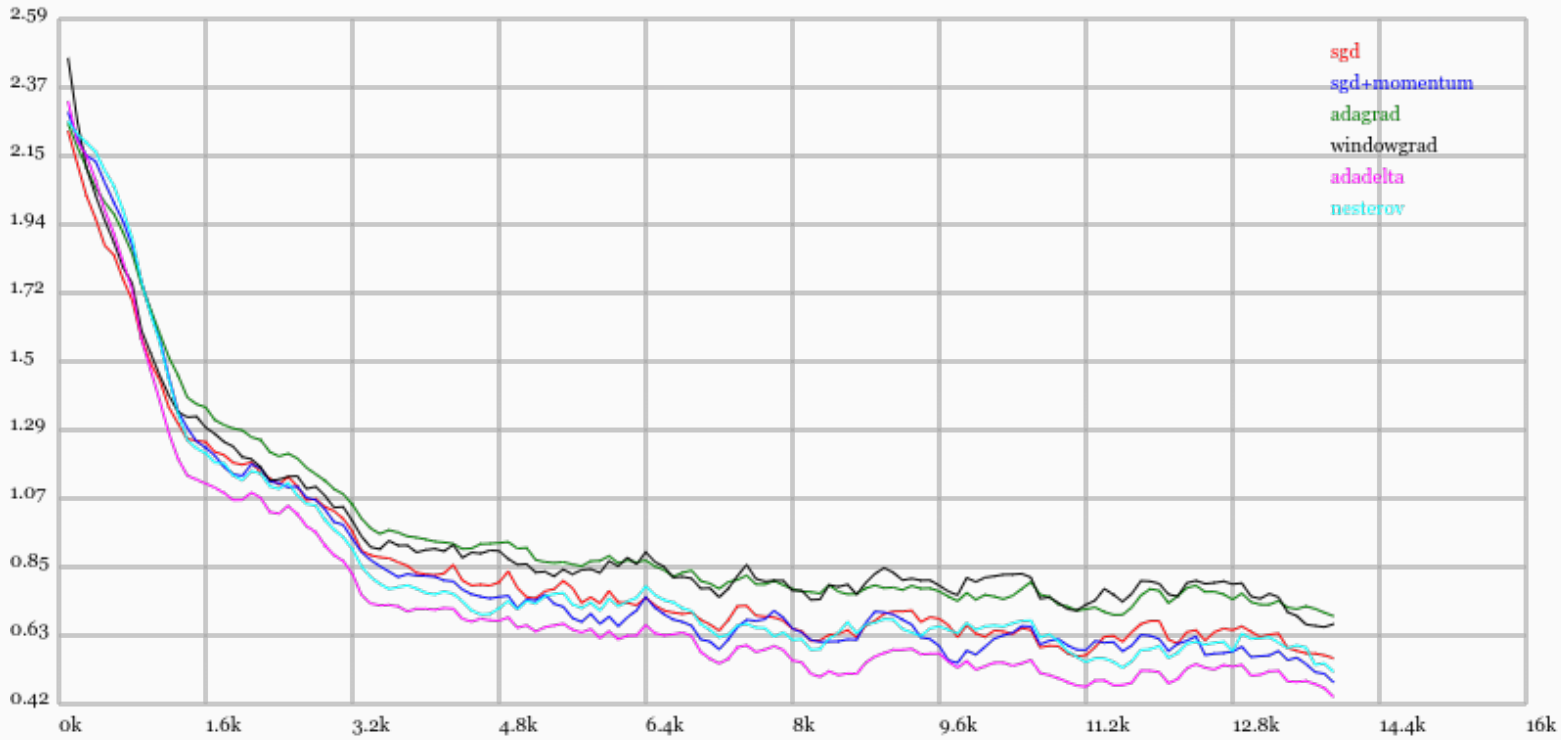
$$\hat{\mathcal{W}} \triangleq \hat{\mathcal{W}} - \frac{\eta}{\sqrt{\mathcal{G} + \epsilon}}\nabla\mathcal{J}\left(\hat{\mathcal{W}}\right) \quad \text{where} \quad \mathcal{G} \triangleq \mathcal{G} + (1-\beta)\left(\nabla\mathcal{J}\left(\hat{\mathcal{W}}\right)\right)^2$$

List of available optimizers (3/3)

Adam

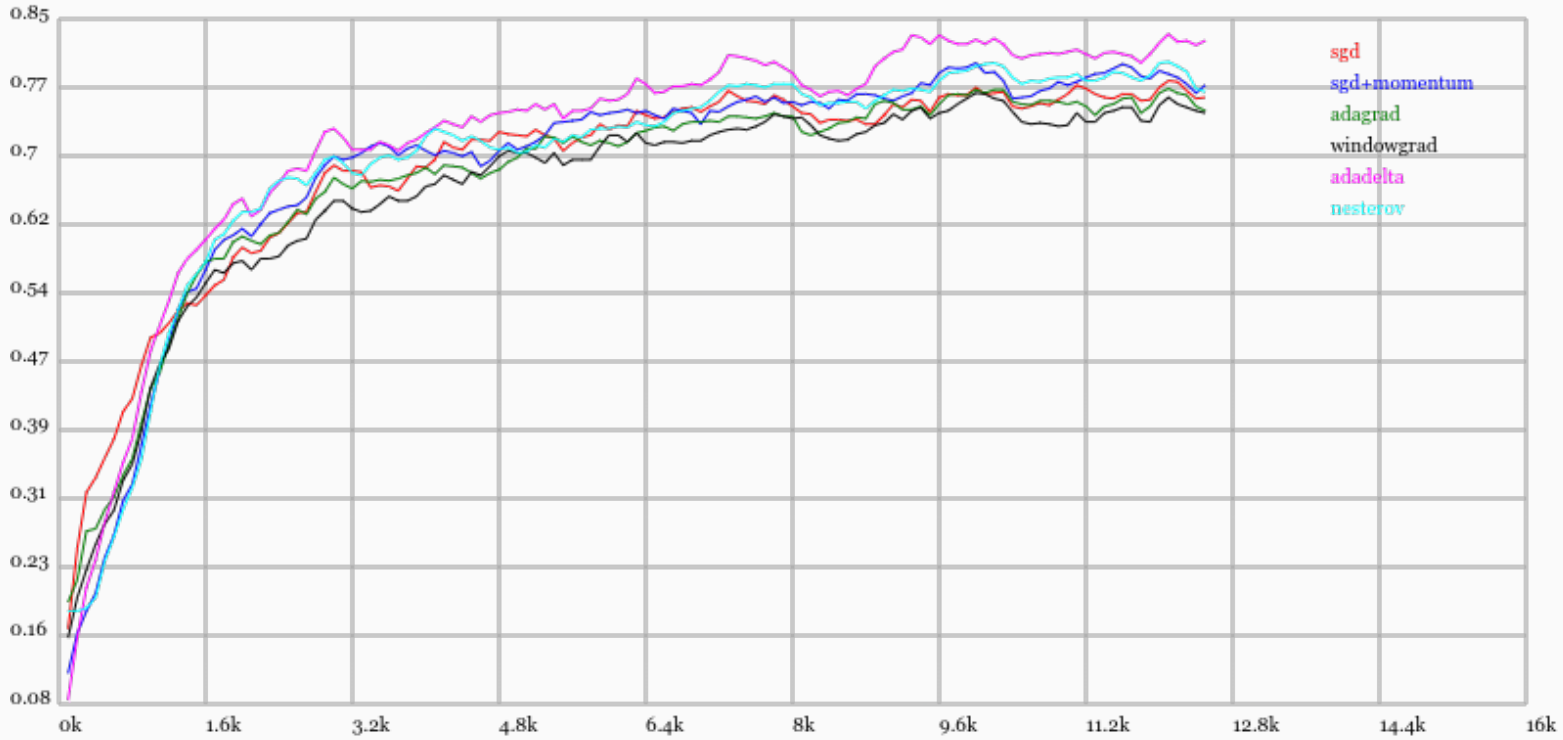
$$
\begin{aligned}
\mathcal{M} &\triangleq \beta_1 \mathcal{M} + (1-\beta_1) \nabla \mathcal{J}\left(\hat{\mathcal{W}}\right) && \longleftarrow \text{Estimate of first moment} \\
\mathcal{V} &\triangleq \beta_2 \mathcal{V} + (1-\beta_2) \left(\nabla \mathcal{J}\left(\hat{\mathcal{W}}\right)\right)^2 && \longleftarrow \text{Estimate of second moment} \\
\hat{\mathcal{M}} &= \frac{\mathcal{M}}{1-\beta_1^k} && \longleftarrow \text{@ every } k^{\text{th}} \text{ iteration} \\
\hat{\mathcal{V}} &= \frac{\mathcal{V}}{1-\beta_2^k} && \longleftarrow \text{@ every } k^{\text{th}} \text{ iteration} \\
\hat{\mathcal{W}} &\triangleq \hat{\mathcal{W}} - \frac{\eta}{\sqrt{\hat{\mathcal{V}}+\epsilon}} \hat{\mathcal{M}}
\end{aligned}
$$

Effect of optimizer on loss values



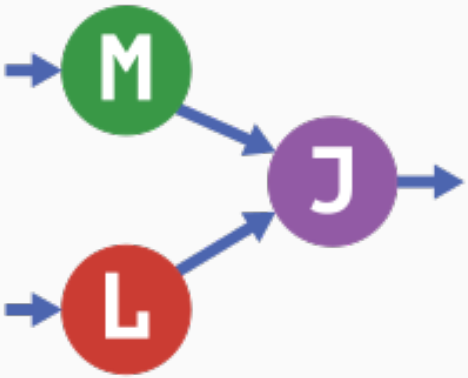
https://cs.stanford.edu/people/karpathy/convnetjs/demo/trainers.html

Effect of optimizer on testing accuracy values



https://cs.stanford.edu/people/karpathy/convnetjs/demo/trainers.html

Frameworks to be used



https://juliapackages.com/p/mlj



https://juliapackages.com/p/flux

ANN



The code is available @ github.com/a-mhamdi/jlai ⟶ *Codes* ⟶ *Julia* ⟶ *Part-1*

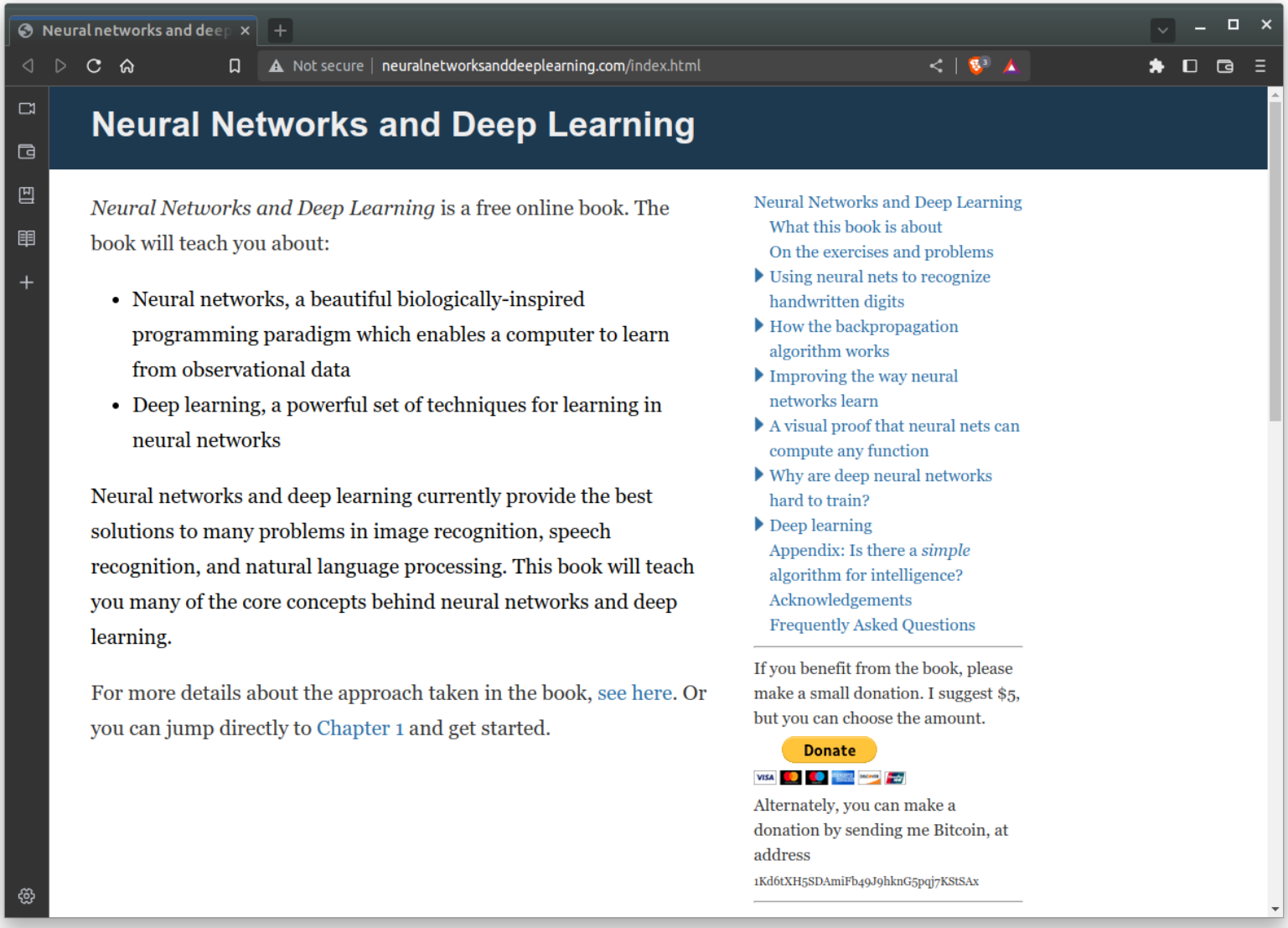
⟶ *Pluto* ⟶ *xor-gate.jl*



⟶ *Jupyter* ⟶ *xor-gate.ipynb*

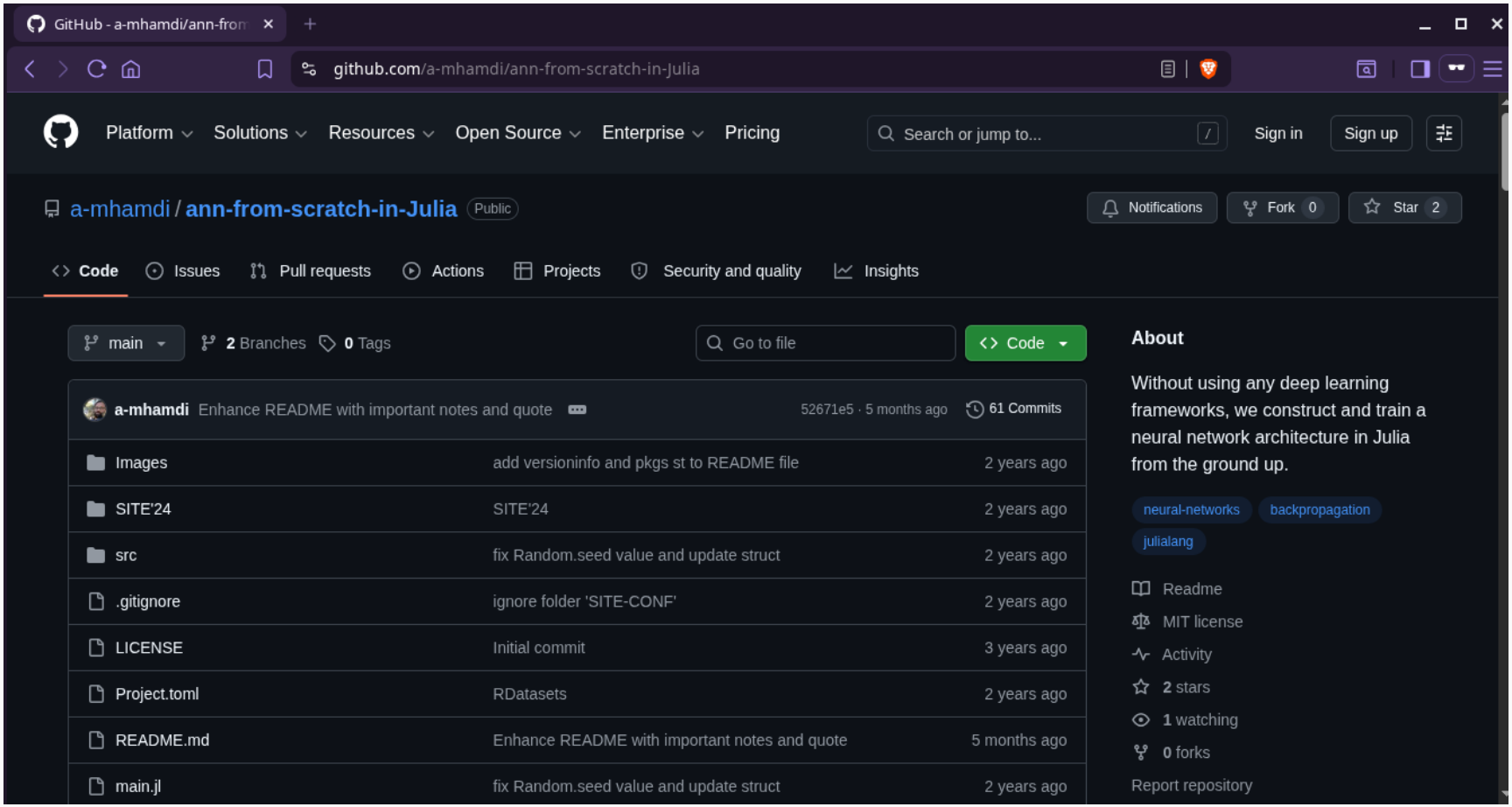


- This will be continued in the following e-book.



http://neuralnetworksanddeeplearning.com/

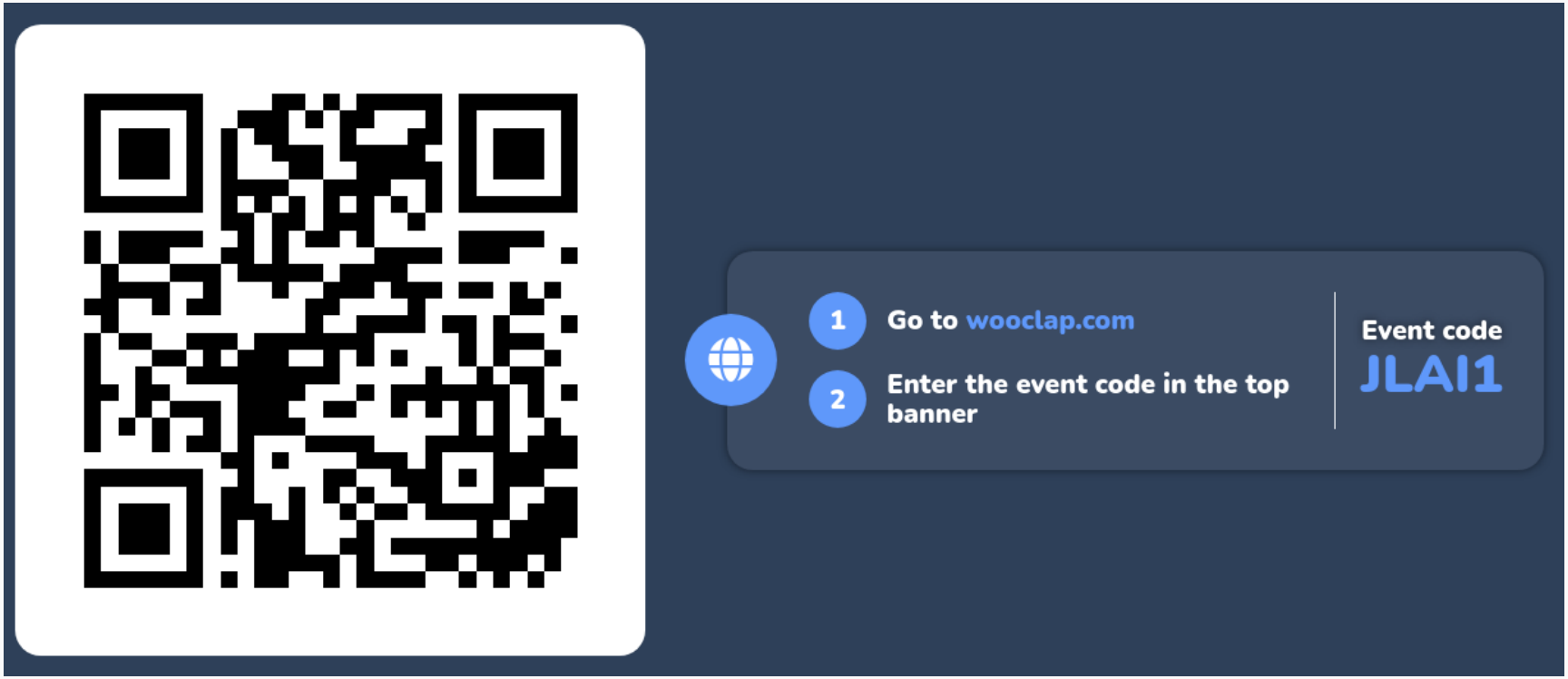
Neural Network from scratch



https://github.com/a-mhamdi/ann-from-scratch-in-Julia

Quizzes

Knowledge Check



https://app.wooclap.com/JLAI1

References

[Bel78] R. E. Bellman. ***An Introduction to Artificial Intelligence: Can Computers Think?*** Boyd & Fraser Publishing Company, Jan. 1, 1978 (cit. on p. 11).

[CMM85] E. Charniak, D. McDermott, and D. V. McDermott. ***Introduction to Artificial Intelligence.*** Addison-Wesley series in computer science and information processing. Addison-Wesley, 1985 (cit. on p. 12).

[Dad12] E. Dadios, ed. ***Fuzzy Logic - Controls, Concepts, Theories and Applications.*** IntechOpen, Mar. 28, 2012. 430 pp.

[ENM15] I. El Naqa and M. J. Murphy. **"What Is Machine Learning?"** In: *Machine Learning in Radiation Oncology: Theory and Applications*. Ed. by I. El Naqa, R. Li, and M. J. Murphy. Cham: Springer International Publishing, 2015, pp. 3–11. DOI: 10.1007/978-3-319-18305-3_1.

[Gac15] L. Gacôgne. ***Intelligence artificielle. Cours, exercices corrigés et projets.*** Ed. by Ellipses. Nov. 2015. 240 pp.

[GBC16] I. Goodfellow, J. Bengio, and A. Courville. ***Deep Learning.*** MIT Press Ltd, Nov. 18, 2016. 800 pp.

Further Reading (2/2)

[Hau89] J. Haugeland. ***Artificial Intelligence: The Very Idea.*** A Bradford book. MIT Press, 1989 (cit. on p. 11).

[JPM21] L. M. John Paul Mueller. ***Machine Learning For Dummies.*** Wiley John + Sons, Apr. 8, 2021. 464 pp.

[Kur92] R. Kurzweil. ***The Age of Intelligent Machines.*** Viking, 1992 (cit. on p. 13).

[LS93] G. F. Luger and W. A. Stubblefield. ***Artificial Intelligence: Structures and Strategies for Complex Problem Solving.*** Artificial intelligence. Benjamin/Cummings Publishing Company, 1993 (cit. on p. 14).

[Mit97] T. M. Mitchell. ***Machine Learning.*** McGraw-Hill International Editions. McGraw-Hill, 1997.

[RK91] E. Rich and K. Knight. ***Artificial Intelligence.*** Artificial Intelligence Series. McGraw-Hill, 1991 (cit. on p. 13).

[Rob14] A. Robinson. ***Construction Informatics.*** Ed. by H. S. of Construction Engineering. 2014.

[Sch90] R. J. Schalkoff. ***Artificial Intelligence: An Engineering Approach.*** McGraw-Hill Computer science series. McGraw-Hill, 1990 (cit. on p. 14).

[SNK12] T. Sai, D. Nakhaeinia, and B. Karasfi. **"Application of Fuzzy Logic in Mobile Robot Navigation".** In: *Fuzzy Logic - Controls, Concepts, Theories and Applications*. InTech, Mar. 2012. DOI: 10.5772/36358.

Further Reading (3/2)

[Win92] P. H. Winston. ***Artificial Intelligence.*** A-W Series in Computer Science. Addison-Wesley Publishing Company, 1992 (cit. on p. 12).

[Woj12] J. Wojtusiak. **"Machine Learning".** In: *Encyclopedia of the Sciences of Learning*. Springer US, 2012, pp. 2082–2083. DOI: 10.1007/978-1-4419-1428-6_1927.